

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hệ thống chống trộm xe ứng dụng IoT

THIỀU VĂN DŨNG

Dung.tv215547@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: PGS. TS. Lã Thế Vinh

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hệ thống chống trộm xe ứng dụng IoT

THIỀU VĂN DŨNG

Dung.tv215547@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: PGS. TS. Lã Thế Vinh

Chữ kí GVHD

Khoa:

Kỹ thuật máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp, em đã nhận được rất nhiều sự giúp đỡ, động viên và hỗ trợ quý báu từ thầy cô, gia đình và bạn bè. Đó chính là nguồn động lực to lớn giúp em có thể hoàn thành tốt đề tài của mình.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới thầy PGS. TS. Lê Thế Vinh. Trong suốt thời gian thực hiện đồ án, thầy đã luôn tận tình chỉ bảo, định hướng, góp ý chi tiết và hỗ trợ em trong từng giai đoạn của quá trình nghiên cứu và triển khai.

Em xin gửi lời cảm ơn sâu sắc tới gia đình, đặc biệt là bố mẹ – những người luôn ở bên cạnh, động viên, khích lệ và tạo mọi điều kiện tốt nhất cho em trong suốt quá trình học tập. Sự yêu thương, hy sinh thầm lặng và niềm tin mà gia đình dành cho em chính là động lực lớn nhất giúp em vượt qua những khó khăn trong học tập cũng như trong quá trình thực hiện đồ án.

Bên cạnh đó, em cũng xin cảm ơn những người bạn thân thiết đã luôn đồng hành, hỗ trợ và chia sẻ những khó khăn trong học tập cũng như trong cuộc sống. Sự giúp đỡ và tinh thần đoàn kết của các bạn đã góp phần giúp em hoàn thành tốt đồ án này.

Trong quá trình thực hiện đề tài, mặc dù em đã cố gắng hết sức nhưng không thể tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của các thầy cô để có thể hoàn thiện hơn trong tương lai.

Cuối cùng, em xin gửi lời cảm ơn chân thành nhất tới tất cả những người đã giúp đỡ em trong suốt thời gian qua.

Em xin chân thành cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh số lượng phương tiện giao thông cá nhân ngày càng gia tăng, tình trạng trộm cắp xe máy vẫn đang là một vấn đề đáng quan ngại tại nhiều khu vực, đặc biệt là các đô thị lớn và cao điểm tại thời điểm diễn ra các giải bóng đá lớn như World Cup 2026. Mặc dù hiện nay đã có nhiều giải pháp được áp dụng như khóa cơ học, khóa điện tử, hệ thống báo động chống trộm và thiết bị định vị GPS, các phương pháp này vẫn tồn tại một số hạn chế như khả năng giám sát thời gian thực còn yếu, thiếu tính chủ động trong cảnh báo, hoặc chưa tích hợp đầy đủ giữa chức năng phát hiện, cảnh báo và định vị. Bên cạnh đó, các nghiên cứu gần đây về hệ thống chống trộm dựa trên Internet of Things (IoT) đã mở ra hướng tiếp cận mới khi kết hợp các cảm biến, vi điều khiển và mạng truyền thông để giám sát và phản ứng theo thời gian thực, tuy nhiên vẫn gặp hạn chế về chi phí, độ phức tạp triển khai và độ ổn định của kết nối.

Trong đồ án này, em lựa chọn hướng tiếp cận xây dựng hệ thống chống trộm xe ứng dụng IoT nhằm tận dụng ưu điểm về khả năng kết nối, mở rộng linh hoạt và chi phí thấp của các nền tảng phần cứng phổ biến như ESP32 và các module cảm biến. Hướng tiếp cận này cho phép tích hợp đồng thời các chức năng phát hiện rung động bất thường, xác định vị trí phương tiện thông qua GPS và truyền cảnh báo đến người dùng thông qua mạng GSM

Giải pháp được xây dựng bao gồm một thiết bị gắn trên xe tích hợp vi điều khiển, cảm biến chuyển động/rung, module định vị và module truyền thông, kết hợp với giao diện giám sát trên thiết bị di động. Khi phát hiện hành vi xâm nhập hoặc di chuyển bất thường, hệ thống sẽ tự động gửi cảnh báo và cung cấp vị trí hiện tại của phương tiện theo thời gian thực.

Đóng góp chính của đồ án là xây dựng được một hệ thống chống trộm xe ứng dụng IoT hoạt động ổn định theo thời gian thực, có khả năng cảnh báo sớm và theo dõi phương tiện từ xa, đồng thời chứng minh tính khả thi của việc áp dụng IoT trong các hệ thống an ninh chi phí thấp. Kết quả đạt được là một hệ thống có thể hoạt động thực tế và có tiềm năng mở rộng trong tương lai.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

DANH MỤC HÌNH VẼ.....	v
DANH MỤC BẢNG BIỂU	vi
DANH MỤC TỪ VIẾT TẮT	vii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.2 Tổng quan chức năng	5
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã Quản lý thiết bị cá nhân	8
2.2.3 Biểu đồ use case phân rã Xem trạng thái thiết bị	9
2.2.4 Biểu đồ use case phân rã Gửi dữ liệu cảm biến	10
2.2.5 Biểu đồ use case phân rã Quan lý tài khoản cá nhân	11
2.2.6 Quy trình nghiệp vụ Kích hoạt thiết bị	12
2.2.7 Quy trình nghiệp vụ Giám sát và cảnh báo trộm.....	13
2.3 Đặc tả chức năng	14
2.3.1 Đặc tả use case Đăng nhập.....	14
2.3.2 Đặc tả use case Kích hoạt thiết bị	15
2.3.3 Đặc tả use case Xem trạng thái thiết bị.....	16
2.3.4 Đặc tả use case Gửi dữ liệu cảm biến	17

2.3.5 Đặc tả use case Phát hiện nguy cơ trộm.....	18
2.3.6 Đặc tả use case Nhận cảnh báo trộm	19
2.4 Yêu cầu phi chức năng	20
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	22
3.1 Kiến trúc công nghệ tổng thể	22
3.2 Công nghệ xây dựng khối máy chủ	22
3.2.1 NestJS.....	23
3.2.2 PostgreSQL	23
3.3 Công nghệ xây dựng ứng dụng di động	24
3.3.1 Flutter	24
3.3.2 Dart	25
3.4 Công nghệ xây dựng khối thiết bị IoT	26
3.4.1 ESP32.....	26
3.4.2 Module A7680C	27
3.4.3 Module GPS NEO-6M	28
3.4.4 Tổng kết khối thiết bị IoT	29
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ VÀ ĐÁNH GIÁ HỆ THỐNG.....	30
4.1 Thiết kế kiến trúc.....	30
4.1.1 Lựa chọn kiến trúc phần mềm	30
4.1.2 Thiết kế tổng quan.....	31
4.1.3 Thiết kế tổng quan hệ thống	31
4.1.4 Thiết kế chi tiết gói	33
4.2 Thiết kế chi tiết.....	35
4.2.1 Thiết kế giao diện	35
4.2.2 Thiết kế lớp	40
4.2.3 Thiết kế cơ sở dữ liệu	46

4.3 Xây dựng ứng dụng.....	49
4.3.1 Thư viện và công cụ sử dụng.....	49
4.3.2 Kết quả đạt được	50
4.3.3 Minh họa các chức năng chính	52
4.4 Kiểm thử.....	59
4.4.1 Kỹ thuật kiểm thử.....	59
4.4.2 Kiểm thử chức năng kích hoạt thiết bị	59
4.4.3 Kiểm thử chức năng phát hiện và gửi cảnh báo	60
4.4.4 Kiểm thử chức năng điều khiển còi	62
4.4.5 Công cụ kiểm thử.....	62
4.4.6 Tổng kết kiểm thử	63
4.5 Triển khai	63
4.5.1 Mô hình triển khai.....	63
4.5.2 Quy trình triển khai	64
4.5.3 Kết quả triển khai thử nghiệm	64
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	67
5.1 Giới thiệu chương	67
5.2 Giải thuật phát hiện xe di chuyển bất thường	67
5.2.1 Bài toán và khó khăn	67
5.2.2 Giải pháp đề xuất	67
5.2.3 Kết quả đạt được và đánh giá.....	69
5.3 Cơ chế cảnh báo đa kênh	69
5.3.1 Bài toán và khó khăn	69
5.3.2 Giải pháp đề xuất	70
5.3.3 Kết quả đạt được và đánh giá.....	70

5.4 Tái tạo hành trình và tính quãng đường từ dữ liệu GPS	71
5.4.1 Bài toán và khó khăn	71
5.4.2 Giải pháp đề xuất	71
5.4.3 Kết quả đạt được và đánh giá.....	72
5.5 Quản lý vòng đời và quyền điều khiển thiết bị IoT.....	72
5.5.1 Bài toán và khó khăn	72
5.5.2 Giải pháp đề xuất	72
5.5.3 Kết quả đạt được và đánh giá.....	73
5.6 Tổng kết chương	73
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	74
6.1 Kết luận	74
6.2 So sánh với các sản phẩm trên thị trường	74
6.3 Bài học kinh nghiệm	75
6.4 Hạn chế	75
6.5 Hướng phát triển.....	76
TÀI LIỆU THAM KHẢO.....	78

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ Use Case tổng quan của hệ thống	8
Hình 2.2	Biểu đồ Use Case Quản lý thiết bị cá nhân	9
Hình 2.3	Biểu đồ Use Case Xem trạng thái thiết bị	10
Hình 2.4	Biểu đồ Use Case Gửi dữ liệu cảm biến	11
Hình 2.5	Biểu đồ Use Case Quản lý tài khoản cá nhân	12
Hình 2.6	Quy trình nghiệp vụ Kích hoạt thiết bị	13
Hình 2.7	Quy trình nghiệp vụ Giám sát và cảnh báo trộm	14
Hình 3.1	Vi điều khiển Esp32	27
Hình 3.2	Module A7680C	28
Hình 3.3	Module NEO-6M	29
Hình 4.1	Biểu đồ gói tổng quan của hệ thống	32
Hình 4.2	Biểu đồ gói chi tiết của hệ thống	33
Hình 4.3	Thiết kế thanh điều hướng dưới của ứng dụng	37
Hình 4.4	Thiết kế giao diện màn hình trang chủ	38
Hình 4.5	Thiết kế giao diện màn hình chi tiết thiết bị	39
Hình 4.6	Thiết kế giao diện màn hình cảnh báo	40
Hình 4.7	Thiết kế lớp UserService	41
Hình 4.8	Thiết kế lớp DeviceService	42
Hình 4.9	Thiết kế lớp SensorDataService	43
Hình 4.10	Thiết kế lớp MqttService	44
Hình 4.11	Biểu đồ trình tự Gửi cảnh báo	45
Hình 4.12	Biểu đồ thực thể liên kết	46
Hình 4.13	Màn hình Trang chủ của ứng dụng	52
Hình 4.14	Màn hình Cảnh báo của ứng dụng	53
Hình 4.15	Màn hình Chi tiết cảnh báo của ứng dụng	54
Hình 4.16	Màn hình Thiết bị của ứng dụng	55
Hình 4.17	Màn hình Chi tiết thiết bị của ứng dụng	56
Hình 4.18	Màn hình Kích hoạt thiết bị của ứng dụng	57
Hình 4.19	Màn hình Tài khoản của ứng dụng	58

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case Đăng nhập	15
Bảng 2.2	Đặc tả use case Kích hoạt thiết bị	16
Bảng 2.3	Đặc tả use case Xem trạng thái thiết bị	17
Bảng 2.4	Đặc tả use case Gửi dữ liệu cảm biến	18
Bảng 2.5	Đặc tả use case Phát hiện nguy cơ trộm	19
Bảng 2.6	Đặc tả use case Nhận cảnh báo trộm	20
Bảng 2.7	Yêu cầu phi chức năng của hệ thống	21
Bảng 4.1	Danh sách các bảng chính trong cơ sở dữ liệu PostgreSQL . .	48
Bảng 4.2	Danh sách thư viện và công cụ sử dụng	50
Bảng 4.3	Các thành phần chính đã xây dựng	51
Bảng 4.4	Thông kê mã nguồn và thành phần triển khai	52
Bảng 4.5	Các kỹ thuật kiểm thử được sử dụng	59
Bảng 4.6	Các ca kiểm thử chức năng kích hoạt thiết bị	60
Bảng 4.7	Các ca kiểm thử chức năng phát hiện và gửi cảnh báo	61
Bảng 4.8	Các ca kiểm thử chức năng điều khiển còi	62
Bảng 4.9	Các công cụ kiểm thử sử dụng trong đề án	63
Bảng 5.1	Kết quả mong đợi của giải thuật phát hiện bất thường	69

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự sang số
API	Application Programming Interface	Giao diện lập trình ứng dụng
FCM	Firebase Cloud Messaging	Dịch vụ gửi thông báo đẩy của Firebase
GPIO	General-Purpose Input/Output	Chân vào/ra đa dụng
GPS	Global Positioning System	Hệ thống định vị toàn cầu
ID	Identifier	Mã định danh
IDE	Integrated Development Environment	Môi trường phát triển tích hợp
IoT	Internet of Things	Internet vạn vật
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu JavaScript Object Notation
JWT	JSON Web Token	Chuẩn mã thông báo dùng để xác thực và trao đổi thông tin
LTE	Long-Term Evolution	Công nghệ truyền thông di động băng rộng
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông publish/subscribe nhẹ cho thiết bị IoT
ORM	Object-Relational Mapping	Ánh xạ đối tượng–quan hệ
P95	95th Percentile	Phân vị thứ 95
REST	Representational State Transfer	Kiểu kiến trúc truyền trạng thái biểu diễn
SPI	Serial Peripheral Interface	Giao diện ngoại vi nối tiếp
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
URL	Uniform Resource Locator	Địa chỉ định vị tài nguyên thống nhất

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong những năm gần đây, số lượng phương tiện cá nhân, đặc biệt là xe máy, tại Việt Nam ngày càng gia tăng mạnh mẽ. Xe máy hiện vẫn là phương tiện di chuyển chủ yếu của người dân nhờ tính linh hoạt và chi phí sử dụng phù hợp. Tuy nhiên, cùng với sự gia tăng về số lượng phương tiện là tình trạng mất cắp xe diễn ra ngày càng phổ biến và phức tạp. Các đối tượng trộm cắp liên tục sử dụng những phương thức tinh vi để vô hiệu hóa các biện pháp bảo vệ truyền thống như khóa cơ, khóa càng hoặc hệ thống báo động đơn giản. Điều này gây thiệt hại đáng kể về tài sản

Xuất phát từ thực trạng trên, nhu cầu nâng cao khả năng bảo vệ phương tiện của người dân trở nên cấp thiết. Nhiều người sử dụng xe máy thường phải đối mặt với tâm lý lo lắng khi gửi xe tại nơi công cộng hoặc khi không thể trực tiếp giám sát phương tiện của mình. Điều này đặt ra yêu cầu cần có những giải pháp hỗ trợ người dùng theo dõi và quản lý phương tiện một cách chủ động hơn, góp phần giảm thiểu nguy cơ mất cắp và nâng cao mức độ an toàn cho tài sản cá nhân.

Nếu bài toán này được giải quyết hiệu quả, người dùng sẽ có khả năng nắm bắt tình trạng hoạt động của phương tiện một cách kịp thời, từ đó phát hiện sớm các dấu hiệu bất thường và có biện pháp xử lý phù hợp. Đồng thời, việc nâng cao hiệu quả bảo vệ phương tiện không chỉ giúp giảm thiểu thiệt hại về tài sản mà còn góp phần tăng cường ý thức quản lý và bảo vệ tài sản cá nhân trong cộng đồng. Bên cạnh lĩnh vực chống trộm xe máy, các giải pháp giám sát thông minh còn có tiềm năng mở rộng sang nhiều lĩnh vực khác như quản lý tài sản, giám sát phương tiện vận tải và theo dõi hàng hóa trong quá trình vận chuyển.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, trên thị trường Việt Nam đã xuất hiện nhiều sản phẩm hỗ trợ định vị và chống trộm xe máy. Một số sản phẩm tiêu biểu có thể kể đến như thiết bị Smart Motor của Viettel, các thiết bị giám sát hành trình của BA GPS/Bình Anh, thiết bị định vị Protrack, cùng nhiều dòng thiết bị định vị GPS 4G nhỏ gọn khác được cung cấp bởi các đơn vị phân phối thiết bị giám sát phương tiện. Nhìn chung, các sản phẩm này thường cung cấp những chức năng như xác định vị trí phương tiện, xem lại lịch sử hành trình, cảnh báo khi xe có dấu hiệu bất thường, thiết lập vùng an toàn hoặc hỗ trợ quản lý phương tiện thông qua ứng dụng trên điện thoại.

Các sản phẩm định vị xe máy hiện có đã góp phần hỗ trợ người dùng trong việc theo dõi và bảo vệ phương tiện cá nhân. Tuy nhiên, qua khảo sát tổng quan có thể

nhận thấy các giải pháp này vẫn tồn tại một số hạn chế nhất định. Một số sản phẩm có chi phí triển khai và duy trì tương đối cao đối với người dùng cá nhân; một số khác phụ thuộc nhiều vào hạ tầng máy chủ và phần mềm của nhà cung cấp nên khả năng tùy biến còn hạn chế. Bên cạnh đó, nhiều thiết bị chủ yếu tập trung vào chức năng định vị và xem lại hành trình, trong khi việc tích hợp đồng thời các chức năng phát hiện bất thường, cảnh báo tức thời, quản lý trạng thái thiết bị và hỗ trợ người dùng tương tác với hệ thống vẫn chưa thật sự linh hoạt.

Từ những phân tích trên, đề tài hướng tới xây dựng một hệ thống chống trộm xe ứng dụng công nghệ IoT có khả năng giám sát trạng thái phương tiện, phát hiện các dấu hiệu bất thường và gửi cảnh báo đến người dùng trong thời gian ngắn. Hệ thống đồng thời hỗ trợ theo dõi vị trí phương tiện, quản lý trạng thái hoạt động và cung cấp giao diện ứng dụng giúp người dùng dễ dàng quan sát, điều khiển và nhận thông tin cảnh báo.

Trong phạm vi đề án, hệ thống tập trung vào các chức năng chính gồm thu thập dữ liệu từ thiết bị gắn trên xe, phát hiện rung động hoặc thay đổi trạng thái bất thường, gửi cảnh báo đến ứng dụng người dùng, hiển thị vị trí phương tiện trên bản đồ và cho phép người dùng quản lý trạng thái chống trộm từ xa. Đề tài tập trung xây dựng mô hình hệ thống ở quy mô thử nghiệm, phục vụ mục tiêu nghiên cứu, thiết kế và đánh giá khả năng hoạt động của giải pháp.

1.3 Định hướng giải pháp

Từ mục tiêu và phạm vi đã xác định, đề tài lựa chọn định hướng xây dựng hệ thống theo mô hình Internet vạn vật, kết hợp giữa thiết bị nhúng, máy chủ trung gian và ứng dụng di động. Cách tiếp cận này cho phép thiết bị gắn trên phương tiện có thể thu thập dữ liệu từ môi trường thực tế, truyền thông tin về hệ thống xử lý trung tâm và cung cấp dữ liệu cho người dùng thông qua ứng dụng trên điện thoại. Định hướng này phù hợp với bài toán chống trộm xe do hệ thống cần có khả năng giám sát từ xa, cảnh báo kịp thời và hỗ trợ người dùng theo dõi trạng thái phương tiện trong thời gian thực.

Giải pháp được đề xuất gồm ba thành phần chính: thiết bị phần cứng gắn trên xe, hệ thống máy chủ và ứng dụng di động. Thiết bị phần cứng có nhiệm vụ thu thập dữ liệu từ các cảm biến, xác định trạng thái của phương tiện và gửi thông tin về máy chủ. Máy chủ đóng vai trò tiếp nhận, xử lý, lưu trữ dữ liệu và cung cấp các giao tiếp cần thiết giữa thiết bị và ứng dụng di động. Ứng dụng di động được xây dựng nhằm hỗ trợ người dùng theo dõi vị trí, trạng thái phương tiện, nhận thông báo cảnh báo và thực hiện một số thao tác quản lý hệ thống từ xa.

Đóng góp chính của đề án là thiết kế và xây dựng một hệ thống chống trộm xe

hoàn chỉnh, có chi phí triển khai thấp và phù hợp với quy mô thử nghiệm thực tế. Kết quả đạt được của đề tài là một hệ thống có khả năng giám sát trạng thái phương tiện, phát hiện dấu hiệu bất thường, gửi cảnh báo kịp thời đến người dùng và hỗ trợ theo dõi vị trí xe khi cần thiết.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp được tổ chức như sau.

Chương 2, Khảo sát bài toán và các giải pháp liên quan, trình bày quá trình khảo sát nhu cầu thực tế đối với hệ thống chống trộm xe máy và các giải pháp định vị, giám sát phương tiện hiện có trên thị trường. Nội dung chương tập trung phân tích một số sản phẩm tiêu biểu, đánh giá ưu điểm và hạn chế của các giải pháp hiện nay, từ đó làm cơ sở để xác định yêu cầu đối với hệ thống được xây dựng trong đồ án.

Chương 3, Cơ sở công nghệ sử dụng trong hệ thống, trình bày các công nghệ, nền tảng và công cụ được sử dụng trong quá trình phát triển hệ thống. Các nội dung chính bao gồm công nghệ Internet vạn vật, thiết bị nhúng, cảm biến, công nghệ định vị, phương thức truyền thông giữa thiết bị và máy chủ, cơ sở dữ liệu, công nghệ xây dựng máy chủ và công nghệ phát triển ứng dụng di động.

Chương 4, Kết quả xây dựng và thực nghiệm hệ thống, em sẽ trình bày kết quả triển khai hệ thống sau khi các thành phần được xây dựng và tích hợp. Chương này mô tả các kịch bản thử nghiệm đối với những chức năng chính như giám sát trạng thái phương tiện, phát hiện dấu hiệu bất thường, gửi cảnh báo, cập nhật vị trí và hiển thị thông tin trên ứng dụng di động. Dựa trên kết quả thu được, chương đưa ra đánh giá về khả năng hoạt động và mức độ đáp ứng yêu cầu của hệ thống.

Chương 5, Giải pháp đề xuất và đóng góp của đồ án, trình bày giải pháp được xây dựng trong đề tài và các đóng góp chính của đồ án. Nội dung chương tập trung mô tả kiến trúc tổng thể, thiết kế các thành phần chức năng, luồng xử lý dữ liệu và cách thức phối hợp giữa các thiết bị. Bên cạnh đó, chương cũng làm rõ những điểm đóng góp của hệ thống so với các giải pháp đã khảo sát.

Chương 6, Kết luận và hướng phát triển, trình bày tổng kết các kết quả đã đạt được trong quá trình thực hiện đồ án. Chương này đánh giá mức độ hoàn thành so với mục tiêu ban đầu, chỉ ra những hạn chế còn tồn tại và đề xuất một số hướng cải tiến trong tương lai nhằm nâng cao độ ổn định, độ chính xác, khả năng mở rộng và tính ứng dụng thực tế của hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Trong Chương 1, đề án đã trình bày bối cảnh thực tế, mục tiêu, phạm vi và định hướng giải pháp cho bài toán xây dựng hệ thống chống trộm xe ứng dụng công nghệ IoT. Trên cơ sở đó, Chương 2 tập trung khảo sát hiện trạng và phân tích các yêu cầu cần có đối với hệ thống. Đây là bước quan trọng nhằm làm rõ nhu cầu thực tế của người dùng, các chức năng cần xây dựng và những ràng buộc mà hệ thống cần đáp ứng trước khi đi vào lựa chọn công nghệ, thiết kế và triển khai.

Nội dung chương này trước hết trình bày khảo sát một số sản phẩm và giải pháp định vị, chống trộm phương tiện đang có trên thị trường. Từ kết quả khảo sát, chương phân tích ưu điểm, hạn chế của các giải pháp hiện có và rút ra các yêu cầu đối với hệ thống được xây dựng trong đề án. Tiếp theo, chương mô tả tổng quan các chức năng chính của hệ thống thông qua biểu đồ use case, xác định các tác nhân tham gia và đặc tả một số chức năng quan trọng. Cuối cùng, chương trình bày các yêu cầu phi chức năng liên quan đến hiệu năng, độ tin cậy, tính dễ sử dụng và khả năng mở rộng của hệ thống.

2.1 Khảo sát hiện trạng

Trong những năm gần đây, nhu cầu giám sát và bảo vệ phương tiện cá nhân ngày càng được quan tâm, đặc biệt đối với xe máy. Bên cạnh các phương pháp bảo vệ truyền thống như khóa cơ, khóa còng hoặc còi báo động độc lập, trên thị trường đã xuất hiện nhiều thiết bị định vị và chống trộm sử dụng công nghệ GPS, mạng di động và ứng dụng trên điện thoại thông minh. Các sản phẩm này cho phép người dùng theo dõi vị trí phương tiện, xem lại lịch sử hành trình, nhận cảnh báo khi có dấu hiệu bất thường và trong một số trường hợp có thể hỗ trợ điều khiển phương tiện từ xa.

Một trong những nhóm sản phẩm phổ biến là các thiết bị định vị xe máy của Viettel Smart Motor. Dòng Smart Motor S1 tập trung vào chức năng định vị GPS, giám sát hành trình và quản lý vị trí xe thông qua ứng dụng. Trong khi đó, dòng Smart Motor S2 được bổ sung thêm các chức năng chống trộm như cảnh báo qua ứng dụng, cảnh báo khi xe bị mở khóa, rung lắc, vượt quá phạm vi cho phép, quá tốc độ, đồng thời hỗ trợ tắt máy xe từ xa. Các thiết bị này có ưu điểm là sử dụng hạ tầng mạng di động và hệ thống dịch vụ có sẵn, phù hợp với người dùng có nhu cầu lắp đặt nhanh và sử dụng trực tiếp. Tuy nhiên, người dùng thường phụ thuộc vào thiết bị, ứng dụng và hạ tầng dịch vụ của nhà cung cấp, trong khi khả năng tùy biến hoặc mở rộng chức năng theo nhu cầu riêng còn hạn chế.

Ngoài ra, thị trường còn có các thiết bị định vị phương tiện như Protrack VT05F và một số thiết bị GPS 4G nhỏ gọn khác. Các thiết bị này thường hỗ trợ định vị GPS, ghi lại lịch sử hành trình, thiết lập hàng rào địa lý, cảnh báo rung lắc, cảnh báo quá tốc độ hoặc cảnh báo khi có thay đổi trạng thái nguồn điện. Một số sản phẩm còn có khả năng chống nước và được sử dụng cho nhiều loại phương tiện như xe máy, ô tô, xe công trình hoặc phương tiện giao hàng. Nhóm sản phẩm này có ưu điểm là đa dạng về mẫu mã, dễ tìm mua và đáp ứng được các nhu cầu giám sát cơ bản. Tuy nhiên, các chức năng thường được thiết kế theo dạng đóng gói sẵn, ít cho phép người dùng can thiệp vào cách xử lý dữ liệu, cách gửi cảnh báo hoặc cách tích hợp với các hệ thống phần mềm khác.

Bên cạnh các thiết bị thương mại, một số giải pháp chống trộm xe máy phổ thông hiện nay vẫn chủ yếu dựa trên còi báo động, cảm biến rung hoặc khóa điện tử. Các giải pháp này có chi phí thấp và dễ lắp đặt, nhưng phạm vi hoạt động còn hạn chế do chỉ cảnh báo tại chỗ hoặc chỉ tác động trực tiếp lên xe. Trong trường hợp người dùng ở xa phương tiện, các cảnh báo dạng âm thanh có thể không phát huy hiệu quả. Ngoài ra, các hệ thống này thường không cung cấp khả năng theo dõi vị trí, lưu trữ lịch sử hoạt động hoặc quản lý trạng thái phương tiện thông qua ứng dụng.

Từ quá trình khảo sát có thể nhận thấy các sản phẩm định vị và chống trộm xe máy hiện nay đã hỗ trợ tốt một số nhu cầu quan trọng như xác định vị trí phương tiện, cảnh báo khi xe có dấu hiệu bất thường và quản lý hành trình. Tuy nhiên, các giải pháp thương mại thường có chi phí triển khai và duy trì nhất định, phụ thuộc nhiều vào nền tảng của nhà cung cấp và khó tùy biến theo mục tiêu nghiên cứu hoặc thử nghiệm. Trong khi đó, các giải pháp chống trộm truyền thống có chi phí thấp hơn nhưng thiếu khả năng giám sát từ xa, thiếu cơ chế lưu trữ dữ liệu và chưa hỗ trợ tương tác linh hoạt với người dùng.

Từ những hạn chế trên, đề án hướng tới xây dựng một hệ thống chống trộm xe ứng dụng công nghệ IoT ở quy mô thử nghiệm, có khả năng kết hợp giữa giám sát trạng thái phương tiện, phát hiện dấu hiệu bất thường, gửi cảnh báo đến người dùng và hỗ trợ theo dõi vị trí xe. Hệ thống được định hướng phát triển theo tiêu chí chi phí thấp, dễ triển khai, có khả năng tùy biến và phù hợp với mục tiêu học tập, nghiên cứu cũng như kiểm chứng hoạt động trên thiết bị thật.

2.2 Tổng quan chức năng

Dựa trên kết quả khảo sát hiện trạng và mục tiêu của đề tài, hệ thống chống trộm xe ứng dụng công nghệ IoT được xây dựng nhằm hỗ trợ người dùng giám sát, quản lý và bảo vệ phương tiện từ xa. Ở mức tổng quan, hệ thống cần cung cấp các chức

năng chính liên quan đến quản lý tài khoản, quản lý thiết bị, theo dõi trạng thái phương tiện, giám sát vị trí, nhận cảnh báo và điều khiển chế độ chống trộm.

Trước hết, hệ thống cho phép người dùng đăng ký, đăng nhập và quản lý thông tin tài khoản để đảm bảo chỉ những người dùng hợp lệ mới có thể truy cập và sử dụng các chức năng của ứng dụng. Sau khi đăng nhập, người dùng có thể liên kết phương tiện với thiết bị chống trộm tương ứng, từ đó quản lý thông tin xe và trạng thái hoạt động của thiết bị thông qua ứng dụng di động.

Đối với chức năng giám sát phương tiện, hệ thống hỗ trợ hiển thị trạng thái hiện tại của xe, bao gồm trạng thái bật hoặc tắt chế độ chống trộm, trạng thái kết nối của thiết bị và các thông tin liên quan đến hoạt động của phương tiện. Khi thiết bị phát hiện dấu hiệu bất thường như rung động hoặc thay đổi trạng thái ngoài ý muốn, hệ thống sẽ gửi cảnh báo đến người dùng nhằm hỗ trợ phát hiện sớm nguy cơ mất an toàn.

Bên cạnh chức năng cảnh báo, hệ thống còn hỗ trợ theo dõi vị trí của phương tiện. Thông tin vị trí được gửi từ thiết bị về máy chủ và hiển thị trên ứng dụng di động, giúp người dùng quan sát vị trí xe trong trường hợp cần thiết. Chức năng này có vai trò quan trọng trong việc hỗ trợ tìm kiếm phương tiện khi xảy ra tình huống mất cắp hoặc di chuyển bất thường.

Ngoài các chức năng giám sát và cảnh báo, người dùng có thể thực hiện một số thao tác điều khiển từ xa đối với hệ thống, chẳng hạn như bật hoặc tắt chế độ chống trộm, kiểm tra trạng thái thiết bị và cập nhật thông tin phương tiện. Các chức năng này giúp người dùng chủ động hơn trong quá trình quản lý phương tiện, đồng thời tăng tính linh hoạt của hệ thống trong các tình huống sử dụng thực tế.

Tổng quan các chức năng chính của hệ thống được thể hiện thông qua biểu đồ use case ở các mục tiếp theo. Biểu đồ use case giúp xác định các tác nhân tham gia hệ thống, các nhóm chức năng chính và mối quan hệ giữa người dùng với hệ thống chống trộm xe.

2.2.1 Biểu đồ use case tổng quát

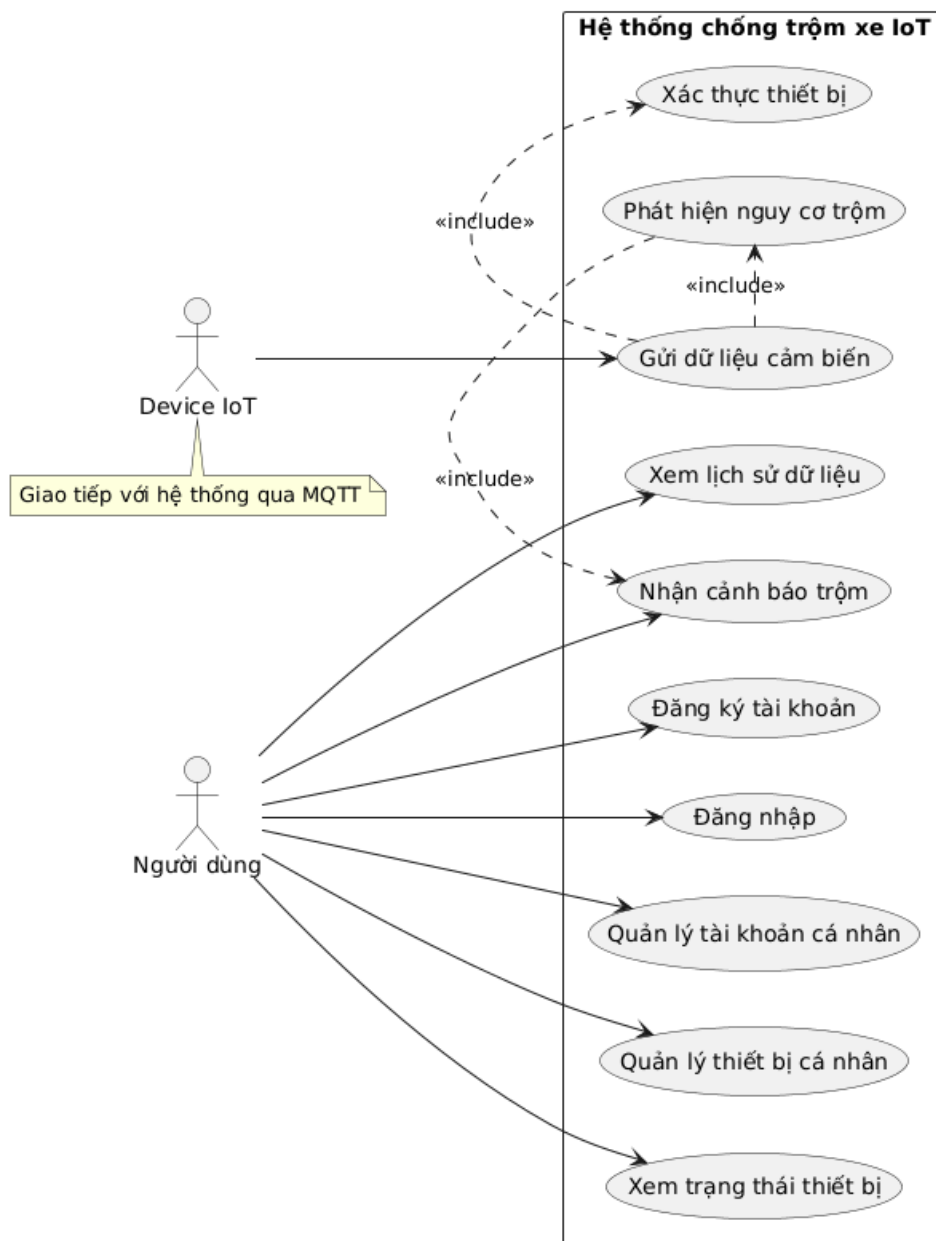
Biểu đồ use case tổng quát của hệ thống được xây dựng nhằm mô tả các chức năng chính của hệ thống và mối quan hệ giữa các tác nhân với các chức năng đó. Trong hệ thống, có hai tác nhân chính tham gia tương tác là Người dùng và Device IoT. Người dùng là tác nhân sử dụng ứng dụng di động để quản lý tài khoản, quản lý thiết bị, theo dõi trạng thái phương tiện và nhận các cảnh báo khi hệ thống phát hiện nguy cơ mất an toàn. Device IoT là thiết bị phần cứng được gắn trên phương tiện, có nhiệm vụ thu thập dữ liệu cảm biến và gửi dữ liệu về hệ thống thông qua

giao thức MQTT.

Đối với tác nhân Người dùng, hệ thống cung cấp các chức năng cơ bản gồm đăng ký tài khoản và đăng nhập. Hai chức năng này giúp xác thực người dùng trước khi cho phép truy cập vào các chức năng quản lý và giám sát phương tiện. Sau khi đăng nhập thành công, người dùng có thể thực hiện quản lý tài khoản cá nhân, quản lý thiết bị cá nhân, xem trạng thái thiết bị, xem lịch sử dữ liệu và nhận cảnh báo trộm. Các chức năng này hỗ trợ người dùng theo dõi thông tin phương tiện, kiểm tra trạng thái hoạt động của thiết bị chống trộm và nắm bắt các sự kiện bất thường trong quá trình sử dụng.

Đối với tác nhân Device IoT, chức năng chính là gửi dữ liệu cảm biến về hệ thống. Trước khi dữ liệu được xử lý, thiết bị cần được xác thực nhằm đảm bảo dữ liệu gửi lên đến từ thiết bị hợp lệ. Sau đó, hệ thống sử dụng dữ liệu cảm biến để phát hiện nguy cơ trộm hoặc các dấu hiệu bất thường liên quan đến phương tiện. Khi phát hiện nguy cơ trộm, hệ thống thực hiện chức năng gửi cảnh báo đến người dùng để hỗ trợ người dùng kịp thời kiểm tra và xử lý tình huống.

Trong biểu đồ, quan hệ *include* được sử dụng để thể hiện các chức năng bắt buộc được thực hiện trong quá trình xử lý một chức năng lớn hơn. Cụ thể, use case “Gửi dữ liệu cảm biến” bao gồm use case “Xác thực thiết bị” và “Phát hiện nguy cơ trộm”, vì hệ thống cần kiểm tra tính hợp lệ của thiết bị trước khi xử lý dữ liệu và đánh giá trạng thái an toàn của phương tiện. Use case “Phát hiện nguy cơ trộm” bao gồm use case “Nhận cảnh báo trộm”, do cảnh báo được gửi đến người dùng khi hệ thống xác định có dấu hiệu bất thường. Tổng thể, biểu đồ use case cho thấy hệ thống tập trung vào hai nhóm chức năng chính: nhóm chức năng phục vụ người dùng quản lý và giám sát phương tiện, và nhóm chức năng phục vụ thiết bị IoT gửi dữ liệu, hỗ trợ phát hiện nguy cơ trộm.



Hình 2.1: Biểu đồ Use Case tổng quan của hệ thống

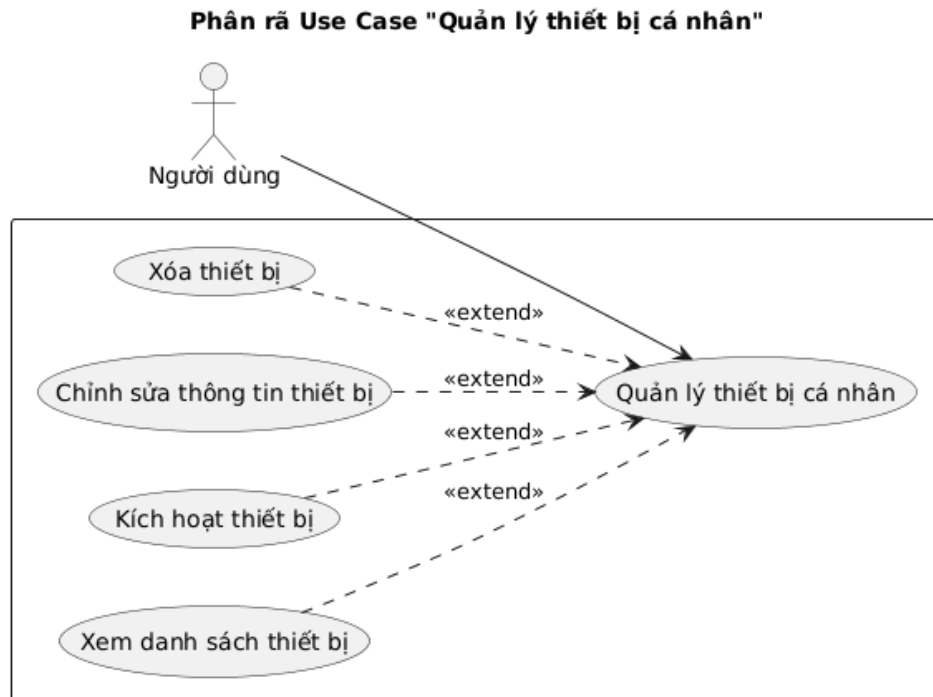
2.2.2 Biểu đồ use case phân rã Quản lý thiết bị cá nhân

Biểu đồ use case phân rã nhóm chức năng “Quản lý thiết bị cá nhân” mô tả các thao tác chính mà người dùng có thể thực hiện đối với thiết bị chống trộm được liên kết với tài khoản của mình. Trong nhóm chức năng này, tác nhân tham gia là Người dùng, thông qua ứng dụng di động để quản lý các thiết bị IoT gắn trên phương tiện.

Use case “Quản lý thiết bị cá nhân” được phân rã thành bốn chức năng chính gồm xem danh sách thiết bị, kích hoạt thiết bị, chỉnh sửa thông tin thiết bị và xóa thiết bị. Chức năng xem danh sách thiết bị giúp người dùng theo dõi các thiết bị đang được quản lý. Chức năng kích hoạt thiết bị cho phép người dùng liên kết thiết bị mới với tài khoản. Chức năng chỉnh sửa thông tin thiết bị hỗ trợ cập nhật các

thông tin liên quan đến thiết bị hoặc phương tiện. Chức năng xóa thiết bị được sử dụng khi người dùng không còn nhu cầu quản lý thiết bị đó trong hệ thống.

Các use case con được biểu diễn dưới dạng các chức năng mở rộng của use case “Quản lý thiết bị cá nhân”, thể hiện rằng người dùng có thể lựa chọn thực hiện từng thao tác tùy theo nhu cầu trong quá trình quản lý thiết bị.



Hình 2.2: Biểu đồ Use Case Quản lý thiết bị cá nhân

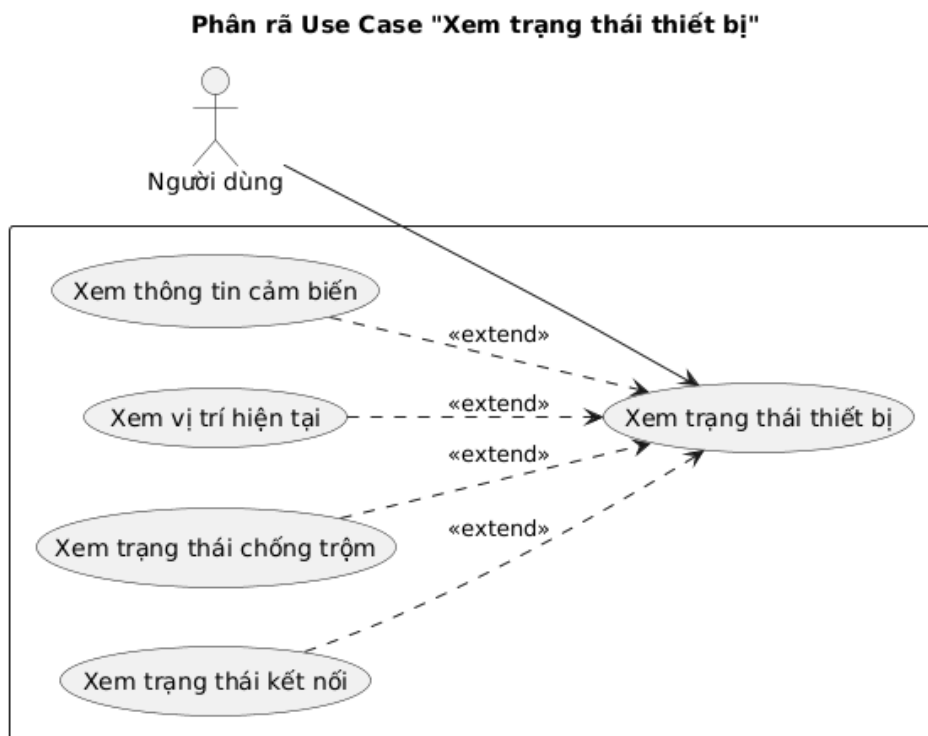
2.2.3 Biểu đồ use case phân rã Xem trạng thái thiết bị

Biểu đồ use case phân rã nhóm chức năng “Xem trạng thái thiết bị” mô tả các thông tin mà người dùng có thể theo dõi đối với thiết bị chống trộm được gắn trên phương tiện. Trong nhóm chức năng này, tác nhân tham gia là Người dùng, thông qua ứng dụng di động để quan sát trạng thái hoạt động hiện tại của thiết bị và phương tiện.

Use case “Xem trạng thái thiết bị” được phân rã thành các chức năng gồm xem trạng thái kết nối, xem trạng thái chống trộm, xem vị trí hiện tại và xem thông tin cảm biến. Chức năng xem trạng thái kết nối giúp người dùng biết thiết bị có đang kết nối với hệ thống hay không. Chức năng xem trạng thái chống trộm cho biết chế độ bảo vệ của phương tiện đang được bật hay tắt. Chức năng xem vị trí hiện tại hỗ trợ người dùng theo dõi vị trí gần nhất của xe. Chức năng xem thông tin cảm biến hiển thị các dữ liệu liên quan đến trạng thái phát hiện của thiết bị.

Các use case con được biểu diễn dưới dạng các chức năng mở rộng của use case

“Xem trạng thái thiết bị”, thể hiện rằng người dùng có thể theo dõi từng nhóm thông tin cụ thể tùy theo nhu cầu trong quá trình giám sát phương tiện.



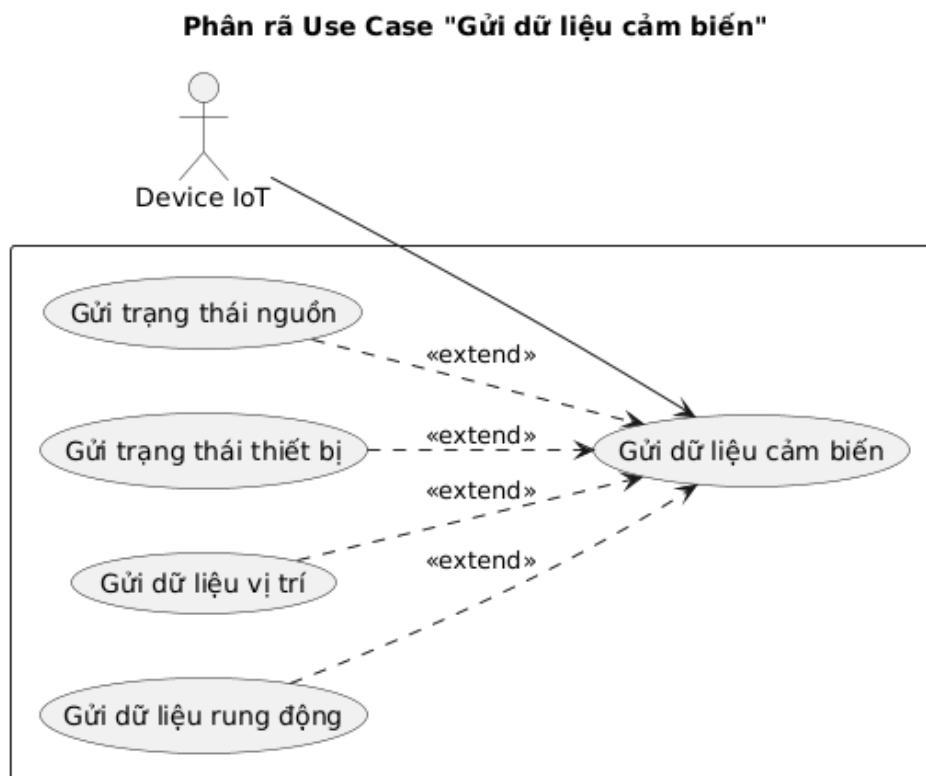
Hình 2.3: Biểu đồ Use Case Xem trạng thái thiết bị

2.2.4 Biểu đồ use case phân rã Gửi dữ liệu cảm biến

Biểu đồ use case phân rã nhóm chức năng “Gửi dữ liệu cảm biến” mô tả các loại dữ liệu chính mà thiết bị IoT gửi về hệ thống trong quá trình hoạt động. Tác nhân tham gia vào nhóm chức năng này là Device IoT, đại diện cho thiết bị phần cứng được gắn trên phương tiện và giao tiếp với hệ thống thông qua giao thức MQTT.

Use case “Gửi dữ liệu cảm biến” được phân rã thành các chức năng gồm gửi dữ liệu rung động, gửi dữ liệu vị trí, gửi trạng thái thiết bị và gửi trạng thái nguồn. Dữ liệu rung động hỗ trợ hệ thống nhận biết các tác động bất thường lên phương tiện. Dữ liệu vị trí phục vụ việc cập nhật vị trí hiện tại của xe. Trạng thái thiết bị và trạng thái nguồn giúp hệ thống theo dõi tình trạng hoạt động của thiết bị trong quá trình vận hành.

Các use case con được biểu diễn như các chức năng mở rộng của use case “Gửi dữ liệu cảm biến”, thể hiện rằng tùy theo trạng thái hoạt động và dữ liệu thu thập được, thiết bị có thể gửi các nhóm thông tin khác nhau về hệ thống.



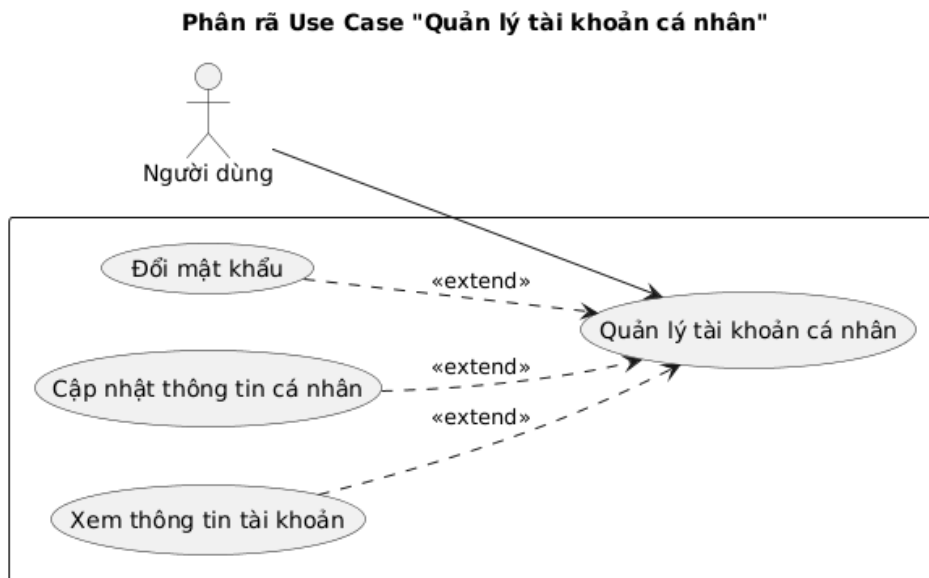
Hình 2.4: Biểu đồ Use Case Gửi dữ liệu cảm biến

2.2.5 Biểu đồ use case phân rã Quản lý tài khoản cá nhân

Biểu đồ use case phân rã nhóm chức năng “Quản lý tài khoản cá nhân” mô tả các thao tác chính mà người dùng có thể thực hiện đối với tài khoản của mình trong hệ thống. Tác nhân tham gia vào nhóm chức năng này là Người dùng, thông qua ứng dụng di động để xem và cập nhật các thông tin cá nhân cần thiết.

Use case “Quản lý tài khoản cá nhân” được phân rã thành ba chức năng chính gồm xem thông tin tài khoản, cập nhật thông tin cá nhân và đổi mật khẩu. Chức năng xem thông tin tài khoản cho phép người dùng kiểm tra các thông tin hiện có của tài khoản. Chức năng cập nhật thông tin cá nhân hỗ trợ người dùng thay đổi một số thông tin như tên hiển thị hoặc số điện thoại. Chức năng đổi mật khẩu giúp người dùng thay đổi thông tin xác thực nhằm đảm bảo an toàn cho tài khoản.

Các use case con được biểu diễn dưới dạng các chức năng mở rộng của use case “Quản lý tài khoản cá nhân”, thể hiện rằng người dùng có thể lựa chọn thực hiện từng thao tác tùy theo nhu cầu trong quá trình sử dụng hệ thống.



Hình 2.5: Biểu đồ Use Case Quản lý tài khoản cá nhân

2.2.6 Quy trình nghiệp vụ Kích hoạt thiết bị

Quy trình nghiệp vụ kích hoạt thiết bị mô tả các bước cần thực hiện để người dùng liên kết một thiết bị chống trộm với tài khoản cá nhân. Quy trình bắt đầu khi người dùng đăng nhập vào ứng dụng di động và chọn chức năng kích hoạt thiết bị. Sau đó, ứng dụng yêu cầu người dùng nhập các thông tin cần thiết gồm DeviceID, DeviceKey và thông tin phương tiện tương ứng.

Sau khi người dùng gửi yêu cầu kích hoạt, hệ thống máy chủ tiếp nhận và kiểm tra tính hợp lệ của DeviceID và DeviceKey. Nếu thông tin thiết bị không hợp lệ, ứng dụng sẽ hiển thị thông báo lỗi để người dùng kiểm tra và nhập lại. Nếu thông tin hợp lệ, máy chủ tiếp tục kiểm tra trạng thái liên kết của thiết bị. Trong trường hợp thiết bị chưa được liên kết với tài khoản nào, hệ thống lưu thông tin phương tiện, liên kết thiết bị với tài khoản người dùng và cập nhật trạng thái thiết bị là đã kích hoạt.

Khi quá trình kích hoạt hoàn tất, ứng dụng di động hiển thị thông báo kích hoạt thành công và người dùng có thể xem thiết bị trong danh sách quản lý. Trường hợp thiết bị đã được liên kết trước đó, hệ thống sẽ thông báo cho người dùng để tránh việc một thiết bị bị kích hoạt lại hoặc gắn với nhiều tài khoản không hợp lệ.



Hình 2.6: Quy trình nghiệp vụ Kích hoạt thiết bị

2.2.7 Quy trình nghiệp vụ Giám sát và cảnh báo trộm

Quy trình nghiệp vụ giám sát và cảnh báo trộm mô tả quá trình hệ thống theo dõi trạng thái phương tiện và gửi cảnh báo đến người dùng khi phát hiện dấu hiệu bất thường. Quy trình bắt đầu khi người dùng bật chế độ chống trộm trên ứng dụng di động. Sau đó, ứng dụng gửi yêu cầu đến máy chủ để cập nhật trạng thái chống trộm của thiết bị tương ứng.

Trong quá trình hoạt động, thiết bị IoT thu thập dữ liệu từ cảm biến và gửi dữ liệu về máy chủ thông qua giao thức MQTT. Sau khi tiếp nhận dữ liệu, máy chủ thực hiện xác thực thiết bị nhằm đảm bảo dữ liệu đến từ thiết bị hợp lệ. Nếu thiết bị không hợp lệ, dữ liệu sẽ bị từ chối. Nếu thiết bị hợp lệ, hệ thống lưu dữ liệu cảm biến, vị trí và kiểm tra trạng thái chống trộm hiện tại của thiết bị.

Khi chế độ chống trộm đang được bật, máy chủ phân tích dữ liệu cảm biến để xác định xem phương tiện có xuất hiện dấu hiệu bất thường hay không. Nếu không phát hiện nguy cơ trộm, hệ thống chỉ cập nhật trạng thái thiết bị bình thường. Ngược lại, nếu phát hiện nguy cơ trộm, hệ thống tạo bản ghi cảnh báo, lưu lịch sử cảnh báo và gửi thông báo đến ứng dụng di động. Người dùng sau đó có thể mở ứng dụng để xem chi tiết cảnh báo, trạng thái thiết bị và vị trí hiện tại của xe.



Hình 2.7: Quy trình nghiệp vụ Giám sát và cảnh báo trộm

2.3 Đặc tả chức năng

2.3.1 Đặc tả use case Đăng nhập

Use case “Đăng nhập” cho phép người dùng truy cập vào hệ thống bằng tài khoản đã được đăng ký trước đó. Đây là chức năng xác thực cơ bản, đảm bảo chỉ những người dùng hợp lệ mới có thể sử dụng các chức năng quản lý, giám sát thiết bị.

Mã UseCase	UC01
Tên UseCase	Đăng nhập
Tác nhân	Người dùng
Mục đích sử dụng	Xác thực tài khoản để cho phép người dùng truy cập vào hệ thống và sử dụng các chức năng quản lý, giám sát thiết bị.
Sự kiện kích hoạt	Người dùng chọn chức năng đăng nhập trên ứng dụng di động.

Điều kiện tiên quyết	Người dùng đã có tài khoản trong hệ thống và thiết bị di động có kết nối Internet.
Luồng sự kiện chính (thành công)	1. Người dùng mở ứng dụng di động. 2. Hệ thống hiển thị giao diện đăng nhập. 3. Người dùng nhập thông tin tài khoản và mật khẩu. 4. Người dùng chọn chức năng đăng nhập. 5. Hệ thống kiểm tra tính hợp lệ của thông tin đăng nhập. 6. Hệ thống xác thực tài khoản người dùng. 7. Hệ thống thông báo đăng nhập thành công và chuyển người dùng đến màn hình chính của ứng dụng.
Luồng sự kiện thay thế	3.1 Người dùng chưa nhập đầy đủ thông tin đăng nhập. Hệ thống hiển thị thông báo yêu cầu nhập đầy đủ thông tin. 5.1 Thông tin tài khoản hoặc mật khẩu không đúng. Hệ thống hiển thị thông báo đăng nhập thất bại. 5.2 Thiết bị di động không có kết nối Internet. Hệ thống hiển thị thông báo lỗi kết nối.
Hậu điều kiện	Người dùng đăng nhập thành công và có thể truy cập các chức năng của hệ thống theo tài khoản đã xác thực.

Bảng 2.1: Đặc tả use case Đăng nhập

2.3.2 Đặc tả use case Kích hoạt thiết bị

Use case “Kích hoạt thiết bị” cho phép người dùng liên kết một thiết bị chống trộm với tài khoản cá nhân. Sau khi kích hoạt thành công, thiết bị được đưa vào danh sách quản lý của người dùng và có thể được sử dụng để giám sát trạng thái phương tiện, gửi dữ liệu cảm biến và cảnh báo khi phát hiện dấu hiệu bất thường.

Mã UseCase	UC02
Tên UseCase	Kích hoạt thiết bị
Tác nhân	Người dùng
Mục đích sử dụng	Liên kết thiết bị IoT với tài khoản người dùng và phương tiện tương ứng.
Sự kiện kích hoạt	Người dùng chọn chức năng kích hoạt thiết bị.

Điều kiện tiên quyết	Người dùng đã đăng nhập và có DeviceID, DeviceKey của thiết bị.
Luồng sự kiện chính (thành công)	1. Người dùng chọn chức năng kích hoạt thiết bị. 2. Hệ thống hiển thị giao diện nhập thông tin kích hoạt. 3. Người dùng nhập DeviceID, DeviceKey và thông tin phương tiện. 4. Hệ thống kiểm tra tính hợp lệ của DeviceID và DeviceKey. 5. Hệ thống kiểm tra trạng thái liên kết của thiết bị. 6. Hệ thống liên kết thiết bị với tài khoản người dùng. 7. Hệ thống thông báo kích hoạt thiết bị thành công.
Luồng sự kiện thay thế	3.1 Người dùng nhập thiếu thông tin. Hệ thống yêu cầu nhập đầy đủ thông tin. 4.1 DeviceID hoặc DeviceKey không hợp lệ. Hệ thống thông báo kích hoạt thất bại. 5.1 Thiết bị đã được liên kết với tài khoản khác. Hệ thống thông báo thiết bị đã được kích hoạt trước đó.
Hậu điều kiện	Thiết bị được liên kết với tài khoản người dùng và xuất hiện trong danh sách thiết bị cá nhân.

Bảng 2.2: Đặc tả use case Kích hoạt thiết bị

2.3.3 Đặc tả use case Xem trạng thái thiết bị

Use case “Xem trạng thái thiết bị” cho phép người dùng theo dõi trạng thái hoạt động của thiết bị chống trộm, trạng thái chống trộm, vị trí hiện tại và một số thông tin cảm biến liên quan đến phương tiện.

Mã UseCase	UC03
Tên UseCase	Xem trạng thái thiết bị
Tác nhân	Người dùng
Mục đích sử dụng	Cho phép người dùng theo dõi trạng thái hiện tại của thiết bị và phương tiện trên ứng dụng di động.
Sự kiện kích hoạt	Người dùng chọn một thiết bị trong danh sách thiết bị cá nhân để xem trạng thái.
Điều kiện tiên quyết	Người dùng đã đăng nhập và đã kích hoạt ít nhất một thiết bị trong hệ thống.

Luồng sự kiện chính (thành công)	1. Người dùng mở danh sách thiết bị cá nhân. 2. Người dùng chọn thiết bị cần xem trạng thái. 3. Hệ thống truy xuất thông tin trạng thái mới nhất của thiết bị. 4. Hệ thống hiển thị trạng thái kết nối của thiết bị. 5. Hệ thống hiển thị trạng thái chống trộm của phương tiện. 6. Hệ thống hiển thị vị trí hiện tại hoặc vị trí gần nhất của thiết bị. 7. Hệ thống hiển thị một số thông tin cảm biến liên quan.
Luồng sự kiện thay thế	1.1 Người dùng chưa có thiết bị nào được kích hoạt. Hệ thống hiển thị thông báo chưa có thiết bị. 3.1 Hệ thống không lấy được dữ liệu mới nhất. Hệ thống hiển thị dữ liệu gần nhất đã được lưu. 6.1 Thiết bị chưa gửi dữ liệu vị trí. Hệ thống hiển thị thông báo chưa có thông tin vị trí.
Hậu điều kiện	Người dùng xem được trạng thái hiện tại hoặc dữ liệu gần nhất của thiết bị trên ứng dụng.

Bảng 2.3: Đặc tả use case Xem trạng thái thiết bị

2.3.4 Đặc tả use case Gửi dữ liệu cảm biến

Use case “Gửi dữ liệu cảm biến” mô tả quá trình thiết bị IoT thu thập dữ liệu từ cảm biến và gửi dữ liệu về máy chủ để phục vụ việc giám sát trạng thái phương tiện.

Mã UseCase	UC04
Tên UseCase	Gửi dữ liệu cảm biến
Tác nhân	Device IoT
Mục đích sử dụng	Gửi dữ liệu cảm biến, trạng thái thiết bị và vị trí phương tiện về máy chủ để hệ thống lưu trữ và xử lý.
Sự kiện kích hoạt	Thiết bị IoT thu thập được dữ liệu mới hoặc đến chu kỳ gửi dữ liệu định kỳ.
Điều kiện tiên quyết	Thiết bị đã được kích hoạt, có kết nối mạng và có thông tin xác thực hợp lệ.

Luồng sự kiện chính (thành công)	1. Device IoT thu thập dữ liệu từ cảm biến. 2. Device IoT lấy thông tin vị trí và trạng thái hiện tại của thiết bị. 3. Device IoT đóng gói dữ liệu cần gửi. 4. Device IoT gửi dữ liệu về máy chủ thông qua MQTT. 5. Hệ thống xác thực thiết bị gửi dữ liệu. 6. Hệ thống tiếp nhận và lưu dữ liệu vào cơ sở dữ liệu. 7. Hệ thống cập nhật trạng thái mới nhất của thiết bị.
Luồng sự kiện thay thế	4.1 Thiết bị mất kết nối mạng. Dữ liệu không được gửi lên máy chủ tại thời điểm đó. 5.1 Thông tin xác thực của thiết bị không hợp lệ. Hệ thống từ chối dữ liệu. 6.1 Dữ liệu gửi lên không đúng định dạng. Hệ thống bỏ qua dữ liệu và ghi nhận lỗi xử lý.
Hậu điều kiện	Dữ liệu cảm biến, vị trí và trạng thái thiết bị được lưu trữ trên hệ thống để phục vụ giám sát, hiển thị và phát hiện bất thường.

Bảng 2.4: Đặc tả use case Gửi dữ liệu cảm biến

2.3.5 Đặc tả use case Phát hiện nguy cơ trộm

Use case “Phát hiện nguy cơ trộm” mô tả quá trình hệ thống xử lý dữ liệu từ thiết bị IoT để xác định các dấu hiệu bất thường liên quan đến phương tiện.

Mã UseCase	UC05
Tên UseCase	Phát hiện nguy cơ trộm
Tác nhân	Device IoT
Mục đích sử dụng	Phân tích dữ liệu cảm biến và trạng thái phương tiện để phát hiện nguy cơ trộm hoặc các dấu hiệu bất thường.
Sự kiện kích hoạt	Hệ thống nhận được dữ liệu cảm biến mới từ thiết bị IoT.
Điều kiện tiên quyết	Thiết bị đã được kích hoạt, dữ liệu gửi lên hợp lệ và chế độ chống trộm của thiết bị đang được bật.

Luồng sự kiện chính (thành công)	1. Device IoT gửi dữ liệu cảm biến và trạng thái thiết bị về hệ thống. 2. Hệ thống tiếp nhận và xác thực dữ liệu từ thiết bị. 3. Hệ thống kiểm tra trạng thái chống trộm của thiết bị. 4. Hệ thống phân tích dữ liệu cảm biến và vị trí phương tiện. 5. Hệ thống xác định dữ liệu có dấu hiệu bất thường. 6. Hệ thống tạo bản ghi cảnh báo trộm. 7. Hệ thống chuyển thông tin cảnh báo đến chức năng gửi cảnh báo cho người dùng.
Luồng sự kiện thay thế	2.1 Dữ liệu gửi lên không hợp lệ. Hệ thống bỏ qua dữ liệu và không thực hiện phát hiện nguy cơ trộm. 3.1 Chế độ chống trộm đang tắt. Hệ thống chỉ lưu dữ liệu và cập nhật trạng thái thiết bị. 5.1 Dữ liệu không có dấu hiệu bất thường. Hệ thống cập nhật trạng thái thiết bị bình thường.
Hậu điều kiện	Nếu phát hiện nguy cơ trộm, hệ thống tạo cảnh báo và lưu lịch sử cảnh báo. Nếu không phát hiện bất thường, hệ thống chỉ cập nhật trạng thái thiết bị.

Bảng 2.5: Đặc tả use case Phát hiện nguy cơ trộm

2.3.6 Đặc tả use case Nhận cảnh báo trộm

Use case “Nhận cảnh báo trộm” mô tả quá trình hệ thống gửi thông báo đến người dùng khi phát hiện phương tiện có dấu hiệu bất thường.

Mã UseCase	UC06
Tên UseCase	Nhận cảnh báo trộm
Tác nhân	Người dùng
Mục đích sử dụng	Thông báo kịp thời cho người dùng khi hệ thống phát hiện nguy cơ trộm hoặc dấu hiệu bất thường từ phương tiện.
Sự kiện kích hoạt	Hệ thống phát hiện nguy cơ trộm từ dữ liệu do thiết bị IoT gửi lên.
Điều kiện tiên quyết	Người dùng đã đăng nhập, thiết bị đã được kích hoạt và ứng dụng đã được cấp quyền nhận thông báo.

Luồng sự kiện chính (thành công)	1. Hệ thống phát hiện nguy cơ trộm từ dữ liệu cảm biến. 2. Hệ thống tạo nội dung cảnh báo. 3. Hệ thống lưu thông tin cảnh báo vào lịch sử. 4. Hệ thống gửi thông báo cảnh báo đến ứng dụng di động của người dùng. 5. Ứng dụng hiển thị thông báo cảnh báo trộm. 6. Người dùng mở thông báo để xem chi tiết cảnh báo. 7. Hệ thống hiển thị thông tin cảnh báo, trạng thái thiết bị và vị trí phương tiện.
Luồng sự kiện thay thế	4.1 Ứng dụng chưa được cấp quyền nhận thông báo. Hệ thống vẫn lưu cảnh báo trong lịch sử nhưng người dùng không nhận được thông báo đẩy. 4.2 Thiết bị di động của người dùng không có kết nối Internet. Thông báo có thể được hiển thị khi thiết bị kết nối mạng trở lại. 6.1 Người dùng không mở thông báo. Cảnh báo vẫn được lưu trong lịch sử để người dùng xem lại sau.
Hậu điều kiện	Người dùng nhận được cảnh báo trộm hoặc có thể xem lại cảnh báo trong lịch sử cảnh báo của ứng dụng.

Bảng 2.6: Đặc tả use case Nhận cảnh báo trộm

2.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng đã trình bày ở các phần trước, hệ thống chống trộm xe IoT cần đáp ứng một số yêu cầu phi chức năng nhằm đảm bảo khả năng hoạt động ổn định, an toàn và phù hợp với điều kiện sử dụng thực tế. Các yêu cầu này liên quan đến hiệu năng, độ tin cậy, bảo mật, tính dễ sử dụng, khả năng bảo trì và khả năng mở rộng của hệ thống.

Nhóm yêu cầu	Mô tả
Hiệu năng	Hệ thống cần tiếp nhận và xử lý dữ liệu từ thiết bị IoT trong thời gian ngắn. Khi phát hiện dấu hiệu bất thường, cảnh báo cần được gửi đến người dùng kịp thời để hỗ trợ xử lý tình huống.

Độ tin cậy	Hệ thống cần đảm bảo dữ liệu cảm biến, vị trí và cảnh báo được ghi nhận chính xác trong quá trình thiết bị hoạt động. Trong trường hợp mất kết nối tạm thời, hệ thống cần có khả năng xử lý phù hợp và cập nhật lại trạng thái khi kết nối được khôi phục.
Bảo mật	Người dùng cần đăng nhập trước khi sử dụng các chức năng quản lý và giám sát thiết bị. Thiết bị IoT cần được xác thực khi gửi dữ liệu lên hệ thống nhằm hạn chế việc dữ liệu giả mạo hoặc không hợp lệ được xử lý.
Tính dễ sử dụng	Giao diện ứng dụng di động cần được thiết kế đơn giản, dễ hiểu và phù hợp với người dùng phổ thông. Các chức năng quan trọng như xem trạng thái thiết bị, kích hoạt thiết bị và nhận cảnh báo cần được trình bày rõ ràng, dễ thao tác.
Khả năng bảo trì	Mã nguồn của hệ thống cần được tổ chức rõ ràng theo từng thành phần như ứng dụng di động, máy chủ và thiết bị IoT. Việc phân chia này giúp thuận tiện cho quá trình kiểm tra, sửa lỗi và phát triển thêm chức năng trong tương lai.
Khả năng mở rộng	Hệ thống cần có khả năng mở rộng để hỗ trợ nhiều người dùng và nhiều thiết bị hơn. Kiến trúc hệ thống cần cho phép bổ sung thêm các loại cảm biến, chức năng cảnh báo hoặc phương thức giám sát mới khi cần thiết.
Yêu cầu kỹ thuật	Hệ thống cần hỗ trợ giao tiếp giữa thiết bị IoT và máy chủ, lưu trữ dữ liệu thiết bị, dữ liệu cảm biến, vị trí và lịch sử cảnh báo. Ứng dụng di động cần kết nối được với máy chủ để hiển thị dữ liệu và nhận thông báo từ hệ thống.

Bảng 2.7: Yêu cầu phi chức năng của hệ thống

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Chương 2 đã trình bày quá trình khảo sát hiện trạng, phân tích yêu cầu và đặc tả các chức năng chính của hệ thống chống trộm xe ứng dụng công nghệ IoT. Từ các yêu cầu đã xác định, hệ thống cần có khả năng thu thập dữ liệu từ thiết bị gắn trên phương tiện, truyền dữ liệu về máy chủ, lưu trữ và xử lý dữ liệu, đồng thời cung cấp giao diện ứng dụng di động để người dùng giám sát trạng thái thiết bị và nhận cảnh báo.

Chương này trình bày các nền tảng lý thuyết và công nghệ được sử dụng để xây dựng hệ thống. Các công nghệ chính bao gồm NestJS cho phía máy chủ, PostgreSQL cho cơ sở dữ liệu, Flutter và ngôn ngữ Dart cho ứng dụng di động, ESP32 cho thiết bị điều khiển trung tâm, module A7680C cho kết nối mạng di động và module GPS NEO-6M cho chức năng định vị. Đối với từng công nghệ, chương trình bày vai trò trong hệ thống, lý do lựa chọn và mối liên hệ với các yêu cầu đã phân tích ở chương trước.

3.1 Kiến trúc công nghệ tổng thể

Hệ thống trong đề án được xây dựng theo mô hình gồm ba nhóm thành phần chính: thiết bị IoT gắn trên phương tiện, hệ thống máy chủ và ứng dụng di động. Thiết bị IoT có nhiệm vụ thu thập dữ liệu cảm biến, dữ liệu vị trí và trạng thái hoạt động của phương tiện. Các dữ liệu này được truyền về máy chủ để xử lý, lưu trữ và cung cấp cho ứng dụng di động. Ứng dụng di động đóng vai trò giao diện tương tác giữa người dùng và hệ thống, cho phép người dùng quản lý thiết bị, xem trạng thái phương tiện, theo dõi vị trí và nhận cảnh báo khi có dấu hiệu bất thường.

Về phía thiết bị, ESP32 được sử dụng làm vi điều khiển trung tâm để điều khiển các module phần cứng và xử lý dữ liệu ban đầu. Module GPS NEO-6M được dùng để thu thập thông tin vị trí của phương tiện. Module A7680C được sử dụng để hỗ trợ kết nối mạng di động, giúp thiết bị có thể truyền dữ liệu về hệ thống trong điều kiện không phụ thuộc vào mạng WiFi. Về phía máy chủ, NestJS được lựa chọn để xây dựng các API, xử lý dữ liệu từ thiết bị và phục vụ ứng dụng di động. PostgreSQL được sử dụng để lưu trữ dữ liệu người dùng, thiết bị, dữ liệu cảm biến, vị trí và lịch sử cảnh báo. Ứng dụng di động được phát triển bằng Flutter và Dart nhằm hỗ trợ triển khai giao diện người dùng trên thiết bị di động.

3.2 Công nghệ xây dựng khối máy chủ

Khối máy chủ là thành phần trung gian giữa thiết bị IoT, ứng dụng di động và cơ sở dữ liệu. Trong hệ thống chống trộm xe IoT, khối máy chủ có nhiệm vụ cung cấp

các API cho ứng dụng di động, tiếp nhận dữ liệu từ thiết bị, xử lý dữ liệu cảm biến, quản lý thông tin người dùng, quản lý thiết bị và hỗ trợ gửi cảnh báo khi phát hiện dấu hiệu bất thường. Để xây dựng khối này, đồ án sử dụng NestJS làm framework phát triển phía server và PostgreSQL làm hệ quản trị cơ sở dữ liệu.

3.2.1 NestJS

a, Giới thiệu công nghệ

NestJS là một framework dùng để xây dựng ứng dụng phía máy chủ trên nền tảng Node.js. Framework này sử dụng TypeScript và hỗ trợ cách tổ chức mã nguồn theo module, controller, service và provider. Cách tổ chức này giúp tách biệt rõ vai trò giữa các thành phần xử lý yêu cầu, xử lý nghiệp vụ và truy cập dữ liệu.

b, Ứng dụng trong đồ án

Trong đồ án, NestJS được sử dụng để xây dựng server trung tâm của hệ thống chống trộm xe IoT. Server có nhiệm vụ xử lý các yêu cầu từ ứng dụng di động như đăng ký, đăng nhập, quản lý tài khoản, kích hoạt thiết bị, xem trạng thái thiết bị và xem lịch sử dữ liệu. Bên cạnh đó, server cũng tiếp nhận dữ liệu do thiết bị IoT gửi lên, kiểm tra tính hợp lệ của thiết bị, lưu dữ liệu vào cơ sở dữ liệu và xử lý các điều kiện phát sinh cảnh báo.

NestJS được lựa chọn vì có cấu trúc rõ ràng, phù hợp với hệ thống gồm nhiều nhóm chức năng khác nhau. Các chức năng của hệ thống có thể được tách thành các module riêng như module xác thực, module người dùng, module thiết bị, module dữ liệu cảm biến và module cảnh báo. Việc tổ chức mã nguồn theo module giúp quá trình phát triển, kiểm thử, bảo trì và mở rộng hệ thống thuận lợi hơn. So với việc sử dụng trực tiếp Express.js, NestJS cung cấp sẵn cấu trúc phát triển chặt chẽ hơn, phù hợp với yêu cầu xây dựng server có nhiều thành phần trong đồ án.

3.2.2 PostgreSQL

a, Giới thiệu công nghệ

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở. Hệ quản trị này hỗ trợ lưu trữ dữ liệu dưới dạng bảng, thiết lập quan hệ giữa các bảng và truy vấn dữ liệu bằng ngôn ngữ SQL. PostgreSQL phù hợp với các hệ thống cần lưu trữ dữ liệu có cấu trúc và yêu cầu tính nhất quán trong quá trình xử lý.

b, Ứng dụng trong đồ án

Trong đồ án, PostgreSQL được sử dụng để lưu trữ dữ liệu của hệ thống, bao gồm thông tin người dùng, thông tin thiết bị, thông tin phương tiện, dữ liệu cảm biến, dữ liệu vị trí và lịch sử cảnh báo. Các dữ liệu này được server truy xuất và cập nhật trong quá trình người dùng sử dụng ứng dụng hoặc khi thiết bị IoT gửi dữ liệu về

hệ thống.

PostgreSQL được lựa chọn vì dữ liệu trong hệ thống có quan hệ rõ ràng giữa các thực thể. Ví dụ, một người dùng có thể quản lý nhiều thiết bị, mỗi thiết bị gắn với một phương tiện và có nhiều bản ghi dữ liệu cảm biến hoặc cảnh báo. Việc sử dụng cơ sở dữ liệu quan hệ giúp tổ chức dữ liệu chặt chẽ, dễ truy vấn và đảm bảo tính toàn vẹn dữ liệu. So với các cơ sở dữ liệu NoSQL, PostgreSQL phù hợp hơn với phạm vi đề án do hệ thống cần quản lý dữ liệu có cấu trúc và thường xuyên truy vấn theo người dùng, thiết bị hoặc thời gian.

3.3 Công nghệ xây dựng ứng dụng di động

Ứng dụng di động là thành phần giao tiếp trực tiếp với người dùng trong hệ thống chống trộm xe IoT. Thông qua ứng dụng, người dùng có thể đăng ký, đăng nhập, quản lý tài khoản, kích hoạt thiết bị, theo dõi trạng thái phương tiện, xem vị trí xe, xem lịch sử dữ liệu và nhận cảnh báo khi hệ thống phát hiện dấu hiệu bất thường. Trong đề án, ứng dụng di động được xây dựng bằng Flutter và ngôn ngữ lập trình Dart.

3.3.1 Flutter

a, Giới thiệu công nghệ

Flutter là một bộ công cụ phát triển giao diện người dùng do Google phát triển, cho phép xây dựng ứng dụng đa nền tảng từ một mã nguồn chung. Flutter hỗ trợ phát triển ứng dụng cho nhiều môi trường như Android, iOS, Web và Desktop. Đặc điểm nổi bật của Flutter là hệ thống giao diện được xây dựng dựa trên các widget, giúp việc thiết kế và tái sử dụng giao diện trở nên thuận tiện hơn.

Flutter cung cấp khả năng hiển thị giao diện linh hoạt, hiệu năng tốt và hỗ trợ cơ chế hot reload trong quá trình phát triển. Cơ chế này cho phép lập trình viên xem nhanh sự thay đổi của giao diện và logic ứng dụng mà không cần biên dịch lại toàn bộ chương trình. Nhờ đó, quá trình phát triển và kiểm thử giao diện ứng dụng có thể được thực hiện nhanh chóng hơn.

b, Ứng dụng trong đề án

Trong đề án, Flutter được sử dụng để xây dựng ứng dụng di động cho người dùng. Ứng dụng này cung cấp các màn hình chính như đăng nhập, đăng ký, quản lý thiết bị cá nhân, kích hoạt thiết bị, xem trạng thái thiết bị, xem vị trí phương tiện, xem lịch sử dữ liệu và nhận cảnh báo trộm. Đây là các chức năng đã được phân tích trong Chương 2, đóng vai trò giúp người dùng tương tác với hệ thống một cách trực quan và thuận tiện.

Flutter được lựa chọn vì phù hợp với yêu cầu xây dựng ứng dụng di động có giao

diện rõ ràng, dễ sử dụng và có khả năng mở rộng. Với cơ chế phát triển dựa trên widget, các thành phần giao diện như nút bấm, biểu mẫu nhập liệu, danh sách thiết bị, màn hình bản đồ và màn hình cảnh báo có thể được tổ chức thành các thành phần riêng biệt. Điều này giúp mã nguồn ứng dụng dễ quản lý, dễ bảo trì và thuận tiện khi cần bổ sung thêm chức năng mới.

Bên cạnh đó, Flutter hỗ trợ tốt việc giao tiếp với server thông qua các API. Trong hệ thống của đồ án, ứng dụng di động gửi yêu cầu đến server để thực hiện các chức năng như đăng nhập, lấy danh sách thiết bị, cập nhật thông tin thiết bị, lấy dữ liệu trạng thái và lịch sử cảnh báo. Server sau đó trả kết quả về cho ứng dụng để hiển thị cho người dùng. Cách tổ chức này giúp ứng dụng di động đóng vai trò là lớp giao diện, còn việc xử lý nghiệp vụ và lưu trữ dữ liệu được thực hiện ở phía máy chủ.

3.3.2 Dart

a, Giới thiệu công nghệ

Dart là ngôn ngữ lập trình được sử dụng chính trong Flutter. Dart là ngôn ngữ hướng đối tượng, hỗ trợ kiểu dữ liệu tĩnh và cung cấp cú pháp tương đối rõ ràng, phù hợp cho việc phát triển ứng dụng giao diện người dùng. Dart cũng hỗ trợ lập trình bất đồng bộ thông qua các cơ chế như *Future* và *async/await*, giúp xử lý các tác vụ cần chờ phản hồi từ mạng hoặc từ hệ thống bên ngoài.

Trong phát triển ứng dụng di động, Dart thường được sử dụng để xây dựng giao diện, xử lý dữ liệu, gọi API, quản lý trạng thái và điều khiển luồng hoạt động của ứng dụng. Khi kết hợp với Flutter, Dart giúp lập trình viên xây dựng các thành phần giao diện và xử lý logic ứng dụng trong cùng một môi trường phát triển thống nhất.

b, Ứng dụng trong đồ án

Trong đồ án, Dart được sử dụng để lập trình toàn bộ logic của ứng dụng di động. Các chức năng như xử lý đăng nhập, gửi yêu cầu kích hoạt thiết bị, lấy dữ liệu trạng thái thiết bị, hiển thị danh sách thiết bị, xử lý dữ liệu vị trí và tiếp nhận thông báo cảnh báo đều được triển khai bằng Dart.

Dart được lựa chọn vì đây là ngôn ngữ chính thức của Flutter, có khả năng kết hợp chặt chẽ với hệ thống widget của Flutter và hỗ trợ tốt các thao tác bất đồng bộ. Điều này phù hợp với ứng dụng trong đồ án, vì ứng dụng cần thường xuyên gửi yêu cầu đến server, nhận dữ liệu phản hồi, cập nhật giao diện và xử lý các sự kiện từ người dùng. Nhờ cơ chế bất đồng bộ, ứng dụng có thể thực hiện các thao tác mạng mà không làm gián đoạn trải nghiệm sử dụng của người dùng.

Việc sử dụng Dart cùng Flutter giúp ứng dụng di động đáp ứng các yêu cầu đã đặt ra ở Chương 2, bao gồm tính dễ sử dụng, khả năng hiển thị trạng thái thiết bị,

hỗ trợ quản lý thiết bị cá nhân và nhận cảnh báo trộm. Đồng thời, cách tổ chức mã nguồn bằng các lớp, widget và service cũng giúp ứng dụng có khả năng bảo trì và mở rộng trong các giai đoạn phát triển tiếp theo.

3.4 Công nghệ xây dựng khối thiết bị IoT

Khối thiết bị IoT là thành phần được gắn trực tiếp trên phương tiện, có nhiệm vụ thu thập dữ liệu từ các cảm biến, xác định vị trí của xe và truyền dữ liệu về máy chủ. Đây là thành phần đầu vào quan trọng của hệ thống chống trộm xe, vì các dữ liệu thu thập được từ thiết bị là cơ sở để hệ thống giám sát trạng thái phương tiện và phát hiện các dấu hiệu bất thường. Trong đồ án, khối thiết bị IoT được xây dựng dựa trên vi điều khiển ESP32, module truyền thông A7680C và module định vị GPS NEO-6M.

3.4.1 ESP32

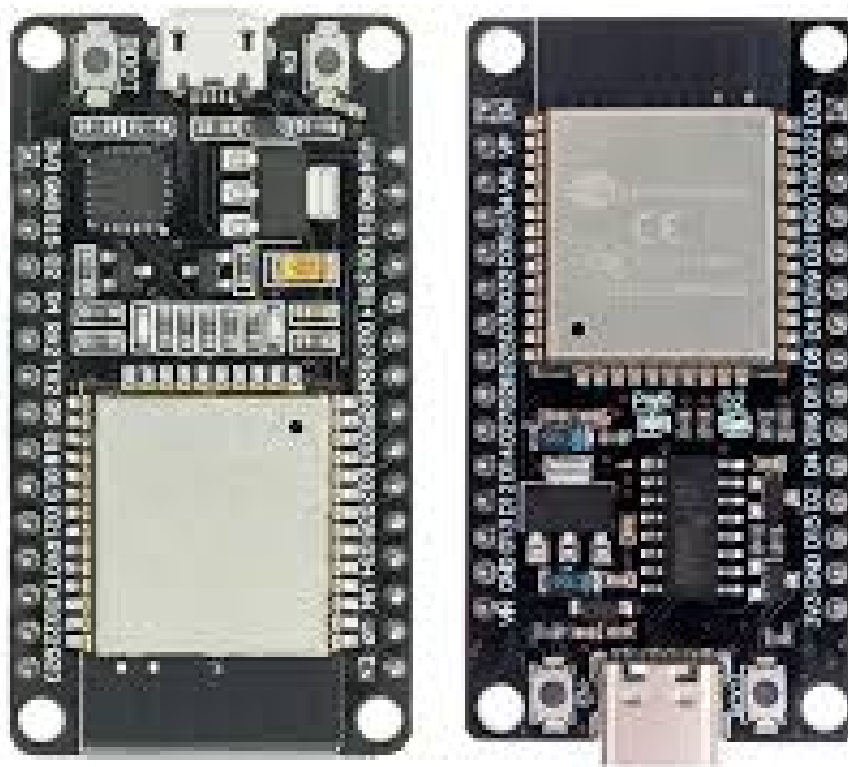
a, Giới thiệu công nghệ

ESP32 là một dòng vi điều khiển được sử dụng phổ biến trong các hệ thống IoT nhờ kích thước nhỏ, chi phí thấp và khả năng xử lý phù hợp với các ứng dụng nhúng. ESP32 hỗ trợ nhiều giao tiếp phần cứng như UART, SPI, I2C, ADC và GPIO, giúp kết nối với nhiều loại cảm biến và module ngoại vi khác nhau. Ngoài ra, ESP32 cũng được sử dụng rộng rãi trong các hệ thống cần thu thập dữ liệu, xử lý tín hiệu đơn giản và truyền dữ liệu đến các thành phần khác.

b, Ứng dụng trong đồ án

Trong đồ án, ESP32 đóng vai trò là bộ điều khiển trung tâm của thiết bị chống trộm gắn trên xe. ESP32 có nhiệm vụ đọc dữ liệu từ cảm biến, nhận dữ liệu vị trí từ module GPS, điều khiển quá trình gửi dữ liệu thông qua module truyền thông và quản lý trạng thái hoạt động của thiết bị. Khi thiết bị phát hiện các tín hiệu bất thường hoặc đến chu kỳ gửi dữ liệu, ESP32 sẽ tổng hợp dữ liệu cần thiết và gửi về máy chủ để xử lý.

ESP32 được lựa chọn vì phù hợp với yêu cầu xây dựng thiết bị IoT chi phí thấp, nhỏ gọn và dễ lập trình. Với số lượng chân giao tiếp và khả năng kết nối ngoại vi đa dạng, ESP32 có thể kết hợp với các module như A7680C và NEO-6M trong cùng một thiết bị. Việc sử dụng ESP32 giúp hệ thống đáp ứng các yêu cầu đã đặt ra ở Chương 2, bao gồm thu thập dữ liệu cảm biến, gửi trạng thái thiết bị, cập nhật vị trí và hỗ trợ phát hiện nguy cơ trộm.



Hình 3.1: Vi điều khiển Esp32

3.4.2 Module A7680C

a, Giới thiệu công nghệ

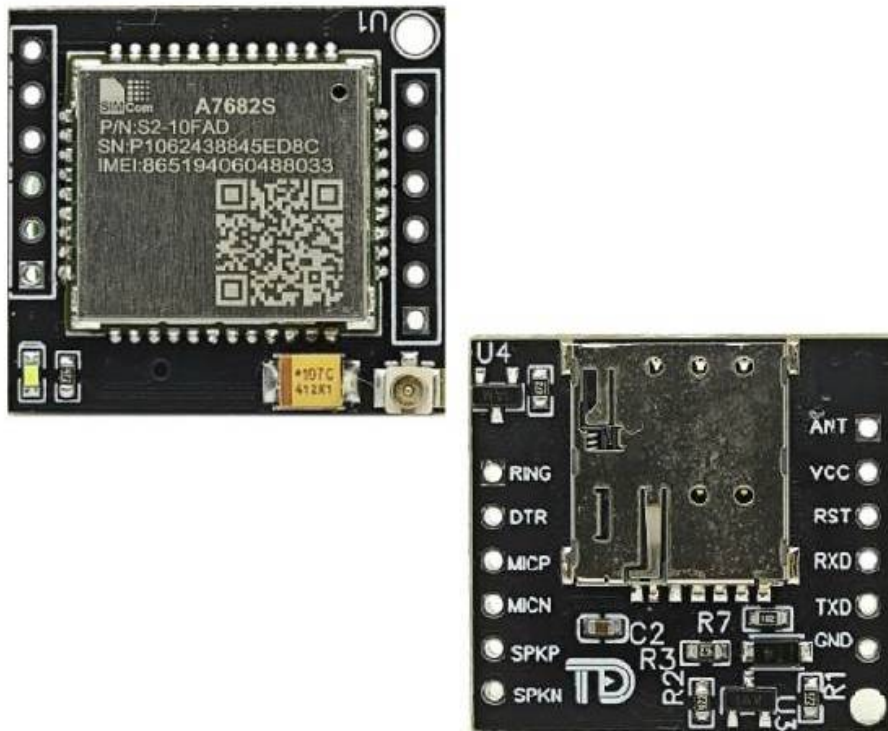
A7680C là module truyền thông di động hỗ trợ kết nối mạng thông qua SIM. Module này thường được sử dụng trong các hệ thống IoT cần truyền dữ liệu từ xa mà không phụ thuộc vào mạng WiFi. Thông qua giao tiếp UART, vi điều khiển có thể điều khiển module bằng các lệnh AT để thực hiện các thao tác như kiểm tra trạng thái mạng, thiết lập kết nối Internet và truyền dữ liệu đến máy chủ.

b, Ứng dụng trong đồ án

Trong đồ án, module A7680C được sử dụng để hỗ trợ thiết bị IoT truyền dữ liệu về máy chủ thông qua mạng di động. Dữ liệu được gửi có thể bao gồm trạng thái thiết bị, dữ liệu cảm biến, dữ liệu vị trí và các thông tin liên quan đến quá trình giám sát phương tiện. Nhờ sử dụng mạng di động, thiết bị có thể hoạt động trong nhiều vị trí khác nhau mà không phụ thuộc vào phạm vi phủ sóng của WiFi.

A7680C được lựa chọn vì phù hợp với bài toán chống trộm xe, trong đó phương tiện có thể di chuyển ngoài phạm vi mạng cục bộ. So với việc chỉ sử dụng WiFi,

kết nối mạng di động giúp tăng tính linh hoạt và khả năng hoạt động thực tế của hệ thống. Công nghệ này hỗ trợ trực tiếp cho các yêu cầu như gửi dữ liệu cảm biến, cập nhật vị trí và gửi trạng thái thiết bị về máy chủ trong quá trình vận hành.



Hình 3.2: Module A7680C

3.4.3 Module GPS NEO-6M

a, Giới thiệu công nghệ

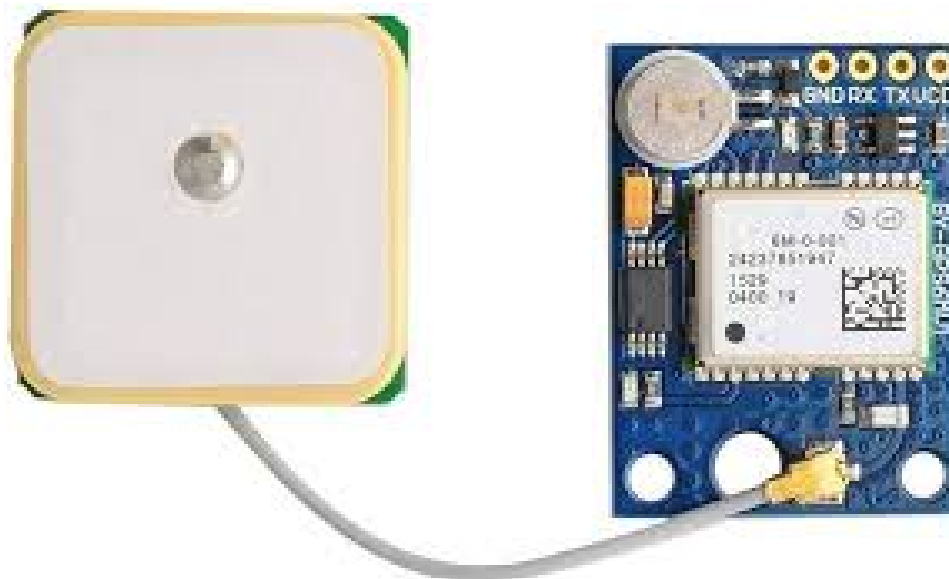
NEO-6M là module định vị GPS thường được sử dụng trong các hệ thống nhúng và IoT. Module này có khả năng nhận tín hiệu từ hệ thống vệ tinh GPS để xác định vị trí của thiết bị, bao gồm các thông tin như kinh độ, vĩ độ, thời gian và một số dữ liệu định vị khác. NEO-6M giao tiếp với vi điều khiển thông qua UART, giúp việc tích hợp với các nền tảng như ESP32 tương đối thuận tiện.

b, Ứng dụng trong đồ án

Trong đồ án, module GPS NEO-6M được sử dụng để thu thập thông tin vị trí của phương tiện. Dữ liệu vị trí được ESP32 đọc từ module GPS, sau đó gửi về máy chủ thông qua module truyền thông. Máy chủ lưu trữ dữ liệu này và cung cấp cho ứng dụng di động để người dùng có thể theo dõi vị trí hiện tại hoặc vị trí gần nhất

của xe.

NEO-6M được lựa chọn vì có chi phí thấp, dễ sử dụng và phù hợp với mô hình thử nghiệm của hệ thống. Việc tích hợp module GPS giúp hệ thống đáp ứng yêu cầu theo dõi vị trí phương tiện đã phân tích ở Chương 2. Trong trường hợp phát hiện nguy cơ trộm, dữ liệu vị trí cũng hỗ trợ người dùng xác định vị trí của xe để có phương án xử lý kịp thời.



Hình 3.3: Module NEO-6M

3.4.4 Tổng kết khối thiết bị IoT

Sự kết hợp giữa ESP32, A7680C và NEO-6M tạo thành khối thiết bị IoT có khả năng thu thập dữ liệu, xác định vị trí và truyền dữ liệu về máy chủ. ESP32 đảm nhiệm vai trò xử lý trung tâm, A7680C đảm nhiệm vai trò truyền thông qua mạng di động, còn NEO-6M cung cấp dữ liệu định vị. Cách kết hợp này phù hợp với mục tiêu xây dựng hệ thống chống trộm xe có chi phí thấp, có khả năng hoạt động trên thiết bị thật và đáp ứng các chức năng giám sát, định vị, cảnh báo của đề án.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Thiết kế kiến trúc

Chương 3 đã trình bày các nền tảng lý thuyết và công nghệ được sử dụng trong quá trình xây dựng hệ thống, bao gồm công nghệ phát triển máy chủ, ứng dụng di động, cơ sở dữ liệu và khối thiết bị IoT. Chương này trình bày quá trình phân tích, thiết kế, triển khai và đánh giá hệ thống chống trộm xe IoT.

Thiết kế kiến trúc là bước quan trọng nhằm xác định cách tổ chức tổng thể của hệ thống và mối quan hệ giữa các thành phần chính. Đối với hệ thống trong đề án, kiến trúc cần đảm bảo sự phối hợp giữa thiết bị IoT gắn trên phương tiện, máy chủ xử lý trung tâm, cơ sở dữ liệu và ứng dụng di động của người dùng. Việc thiết kế kiến trúc rõ ràng giúp hệ thống dễ triển khai, dễ bảo trì và thuận tiện cho việc mở rộng trong tương lai.

Trong mục này, đề án trước hết trình bày lựa chọn kiến trúc phần mềm được sử dụng cho hệ thống. Tiếp theo, kiến trúc tổng quan của hệ thống được mô tả nhằm làm rõ vai trò của từng thành phần và luồng trao đổi dữ liệu giữa các thành phần. Cuối cùng, mục này trình bày thiết kế chi tiết các gói chức năng chính, làm cơ sở cho việc triển khai server, ứng dụng di động và thiết bị IoT trong các phần tiếp theo.

4.1.1 Lựa chọn kiến trúc phần mềm

Đối với hệ thống chống trộm xe IoT, đề án lựa chọn kiến trúc client-server kết hợp mô hình IoT nhiều tầng. Kiến trúc này phù hợp với hệ thống do các thành phần tham gia có vai trò khác nhau, bao gồm thiết bị IoT gắn trên phương tiện, máy chủ xử lý trung tâm, cơ sở dữ liệu và ứng dụng di động của người dùng. Việc tổ chức hệ thống theo các tầng giúp tách biệt nhiệm vụ giữa thu thập dữ liệu, xử lý nghiệp vụ, lưu trữ dữ liệu và hiển thị thông tin cho người dùng.

Trong kiến trúc được lựa chọn, tầng thiết bị chịu trách nhiệm thu thập dữ liệu từ phần cứng, bao gồm dữ liệu cảm biến, trạng thái thiết bị và thông tin vị trí. Thiết bị IoT gửi dữ liệu về máy chủ thông qua giao thức MQTT. Giao thức này phù hợp với hệ thống IoT do hỗ trợ truyền dữ liệu nhẹ, phù hợp với các thiết bị nhúng và các tình huống cần gửi dữ liệu thường xuyên từ thiết bị lên hệ thống trung tâm.

Tầng máy chủ được xây dựng bằng NestJS và đóng vai trò trung gian giữa thiết bị IoT, cơ sở dữ liệu và ứng dụng di động. Máy chủ tiếp nhận dữ liệu từ thiết bị thông qua module MQTT, xử lý dữ liệu cảm biến, phát hiện nguy cơ trộm, lưu trữ dữ liệu và gửi cảnh báo đến người dùng khi cần thiết. Bên cạnh đó, máy chủ cung

cấp các REST API để ứng dụng di động thực hiện các chức năng như đăng nhập, quản lý tài khoản, kích hoạt thiết bị, xem trạng thái thiết bị và xem lịch sử dữ liệu. Ngoài REST API, hệ thống còn sử dụng WebSocket để hỗ trợ cập nhật dữ liệu trạng thái theo thời gian thực cho ứng dụng di động.

Dữ liệu của hệ thống được lưu trữ trong PostgreSQL và được truy cập thông qua Prisma ORM. Việc sử dụng Prisma giúp thao tác với cơ sở dữ liệu rõ ràng hơn, đồng thời tạo lớp trung gian giữa mã nguồn server và hệ quản trị cơ sở dữ liệu. Các dữ liệu chính được lưu trữ bao gồm thông tin người dùng, thông tin thiết bị, dữ liệu cảm biến, dữ liệu vị trí và lịch sử cảnh báo.

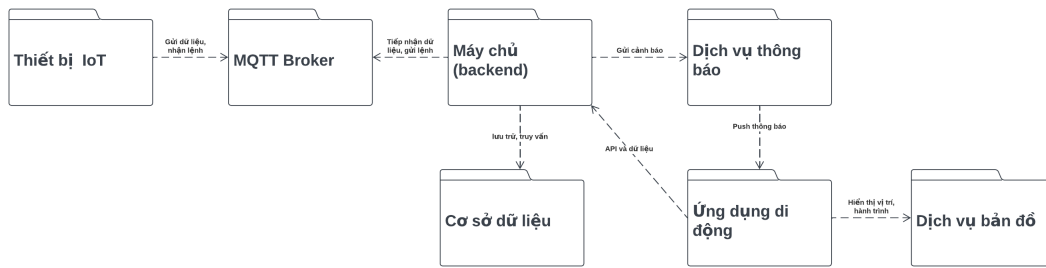
Tầng ứng dụng di động được xây dựng bằng Flutter và Dart, đóng vai trò là giao diện tương tác trực tiếp với người dùng. Ứng dụng gọi các REST API do server cung cấp để thực hiện các chức năng nghiệp vụ và sử dụng WebSocket để nhận các cập nhật trạng thái cần hiển thị kịp thời. Trong quá trình tổ chức mã nguồn, ứng dụng được xây dựng theo mô hình MVC/MVVM đơn giản nhằm tách biệt tương đối giữa giao diện, dữ liệu và xử lý logic. Đối với chức năng bản đồ, ứng dụng sử dụng OpenStreetMap để hiển thị vị trí phương tiện cho người dùng.

Ở phía server, mã nguồn được tổ chức thành các module chức năng như auth, user, device, mqtt, sensor-data, firebase và middleware. Module auth phụ trách xác thực và đăng nhập người dùng. Module user xử lý thông tin tài khoản. Module device quản lý thông tin thiết bị và quá trình kích hoạt thiết bị. Module mqtt tiếp nhận dữ liệu từ thiết bị IoT. Module sensor-data xử lý và lưu trữ dữ liệu cảm biến. Module firebase hỗ trợ gửi thông báo cảnh báo đến ứng dụng di động. Module middleware hỗ trợ xử lý các bước trung gian như kiểm tra xác thực và kiểm soát truy cập.

4.1.2 Thiết kế tổng quan

4.1.3 Thiết kế tổng quan hệ thống

Sau khi lựa chọn kiến trúc client-server kết hợp mô hình IoT nhiều tầng, hệ thống được thiết kế tổng quan thành các gói thành phần chính nhằm tách biệt vai trò và trách nhiệm của từng khối chức năng. Biểu đồ gói tổng quan của hệ thống được trình bày trong Hình 4.1.



Hình 4.1: Biểu đồ gói tổng quan của hệ thống

Như thể hiện trong Hình 4.1, hệ thống được chia thành bảy gói chính gồm: Thiết bị IoT, Ứng dụng di động, Hệ thống máy chủ, Cơ sở dữ liệu, Dịch vụ MQTT, Dịch vụ thông báo và Dịch vụ bản đồ. Mỗi gói đảm nhiệm một vai trò riêng trong quá trình thu thập dữ liệu, xử lý nghiệp vụ, lưu trữ dữ liệu và hiển thị thông tin cho người dùng.

Gói Thiết bị IoT đại diện cho thiết bị phần cứng được gắn trên phương tiện. Thành phần này có nhiệm vụ thu thập dữ liệu cảm biến, dữ liệu vị trí và trạng thái hoạt động của xe. Thiết bị IoT không giao tiếp trực tiếp với ứng dụng di động mà gửi dữ liệu về hệ thống thông qua Dịch vụ MQTT. Ngoài việc gửi dữ liệu, thiết bị cũng có thể nhận các lệnh điều khiển hoặc cấu hình từ hệ thống máy chủ thông qua kênh MQTT.

Gói Dịch vụ MQTT đóng vai trò trung gian truyền thông giữa Thiết bị IoT và Hệ thống máy chủ. Việc tách riêng dịch vụ MQTT giúp quá trình trao đổi dữ liệu giữa thiết bị và server trở nên linh hoạt hơn, phù hợp với đặc thù của hệ thống IoT, trong đó thiết bị cần gửi dữ liệu định kỳ hoặc gửi dữ liệu khi phát hiện trạng thái bất thường.

Gói Hệ thống máy chủ là thành phần xử lý trung tâm của hệ thống. Máy chủ tiếp nhận dữ liệu từ Dịch vụ MQTT, xử lý dữ liệu cảm biến, cập nhật trạng thái thiết bị, phát hiện nguy cơ trộm và cung cấp các API cho ứng dụng di động. Ngoài ra, máy chủ còn thực hiện truy vấn và lưu trữ dữ liệu thông qua gói Cơ sở dữ liệu, đồng thời kết nối với Dịch vụ thông báo để gửi cảnh báo đến người dùng khi cần thiết.

Gói Cơ sở dữ liệu có nhiệm vụ lưu trữ các dữ liệu của hệ thống, bao gồm thông tin người dùng, thông tin thiết bị, dữ liệu cảm biến, dữ liệu vị trí và lịch sử cảnh báo. Hệ thống máy chủ là thành phần trực tiếp thực hiện các thao tác lưu trữ và truy vấn dữ liệu, trong khi các thành phần khác không truy cập trực tiếp vào cơ sở dữ liệu. Cách tổ chức này giúp kiểm soát dữ liệu tập trung và đảm bảo tính nhất quán trong quá trình xử lý.

Gói Ứng dụng di động là thành phần giao tiếp trực tiếp với người dùng. Ứng

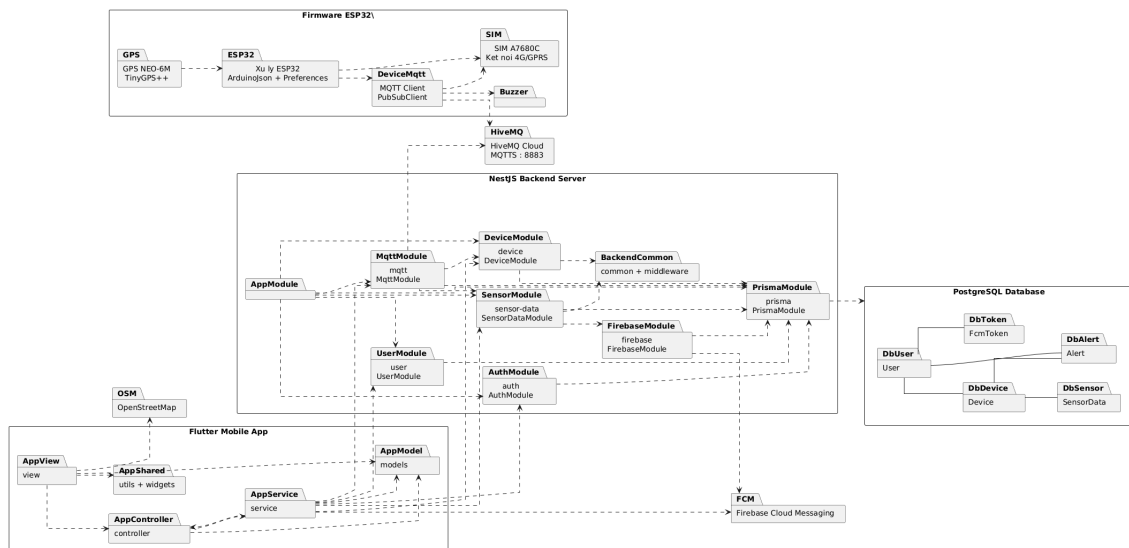
dụng kết nối với Hệ thống máy chủ thông qua API và dữ liệu thời gian thực để thực hiện các chức năng như đăng nhập, quản lý thiết bị, xem trạng thái xe, xem lịch sử dữ liệu và nhận cập nhật trạng thái. Khi người dùng cần theo dõi vị trí hoặc hành trình của phương tiện, ứng dụng sử dụng Dịch vụ bản đồ để hiển thị thông tin vị trí một cách trực quan.

Gói Dịch vụ thông báo được sử dụng để gửi cảnh báo đến ứng dụng di động khi hệ thống phát hiện dấu hiệu bất thường. Khi Hệ thống máy chủ xác định có nguy cơ trộm, máy chủ sẽ gửi yêu cầu đến Dịch vụ thông báo. Sau đó, dịch vụ này thực hiện gửi push notification đến thiết bị di động của người dùng. Cơ chế này giúp người dùng nhận cảnh báo kịp thời ngay cả khi không trực tiếp mở ứng dụng.

Gói Dịch vụ bản đồ hỗ trợ ứng dụng di động trong việc hiển thị vị trí và hành trình của phương tiện. Trong đồ án, dịch vụ bản đồ được sử dụng để trực quan hóa dữ liệu vị trí do thiết bị IoT gửi về, giúp người dùng dễ dàng theo dõi vị trí hiện tại hoặc vị trí gần nhất của xe.

4.1.4 Thiết kế chi tiết gói

Sau khi xác định kiến trúc tổng quan của hệ thống, đồ án tiếp tục thiết kế chi tiết các gói thành phần nhằm làm rõ cách tổ chức bên trong từng khối. Biểu đồ gói chi tiết của hệ thống được trình bày trong Hình 4.2. Biểu đồ này thể hiện các gói chính trong firmware ESP32, ứng dụng di động Flutter, backend NestJS, cơ sở dữ liệu PostgreSQL và các dịch vụ bên ngoài được sử dụng trong hệ thống.



Hình 4.2: Biểu đồ gói chi tiết của hệ thống

Như thể hiện trong Hình 4.2, hệ thống được chia thành bốn nhóm thành phần chính gồm firmware ESP32, ứng dụng di động Flutter, backend server NestJS và cơ sở dữ liệu PostgreSQL. Bên cạnh đó, hệ thống còn sử dụng một số dịch vụ bên

ngoài như HiveMQ Cloud, Firebase Cloud Messaging và OpenStreetMap để hỗ trợ truyền dữ liệu, gửi thông báo và hiển thị bản đồ.

Gói *Firmware ESP32* đại diện cho chương trình nhúng được triển khai trên thiết bị IoT gắn trên phương tiện. Gói xử lý ESP32 sử dụng ArduinoJson và Preferences để xử lý dữ liệu, đóng gói thông tin và lưu một số cấu hình cần thiết trên thiết bị. Gói MQTT Client sử dụng thư viện PubSubClient để kết nối tới HiveMQ Cloud, gửi dữ liệu cảm biến, dữ liệu vị trí và nhận lệnh điều khiển từ hệ thống.

Gói *Flutter Mobile App* là thành phần giao tiếp trực tiếp với người dùng. Ứng dụng được tổ chức theo các gói nhỏ gồm view, controller, service, models, utils và widgets. Gói view chứa các màn hình giao diện như màn hình đăng nhập, quản lý thiết bị, xem trạng thái, xem lịch sử và bản đồ. Gói controller chịu trách nhiệm xử lý logic trung gian giữa giao diện và dữ liệu. Gói service thực hiện các thao tác giao tiếp với backend thông qua API, nhận dữ liệu thời gian thực và xử lý các dịch vụ liên quan. Gói models định nghĩa các lớp dữ liệu được sử dụng trong ứng dụng, chẳng hạn như người dùng, thiết bị, dữ liệu cảm biến và cảnh báo. Các thành phần dùng chung như hàm tiện ích và widget tái sử dụng được đặt trong gói utils và widgets. Cách tổ chức này giúp ứng dụng tách biệt tương đối giữa giao diện, xử lý logic và dữ liệu.

Gói *NestJS Backend Server* là thành phần xử lý trung tâm của hệ thống. AppModule đóng vai trò module gốc, liên kết các module chức năng chính của backend. AuthModule chịu trách nhiệm xử lý đăng nhập, xác thực người dùng và kiểm soát quyền truy cập. UserModule quản lý thông tin tài khoản người dùng. DeviceModule xử lý các chức năng liên quan đến thiết bị như kích hoạt thiết bị, cập nhật thông tin thiết bị và truy xuất danh sách thiết bị của người dùng. MqttModule đảm nhiệm việc kết nối tới HiveMQ Cloud để tiếp nhận dữ liệu do thiết bị IoT gửi lên và gửi lệnh điều khiển đến thiết bị khi cần thiết. SensorDataModule xử lý dữ liệu cảm biến, dữ liệu vị trí, cập nhật trạng thái thiết bị và phát hiện các dấu hiệu bất thường. FirebaseModule hỗ trợ gửi thông báo cảnh báo đến ứng dụng di động thông qua Firebase Cloud Messaging. PrismaModule đóng vai trò lớp truy cập dữ liệu, cung cấp các thao tác làm việc với cơ sở dữ liệu PostgreSQL. Ngoài ra, gói common và middleware chứa các thành phần dùng chung như hàm tiện ích, xử lý xác thực, kiểm tra dữ liệu và các xử lý trung gian khác.

Gói *PostgreSQL Database* lưu trữ các thực thể dữ liệu chính của hệ thống. Bảng User lưu thông tin tài khoản người dùng. Bảng Device lưu thông tin thiết bị và phương tiện được liên kết với người dùng. Bảng SensorData lưu các dữ liệu cảm biến, vị trí và trạng thái được gửi từ thiết bị IoT. Bảng Alert lưu lịch sử các cảnh

báo được hệ thống tạo ra khi phát hiện dấu hiệu bất thường. Bảng FcmToken lưu token thông báo của thiết bị di động để phục vụ việc gửi push notification. Các quan hệ dữ liệu chính gồm một người dùng có thể quản lý nhiều thiết bị, một thiết bị có nhiều bản ghi dữ liệu cảm biến, một thiết bị có thể phát sinh nhiều cảnh báo và một người dùng có thể có nhiều token thông báo.

Các dịch vụ bên ngoài được tách thành các gói riêng để thể hiện rõ vai trò hỗ trợ của chúng trong hệ thống. HiveMQ Cloud được sử dụng làm MQTT broker, đóng vai trò trung gian truyền dữ liệu giữa thiết bị IoT và backend. Firmware ESP32 gửi dữ liệu lên HiveMQ Cloud thông qua MQTT, trong khi MqttModule của backend đăng ký nhận dữ liệu từ broker để xử lý. Firebase Cloud Messaging được sử dụng để gửi thông báo cảnh báo đến ứng dụng di động khi hệ thống phát hiện nguy cơ trộm. OpenStreetMap được ứng dụng di động sử dụng để hiển thị vị trí và hành trình của phương tiện trên bản đồ.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

a, Mục tiêu thiết kế giao diện

Ứng dụng di động được thiết kế hướng tới người dùng sử dụng điện thoại Android với kích thước màn hình từ nhỏ đến trung bình lớn. Giao diện chủ yếu được thiết kế theo hướng màn hình dọc, phù hợp với thói quen sử dụng điện thoại và các thao tác như xem trạng thái thiết bị, nhận cảnh báo, theo dõi vị trí phương tiện.

Về kích thước màn hình, ứng dụng hướng tới các thiết bị Android phổ biến có kích thước khoảng từ 5 inch đến 6.7 inch. Về độ phân giải, giao diện được thiết kế để hiển thị tốt trên các thiết bị có độ phân giải từ HD+ đến Full HD+. Các thành phần giao diện như văn bản, nút bấm, biểu tượng, thẻ thông tin và bản đồ cần được bố trí linh hoạt để hạn chế tình trạng tràn nội dung hoặc khó thao tác trên màn hình nhỏ.

Ứng dụng hỗ trợ hiển thị trên màn hình màu của điện thoại thông minh. Giao diện sử dụng màu sắc đơn giản, có màu chủ đạo cho các thành phần chính và màu nhấn để thể hiện trạng thái thiết bị hoặc cảnh báo. Các trạng thái quan trọng như bình thường, mất kết nối hoặc cảnh báo cần được thể hiện rõ ràng để người dùng dễ nhận biết.

b, Quy chuẩn thiết kế giao diện

Để đảm bảo tính thống nhất và dễ sử dụng, ứng dụng áp dụng một số quy chuẩn chung trong quá trình thiết kế giao diện. Các màn hình được thiết kế với lề ngang 16px, khoảng cách giữa các thành phần sử dụng theo các mức 8px, 12px, 16px và

24px. Các card và nút chức năng chủ yếu sử dụng bán kính bo góc 8px nhằm tạo cảm giác mềm mại và dễ nhìn. Các vùng tương tác như nút bấm và biểu tượng được thiết kế với kích thước chạm tối thiểu 48×48px để thuận tiện cho thao tác trên màn hình cảm ứng.

Tiêu đề của mỗi màn hình được đặt ở phía bên trái của AppBar, trong khi các thao tác phụ như làm mới, chỉnh sửa hoặc mở rộng bản đồ được đặt ở phía bên phải. Điều hướng chính của ứng dụng sử dụng thanh điều hướng dưới, bao gồm các mục Trang chủ, Thiết bị, Cảnh báo và Tài khoản. Đối với các nội dung dài như danh sách dữ liệu cảm biến, lịch sử cảnh báo hoặc lịch sử hành trình, ứng dụng sử dụng cơ chế cuộn hoặc phân trang, tránh tải toàn bộ dữ liệu lên giao diện cùng một lúc.

Các nút và điều khiển trong ứng dụng được sử dụng theo từng mục đích cụ thể. *FilledButton* được dùng cho các thao tác chính như lưu, xác nhận hoặc kích hoạt thiết bị. *OutlinedButton* được dùng cho các thao tác phụ như bật bảo vệ hoặc điều khiển còi. *TextButton* được dùng cho các thao tác ít quan trọng như hủy hoặc quay lại. *IconButton* được dùng cho các thao tác nhanh như làm mới, chỉnh sửa, mở rộng bản đồ hoặc xóa bộ lọc. Đối với các thao tác nguy hiểm như xóa thiết bị, giao diện sử dụng màu đỏ và hiển thị hộp thoại xác nhận trước khi thực hiện. Trong quá trình gửi yêu cầu lên hệ thống, nút thao tác được vô hiệu hóa và hiển thị biểu tượng tải để tránh người dùng gửi lặp lại nhiều lần.

Về màu sắc, ứng dụng định nghĩa màu được sử dụng như sau. Màu xanh dương (#2563EB) được sử dụng cho các thành phần chính, liên kết và vị trí. Màu xanh lá (#16A34A) biểu thị trạng thái an toàn hoặc đang bảo vệ. Màu vàng cam (#F59E0B) biểu thị trạng thái cảnh báo hoặc chưa được bảo vệ. Màu đỏ (#DC2626) được dùng cho trạng thái nguy hiểm, mất cấp hoặc lỗi. Bên cạnh đó, ứng dụng sử dụng màu (#111827) cho văn bản chính, (#6B7280) cho văn bản phụ, (#F6F8FB) cho nền sáng, (#111827) cho nền tối và (#1F2937) cho bề mặt tối. Khi biểu diễn trạng thái, màu sắc luôn đi kèm biểu tượng hoặc nội dung chữ, không sử dụng màu làm dấu hiệu duy nhất.

Về phân cấp chữ, tiêu đề màn hình sử dụng cỡ chữ khoảng 20px và kiểu chữ đậm. Tiêu đề card hoặc khu vực sử dụng cỡ chữ 17–18px. Nội dung chính sử dụng cỡ chữ 13–16px. Các thông tin phụ như thời gian, tọa độ hoặc mô tả ngắn sử dụng cỡ chữ 11–12px và màu chữ phụ. Với các nội dung dài, ứng dụng giới hạn số dòng hiển thị và sử dụng dấu ba chấm để tránh làm rối giao diện.

Bố cục chung của các màn hình được tổ chức theo thứ tự gồm AppBar, vùng thông tin tổng quan, khu vực tìm kiếm hoặc bộ lọc, nội dung chính, trạng thái phân trang hoặc trạng thái trống và thanh điều hướng dưới. Ứng dụng hỗ trợ cả giao diện

sáng và tối, trong đó ý nghĩa màu sắc, độ tương phản và vị trí các điều khiển chính được giữ nhất quán giữa hai chế độ.

c, Một số hình ảnh thiết kế

Thanh điều hướng dưới : Thanh điều hướng dưới được thiết kế để giúp người dùng di chuyển nhanh giữa các chức năng chính của ứng dụng. Như minh họa trong Hình 4.3, thanh điều hướng gồm bốn mục: Trang chủ, Thiết bị, Cảnh báo và User. Mỗi mục bao gồm biểu tượng và nhãn văn bản để người dùng dễ nhận biết chức năng tương ứng.

Mục đang được chọn được làm nổi bật bằng màu sắc hoặc nền khác biệt, giúp người dùng biết được vị trí hiện tại trong ứng dụng. Cách thiết kế này giúp việc điều hướng giữa các màn hình chính trở nên đơn giản, trực quan và nhất quán.



Hình 4.3: Thiết kế thanh điều hướng dưới của ứng dụng

Màn hình trang chủ : Màn hình trang chủ được thiết kế nhằm cung cấp cái nhìn tổng quan về trạng thái các phương tiện mà người dùng đang quản lý. Như minh họa trong Hình 4.4, phần đầu màn hình hiển thị khu vực thông báo chung cho tất cả các xe, bao gồm các chỉ số tổng quan như tổng số xe, số xe đang an toàn và số xe cần kiểm tra. Cách bố trí này giúp người dùng nhanh chóng nắm được tình trạng chung của hệ thống ngay khi mở ứng dụng.

Bên dưới khu vực tổng quan là phần “Xe cần kiểm tra”, dùng để hiển thị các phương tiện đang có trạng thái bất thường hoặc cần người dùng chú ý. Tiếp theo là phần “Cảnh báo gần đây”, hiển thị các cảnh báo mới phát sinh trong hệ thống. Các thông tin này được trình bày theo dạng thẻ nhằm giúp nội dung dễ quan sát và thuận tiện khi mở rộng thêm chi tiết.

Phía dưới màn hình là thanh điều hướng chính, cho phép người dùng chuyển nhanh sang các nhóm chức năng khác như thiết bị, cảnh báo và tài khoản. Cách bố trí này giúp màn hình trang chủ vừa đóng vai trò tổng quan, vừa là điểm truy cập nhanh đến các chức năng quan trọng của ứng dụng.

The wireframe illustrates the main screen layout. At the top, a header box contains the text "Thông báo chung cho tất cả các xe" and three buttons labeled "Tổng số", "An toàn", and "Cẩn kiểm tra". Below the header, the text "Xe cần kiểm tra" is followed by a large empty rectangular box. Further down, the text "Cảnh báo gần đây" is followed by another large empty rectangular box. At the bottom, a footer box contains the text "AppBar".

Hình 4.4: Thiết kế giao diện màn hình trang chủ

Màn hình chi tiết thiết bị : Màn hình chi tiết thiết bị được thiết kế để hiển thị thông tin và trạng thái hoạt động của một thiết bị cụ thể. Như minh họa trong Hình 4.5, phần đầu màn hình hiển thị biển số xe, mã thiết bị và ngày kích hoạt, giúp người dùng nhận biết nhanh phương tiện đang được theo dõi.

Bên dưới là các nút thao tác chính gồm bật/tắt chế độ bảo vệ, bật/tắt còi và xóa thiết bị. Khu vực hành trình hiển thị bản đồ vị trí xe, kèm các lựa chọn lọc theo khoảng thời gian như 24 giờ, 7 ngày hoặc tất cả. Phần dữ liệu cảm biến cho phép người dùng lọc theo ngày và xem các thông tin như tốc độ, vị trí và thời gian cập nhật.

Chi tiết thiết bị

Settings

Biển số xe

Mã thiết bị

Ngày kích hoạt

Bật/ Tắt bảo vệ

Bật/Tắt còi

Hành trình

24 giờ

7 ngày

Tất cả

Map

Dữ liệu cảm biến

Lọc theo ngày

Tốc độ

Vị trí

Thời gian

Thanh điều hướng dưới

Hình 4.5: Thiết kế giao diện màn hình chi tiết thiết bị

Màn hình cảnh báo : Màn hình cảnh báo được thiết kế để hiển thị các thông báo bất thường phát sinh từ hệ thống. Như minh họa trong Hình 4.6, phần đầu màn hình hiển thị tổng số cảnh báo và cảnh báo mới nhất, giúp người dùng nhanh chóng nắm bắt tình hình chung.

Bên dưới là các công cụ lọc và tìm kiếm, bao gồm lọc theo ngày, lọc theo thiết bị và ô tìm kiếm. Danh sách cảnh báo được hiển thị dưới dạng thẻ, trong đó mỗi thẻ gồm biển số xe, thời gian, nội dung cảnh báo, mã thiết bị và dữ liệu liên quan. Cách bố trí này giúp người dùng dễ dàng tra cứu, phân loại và xem lại các cảnh báo đã phát sinh.

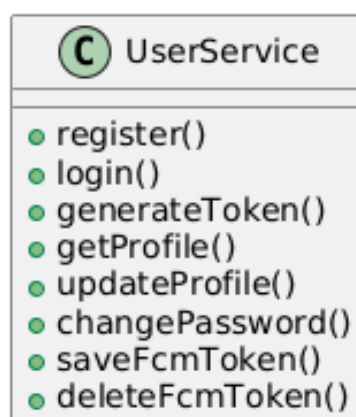
The wireframe shows a mobile application interface for alarm management. At the top, there is a header bar with the title "Cảnh báo" (Alarm) on the left and a "Settings" icon on the right. Below the header, there is a section titled "Tổng số cảnh báo" (Total number of alarms) with a sub-label "Mới nhất" (Newest). This section contains three buttons: "Lọc theo ngày" (Filter by day), "Lọc theo thiết bị" (Filter by device), and "Tìm kiếm" (Search). Below this is another section titled "Biển số" (License plate) with a sub-label "Thời gian" (Time). This section contains a "Message" button, a "Mã thiết bị" (Device code) button, and a "Dữ liệu" (Data) button. At the bottom of the screen, there is a footer bar with the text "Thanh điều hướng dưới" (Bottom navigation bar).

Hình 4.6: Thiết kế giao diện màn hình cảnh báo

4.2.2 Thiết kế lớp

a, Lớp UserService

Lớp *UserService* chịu trách nhiệm xử lý các nghiệp vụ liên quan đến người dùng trong hệ thống. Đây là lớp trung gian giữa controller và cơ sở dữ liệu, thực hiện các thao tác như đăng ký tài khoản, đăng nhập, tạo token xác thực, truy xuất và cập nhật thông tin cá nhân của người dùng.

Class UserService**Hình 4.7:** Thiết kế lớp UserService

Phương thức *register()* được sử dụng để tạo tài khoản người dùng mới. Phương thức *login()* xử lý quá trình đăng nhập và kiểm tra thông tin xác thực của người dùng. Sau khi đăng nhập thành công, phương thức *generateToken()* được sử dụng để sinh token phục vụ cho việc xác thực các yêu cầu tiếp theo từ ứng dụng di động.

Bên cạnh các chức năng xác thực, *UserService* còn cung cấp các phương thức quản lý thông tin cá nhân. Phương thức *getProfile()* dùng để lấy thông tin tài khoản hiện tại của người dùng, trong khi *updateProfile()* cho phép cập nhật các thông tin cá nhân. Phương thức *changePassword()* được sử dụng khi người dùng muốn thay đổi mật khẩu tài khoản.

Ngoài ra, lớp này còn hỗ trợ quản lý token thông báo của thiết bị di động thông qua hai phương thức *saveFcmToken()* và *deleteFcmToken()*. Các phương thức này giúp hệ thống lưu hoặc xóa token Firebase Cloud Messaging, phục vụ cho chức năng gửi cảnh báo đến đúng thiết bị của người dùng.

b, Lớp DeviceService

Lớp *DeviceService* chịu trách nhiệm xử lý các nghiệp vụ liên quan đến thiết bị chống trộm trong hệ thống. Đây là lớp trung gian giữa các yêu cầu từ ứng dụng di động, dữ liệu thiết bị trong cơ sở dữ liệu và các lệnh điều khiển gửi đến thiết bị IoT.



Hình 4.8: Thiết kế lớp DeviceService

Phương thức *createDevice()* được sử dụng để tạo mới thông tin thiết bị trong hệ thống. Phương thức *activateDevice()* cho phép người dùng kích hoạt và liên kết thiết bị với tài khoản cá nhân. Trước khi thiết bị được kích hoạt hoặc gửi dữ liệu lên hệ thống, phương thức *validateDevice()* được sử dụng để kiểm tra tính hợp lệ của thông tin thiết bị.

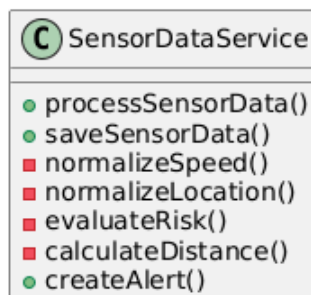
Đối với các chức năng quản lý thiết bị, *DeviceService* cung cấp phương thức *getUserDevices()* để lấy danh sách thiết bị thuộc tài khoản người dùng, *updateDevice()* để cập nhật thông tin thiết bị và *deleteDevice()* để xóa thiết bị khi người dùng không còn nhu cầu quản lý. Phương thức *updateProtectionStatus()* được sử dụng để cập nhật trạng thái bảo vệ của thiết bị, chẳng hạn bật hoặc tắt chế độ chống trộm.

Ngoài các chức năng quản lý, lớp này còn hỗ trợ tương tác với thiết bị IoT. Phương thức *controlBuzzer()* được sử dụng để gửi lệnh điều khiển còi đến thiết bị. Phương thức *getSensorHistory()* cho phép truy xuất lịch sử dữ liệu cảm biến của thiết bị, trong khi *getJourney()* được sử dụng để lấy dữ liệu hành trình hoặc vị trí của phương tiện. Nhờ đó, *DeviceService* đóng vai trò quan trọng trong việc kết nối các chức năng quản lý thiết bị, giám sát trạng thái và điều khiển thiết bị từ xa.

c, Lớp SensorDataService

Lớp *SensorDataService* chịu trách nhiệm xử lý dữ liệu cảm biến do thiết bị IoT gửi về hệ thống. Đây là lớp quan trọng trong backend vì dữ liệu cảm biến là cơ sở để cập nhật trạng thái phương tiện, lưu lịch sử hoạt động và phát hiện các dấu hiệu bất thường có thể liên quan đến nguy cơ trộm.

Class SensorDataService



Hình 4.9: Thiết kế lớp *SensorDataService*

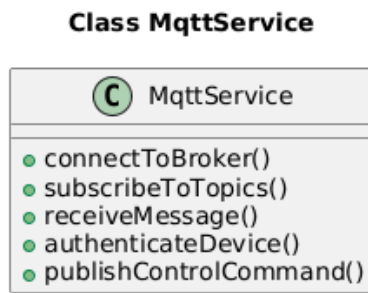
Phương thức *processSensorData()* được sử dụng để tiếp nhận và xử lý dữ liệu cảm biến sau khi dữ liệu được gửi từ thiết bị lên server. Trong quá trình xử lý, dữ liệu có thể được chuẩn hóa thông qua các phương thức nội bộ như *normalizeSpeed()* để chuẩn hóa tốc độ và *normalizeLocation()* để chuẩn hóa thông tin vị trí. Sau đó, phương thức *saveSensorData()* thực hiện lưu dữ liệu đã xử lý vào cơ sở dữ liệu để phục vụ việc xem trạng thái và tra cứu lịch sử.

Bên cạnh chức năng lưu trữ, *SensorDataService* còn đảm nhiệm việc đánh giá nguy cơ bất thường. Phương thức *evaluateRisk()* được sử dụng để kiểm tra dữ liệu cảm biến, trạng thái thiết bị và vị trí nhằm xác định có phát sinh nguy cơ trộm hay không. Trong quá trình này, phương thức *calculateDistance()* hỗ trợ tính toán sự thay đổi vị trí của phương tiện, phục vụ việc đánh giá mức độ bất thường khi xe bị di chuyển.

Khi hệ thống xác định dữ liệu có dấu hiệu nguy hiểm, phương thức *createAlert()* được sử dụng để tạo bản ghi cảnh báo trong cơ sở dữ liệu và kích hoạt quá trình gửi thông báo đến người dùng. Nhờ đó, *SensorDataService* đóng vai trò trung tâm trong luồng xử lý dữ liệu cảm biến, từ tiếp nhận, chuẩn hóa, lưu trữ cho đến phát hiện nguy cơ trộm và tạo cảnh báo.

d, Lớp *MqttService*

Lớp *MqttService* chịu trách nhiệm quản lý quá trình giao tiếp giữa backend và thiết bị IoT thông qua giao thức MQTT. Đây là lớp trung gian giúp hệ thống máy chủ nhận dữ liệu từ thiết bị, chuyển dữ liệu cảm biến đến các lớp xử lý nghiệp vụ và gửi lệnh điều khiển ngược lại thiết bị khi cần thiết.



Hình 4.10: Thiết kế lớp MqttService

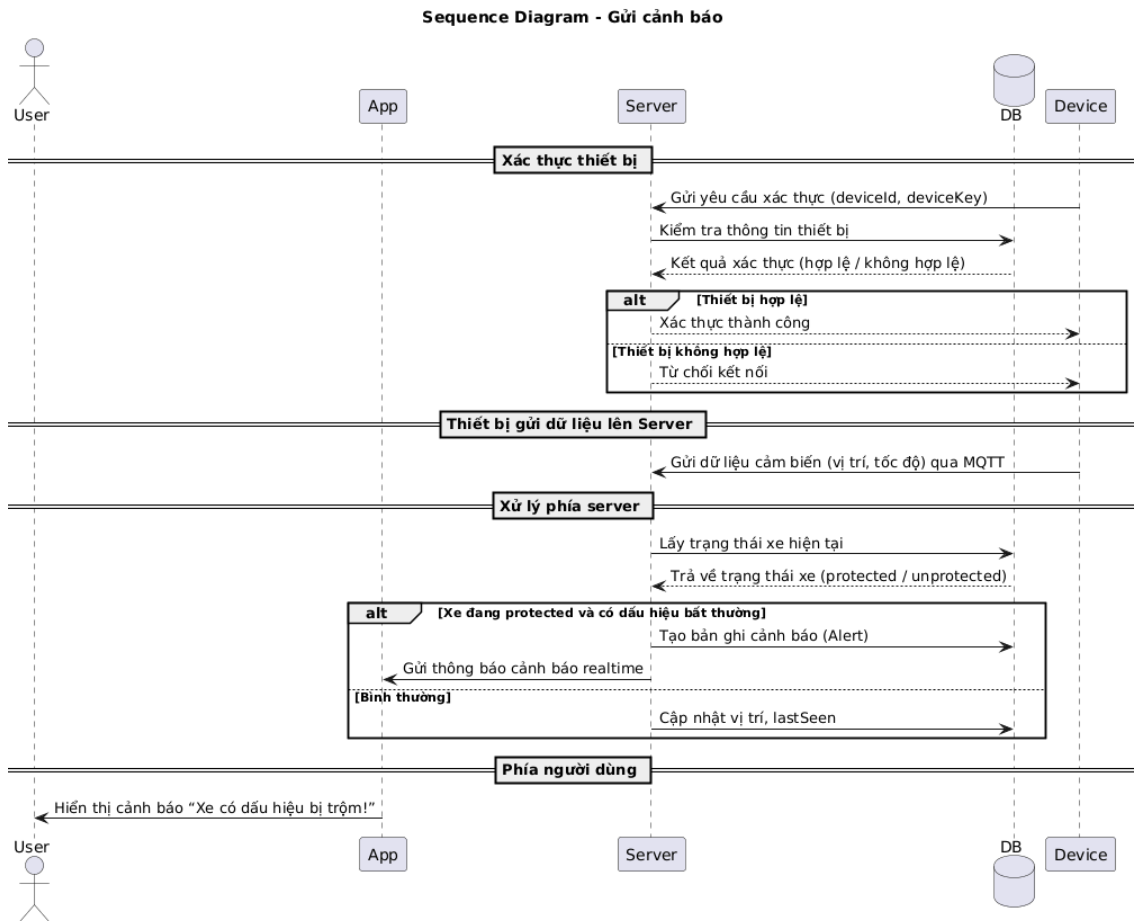
Phương thức *connectToBroker()* được sử dụng để thiết lập kết nối từ backend đến MQTT broker. Sau khi kết nối thành công, phương thức *subscribeToTopics()* thực hiện đăng ký các topic cần lắng nghe để nhận dữ liệu từ thiết bị IoT. Khi thiết bị gửi dữ liệu lên broker, phương thức *receiveMessage()* tiếp nhận bản tin MQTT và phân tích nội dung dữ liệu nhận được.

Trước khi dữ liệu được đưa vào xử lý, phương thức *authenticateDevice()* được sử dụng để kiểm tra tính hợp lệ của thiết bị gửi dữ liệu. Việc xác thực này giúp hạn chế dữ liệu không hợp lệ hoặc dữ liệu từ thiết bị không thuộc hệ thống. Sau khi dữ liệu được xác thực, *MqttService* chuyển dữ liệu đến các lớp xử lý liên quan như *SensorDataService* để lưu trữ, phân tích và phát hiện nguy cơ bất thường.

Ngoài nhiệm vụ nhận dữ liệu, *MqttService* còn hỗ trợ gửi lệnh điều khiển đến thiết bị IoT thông qua phương thức *publishControlCommand()*. Phương thức này được sử dụng trong các chức năng như bật hoặc tắt còi, thay đổi trạng thái bảo vệ hoặc gửi lệnh điều khiển từ ứng dụng di động đến thiết bị. Nhờ đó, *MqttService* đóng vai trò quan trọng trong việc duy trì kênh giao tiếp hai chiều giữa backend service và thiết bị IoT.

e, Biểu đồ tuần tự gửi cảnh báo

Biểu đồ tuần tự gửi cảnh báo mô tả luồng xử lý khi thiết bị IoT gửi dữ liệu cảm biến lên hệ thống và server phát hiện dấu hiệu bất thường cần cảnh báo cho người dùng. Các thành phần tham gia trong biểu đồ gồm người dùng, ứng dụng di động, server, cơ sở dữ liệu và thiết bị.



Hình 4.11: Biểu đồ trình tự Gửi cảnh báo

Trước khi gửi dữ liệu, thiết bị thực hiện xác thực với server bằng cách gửi thông tin *deviceId* và *deviceKey*. Server truy vấn cơ sở dữ liệu để kiểm tra thông tin thiết bị. Nếu thiết bị hợp lệ, server phản hồi xác thực thành công và cho phép thiết bị tiếp tục gửi dữ liệu. Ngược lại, nếu thông tin thiết bị không hợp lệ, server từ chối kết nối hoặc không tiếp nhận dữ liệu từ thiết bị.

Sau khi được xác thực, thiết bị gửi dữ liệu cảm biến lên server thông qua giao thức MQTT. Dữ liệu gửi lên bao gồm các thông tin như vị trí và tốc độ của phương tiện. Khi nhận được dữ liệu, server lấy trạng thái hiện tại của xe từ cơ sở dữ liệu để kiểm tra xem xe đang ở chế độ bảo vệ hay không.

Trong trường hợp xe đang ở trạng thái được bảo vệ và dữ liệu cảm biến có dấu hiệu bất thường, server tạo một bản ghi cảnh báo trong cơ sở dữ liệu. Sau đó, server gửi thông báo cảnh báo theo thời gian thực đến ứng dụng di động. Ứng dụng hiển thị cảnh báo cho người dùng với nội dung xe có dấu hiệu bị trộm, giúp người dùng kịp thời nắm bắt tình huống.

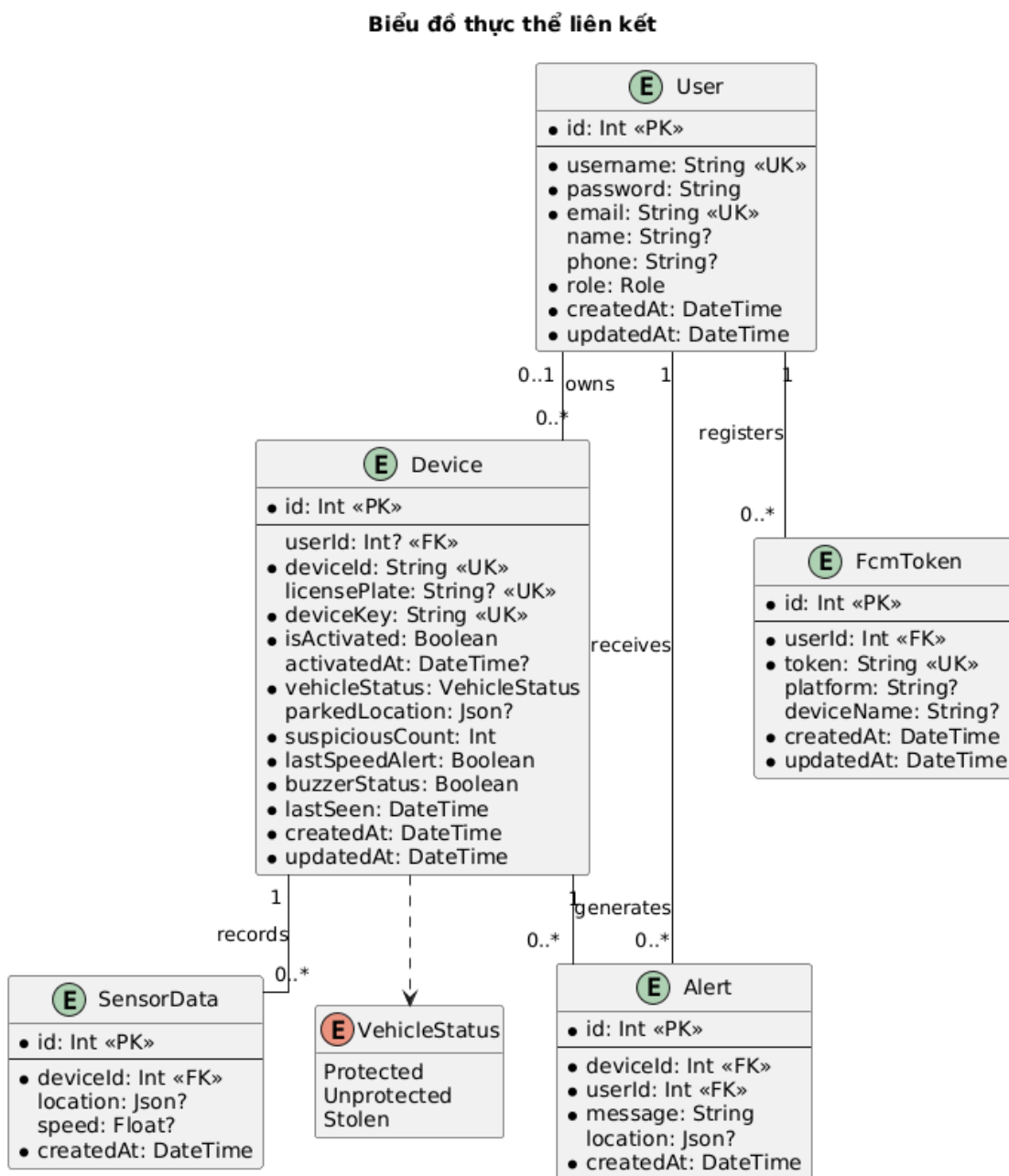
Nếu dữ liệu gửi lên không có dấu hiệu bất thường hoặc xe không ở trạng thái bảo vệ, server chỉ cập nhật thông tin vị trí và thời gian hoạt động gần nhất của thiết

bị. Nhờ luồng xử lý này, hệ thống có thể vừa lưu trữ dữ liệu giám sát, vừa phát hiện và gửi cảnh báo kịp thời khi phương tiện có nguy cơ bị xâm phạm.

4.2.3 Thiết kế cơ sở dữ liệu

a, Biểu đồ thực thể liên kết

Biểu đồ thực thể liên kết mô tả cấu trúc dữ liệu chính của hệ thống chống trộm xe IoT. Các thực thể trong biểu đồ gồm *User*, *Device*, *SensorData*, *Alert*, *FcmToken* và kiểu liệt kê *VehicleStatus*. Các thực thể này được thiết kế nhằm lưu trữ thông tin người dùng, thiết bị, dữ liệu cảm biến, cảnh báo và token thông báo của ứng dụng di động.



Hình 4.12: Biểu đồ thực thể liên kết

Thực thể *User* lưu trữ thông tin tài khoản người dùng, bao gồm tên đăng nhập, mật khẩu, email, họ tên, số điện thoại, vai trò và thời gian tạo/cập nhật. Trong đó, *username* và *email* là các trường có tính duy nhất để đảm bảo mỗi tài khoản được định danh rõ ràng trong hệ thống.

Thực thể *Device* lưu trữ thông tin thiết bị chống trộm được gắn trên phương tiện. Mỗi thiết bị có mã thiết bị *deviceId*, khóa xác thực *deviceKey*, biển số xe, trạng thái kích hoạt, thời điểm kích hoạt, trạng thái phương tiện, vị trí đỗ xe, trạng thái còi và thời gian hoạt động gần nhất. Trường *userId* cho phép liên kết thiết bị với tài khoản người dùng sau khi thiết bị được kích hoạt.

Thực thể *SensorData* lưu trữ dữ liệu cảm biến do thiết bị gửi lên hệ thống, bao gồm vị trí, tốc độ và thời gian ghi nhận dữ liệu. Mỗi bản ghi dữ liệu cảm biến liên kết với một thiết bị cụ thể thông qua khóa ngoại *deviceId*. Dữ liệu này được sử dụng để hiển thị lịch sử hoạt động, hành trình và hỗ trợ quá trình phát hiện bất thường.

Thực thể *Alert* lưu trữ các cảnh báo được tạo ra khi hệ thống phát hiện dấu hiệu bất thường. Mỗi cảnh báo gắn với một thiết bị và một người dùng, bao gồm nội dung cảnh báo, vị trí liên quan và thời điểm phát sinh. Dữ liệu này phục vụ cho chức năng hiển thị danh sách cảnh báo và lịch sử cảnh báo trên ứng dụng di động.

Thực thể *FcmToken* lưu trữ token thông báo của thiết bị di động. Mỗi token liên kết với một người dùng, có thể kèm theo thông tin nền tảng và tên thiết bị. Thực thể này được sử dụng để hệ thống gửi thông báo cảnh báo đến đúng thiết bị di động của người dùng thông qua Firebase Cloud Messaging.

Các quan hệ trong biểu đồ thể hiện mối liên kết giữa các thực thể. Một người dùng có thể sở hữu nhiều thiết bị, nhận nhiều cảnh báo và đăng ký nhiều FCM token. Một thiết bị có thể sinh ra nhiều bản ghi dữ liệu cảm biến và nhiều cảnh báo. Kiểu liệt kê *VehicleStatus* được sử dụng để biểu diễn trạng thái của phương tiện, gồm các trạng thái *Protected*, *Unprotected* và *Stolen*. Cách thiết kế này giúp dữ liệu trong hệ thống được tổ chức rõ ràng, hỗ trợ tốt cho các chức năng quản lý thiết bị, giám sát dữ liệu và gửi cảnh báo đến người dùng.

b, Thiết kế cơ sở dữ liệu trong PostgreSQL

Từ biểu đồ thực thể liên kết đã trình bày ở phần trên, cơ sở dữ liệu của hệ thống được thiết kế theo mô hình quan hệ và triển khai trên hệ quản trị cơ sở dữ liệu PostgreSQL. Các thực thể trong biểu đồ được ánh xạ thành các bảng dữ liệu, trong đó mỗi bảng có khóa chính để định danh bản ghi và các khóa ngoại để thể hiện quan hệ giữa các bảng.

Trong quá trình triển khai, hệ thống sử dụng Prisma ORM làm lớp trung gian

giữa mã nguồn backend NestJS và PostgreSQL. Các kiểu dữ liệu trong mô hình Prisma được ánh xạ sang các kiểu dữ liệu tương ứng trong PostgreSQL. Ví dụ, kiểu *Int* được ánh xạ sang kiểu số nguyên, *String* được ánh xạ sang kiểu chuỗi, *Boolean* dùng cho dữ liệu đúng/sai, *DateTime* dùng cho dữ liệu thời gian, còn *Json* được sử dụng để lưu các dữ liệu có cấu trúc linh hoạt như vị trí phương tiện.

Các bảng chính trong cơ sở dữ liệu được mô tả trong Bảng 4.1.

Tên bảng	Mô tả
<i>User</i>	Lưu thông tin tài khoản người dùng, bao gồm tên đăng nhập, email, mật khẩu, thông tin cá nhân và vai trò trong hệ thống.
<i>Device</i>	Lưu thông tin thiết bị chống trộm, mã thiết bị, khóa xác thực, biển số xe, trạng thái kích hoạt, trạng thái phương tiện và thông tin hoạt động gần nhất.
<i>SensorData</i>	Lưu dữ liệu cảm biến do thiết bị IoT gửi lên, bao gồm vị trí, tốc độ và thời gian ghi nhận dữ liệu.
<i>Alert</i>	Lưu thông tin cảnh báo được tạo ra khi hệ thống phát hiện dấu hiệu bất thường liên quan đến phương tiện.
<i>FcmToken</i>	Lưu token thông báo của thiết bị di động, phục vụ chức năng gửi cảnh báo đến người dùng thông qua Firebase Cloud Messaging.

Bảng 4.1: Danh sách các bảng chính trong cơ sở dữ liệu PostgreSQL

Bảng *User* là bảng trung tâm dùng để lưu trữ thông tin tài khoản người dùng. Mỗi người dùng có một khóa chính *id*. Các trường *username* và *email* được đặt ràng buộc duy nhất để đảm bảo không có hai tài khoản trùng thông tin đăng nhập. Bảng này có quan hệ với bảng *Device*, *Alert* và *FcmToken*.

Bảng *Device* lưu thông tin các thiết bị IoT được sử dụng trong hệ thống. Mỗi thiết bị có một mã định danh *deviceId* và khóa xác thực *deviceKey*. Trường *userId* là khóa ngoại liên kết thiết bị với người dùng sau khi thiết bị được kích hoạt. Ngoài ra, bảng này còn lưu trạng thái phương tiện, vị trí đỗ xe, trạng thái còi, số lần phát hiện bất thường và thời gian thiết bị hoạt động gần nhất.

Bảng *SensorData* lưu các bản ghi dữ liệu được gửi từ thiết bị lên server. Mỗi bản ghi liên kết với một thiết bị thông qua khóa ngoại *deviceId*. Dữ liệu trong bảng này được sử dụng để hiển thị lịch sử cảm biến, hành trình di chuyển và hỗ trợ quá trình phát hiện nguy cơ trộm.

Bảng *Alert* lưu các cảnh báo phát sinh trong hệ thống. Mỗi cảnh báo gắn với

một thiết bị và một người dùng cụ thể thông qua các khóa ngoại *deviceId* và *userId*. Bảng này lưu nội dung cảnh báo, vị trí liên quan và thời điểm phát sinh, phục vụ cho chức năng thông báo và xem lịch sử cảnh báo trên ứng dụng di động.

Bảng *FcmToken* lưu token thông báo của thiết bị di động. Một người dùng có thể có nhiều token khác nhau trong trường hợp đăng nhập trên nhiều thiết bị. Các token này được sử dụng để server gửi cảnh báo đến đúng thiết bị di động của người dùng thông qua Firebase Cloud Messaging.

Các quan hệ chính trong cơ sở dữ liệu gồm: một người dùng có thể sở hữu nhiều thiết bị, một thiết bị có thể sinh ra nhiều bản ghi dữ liệu cảm biến, một thiết bị có thể phát sinh nhiều cảnh báo, và một người dùng có thể đăng ký nhiều FCM token. Việc thiết kế cơ sở dữ liệu theo mô hình quan hệ giúp dữ liệu được tổ chức rõ ràng, đảm bảo tính nhất quán và thuận tiện cho các truy vấn như xem danh sách thiết bị, xem lịch sử dữ liệu, truy xuất cảnh báo và gửi thông báo đến người dùng.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Trong quá trình xây dựng hệ thống, đồ án sử dụng một số công cụ và thư viện phục vụ cho phát triển ứng dụng di động, xây dựng máy chủ, thiết kế cơ sở dữ liệu, lập trình thiết bị IoT và quản lý mã nguồn. Danh sách các công cụ và thư viện chính được trình bày trong Bảng 4.2.

Mục đích	Công cụ / Thư viện	Địa chỉ URL
Soạn thảo mã nguồn	Visual Studio Code	https://code.visualstudio.com/
Phát triển ứng dụng Android	Android Studio	https://developer.android.com/studio
Phát triển ứng dụng di động	Flutter	https://flutter.dev/
Ngôn ngữ lập trình ứng dụng di động	Dart	https://dart.dev/
Phát triển máy chủ	NestJS	https://nestjs.com/
Môi trường chạy backend	Node.js	https://nodejs.org/
Quản lý package backend	npm	https://www.npmjs.com/
Tương tác cơ sở dữ liệu	Prisma ORM	https://www.prisma.io/
Hệ quản trị cơ sở dữ liệu	PostgreSQL	https://www.postgresql.org/
Phát triển firmware ESP32	Arduino IDE	https://www.arduino.cc/en/software
Nền tảng ESP32	Arduino ESP32 Core	https://github.com/espressif/arduino-esp32
Xử lý dữ liệu JSON trên thiết bị	ArduinoJson	https://arduinojson.org/
Giao tiếp MQTT trên ESP32	PubSubClient	https://pubsubclient.knolleary.net/
Đọc dữ liệu GPS	TinyGPSPlus	https://github.com/mikalhart/TinyGPSPlus
MQTT Broker	HiveMQ Cloud	https://www.hivemq.com/products/mqtt-cloud-broker/
Gửi thông báo đẩy	Firebase Cloud Messaging	https://firebase.google.com/docs/cloud-messaging
Hiển thị bản đồ	OpenStreetMap	https://www.openstreetmap.org/
Quản lý phiên bản mã nguồn	Git	https://git-scm.com/

Bảng 4.2: Danh sách thư viện và công cụ sử dụng

4.3.2 Kết quả đạt được

Sau quá trình phân tích, thiết kế và triển khai, đồ án đã xây dựng được hệ thống chống trộm xe máy sử dụng thiết bị IoT kết hợp với ứng dụng di động và máy chủ xử lý dữ liệu. Hệ thống bao gồm ba thành phần chính: thiết bị IoT gắn trên xe, ứng dụng di động dành cho người dùng và máy chủ backend. Thiết bị IoT có nhiệm vụ thu thập dữ liệu cảm biến và vị trí của xe, sau đó gửi dữ liệu về hệ thống thông qua

giao thức MQTT. Máy chủ backend tiếp nhận, xử lý, lưu trữ dữ liệu và phát hiện các tình huống bất thường dựa trên tốc độ, khoảng cách di chuyển và trạng thái thiết bị. Ứng dụng di động cho phép người dùng quản lý thiết bị, theo dõi trạng thái xe, xem lịch sử dữ liệu và nhận cảnh báo khi hệ thống phát hiện nguy cơ mất an toàn.

Sản phẩm sau cùng của đồ án bao gồm mã nguồn ứng dụng di động, mã nguồn máy chủ backend, cơ sở dữ liệu và chương trình điều khiển thiết bị IoT. Ứng dụng di động được xây dựng nhằm cung cấp giao diện trực quan cho người dùng cuối. Máy chủ backend đóng vai trò trung gian giữa thiết bị IoT, cơ sở dữ liệu và ứng dụng di động. Cơ sở dữ liệu lưu trữ thông tin người dùng, thiết bị, dữ liệu cảm biến, vị trí và lịch sử cảnh báo. Thiết bị IoT đảm nhận việc thu thập dữ liệu thực tế từ xe và gửi dữ liệu định kỳ về hệ thống.

Thành phần	Kết quả đạt được
Ứng dụng di động	Xây dựng ứng dụng cho phép người dùng đăng nhập, quản lý thiết bị, theo dõi trạng thái xe, xem lịch sử dữ liệu và nhận cảnh báo từ hệ thống.
Máy chủ backend	Xây dựng hệ thống API phục vụ ứng dụng di động, tiếp nhận dữ liệu từ thiết bị IoT, xử lý dữ liệu cảm biến, lưu trữ dữ liệu và gửi cảnh báo đến người dùng.
Thiết bị IoT	Xây dựng chương trình thu thập dữ liệu cảm biến, dữ liệu vị trí và gửi dữ liệu về máy chủ thông qua giao thức MQTT.
Cơ sở dữ liệu	Thiết kế và triển khai cơ sở dữ liệu lưu trữ thông tin người dùng, thiết bị, dữ liệu cảm biến, vị trí và lịch sử cảnh báo.
Hệ thống cảnh báo	Xây dựng cơ chế phát hiện bất thường dựa trên dữ liệu tốc độ, khoảng cách di chuyển và trạng thái thiết bị, từ đó gửi thông báo cảnh báo đến ứng dụng di động.

Bảng 4.3: Các thành phần chính đã xây dựng

Ngoài các thành phần chức năng, đồ án cũng thống kê một số thông tin kỹ thuật của hệ thống nhằm thể hiện quy mô triển khai. Các thông tin thống kê bao gồm số lượng tệp mã nguồn, số dòng mã, số module chính và dung lượng mã nguồn của từng thành phần. Kết quả thống kê được trình bày trong Bảng 4.4.

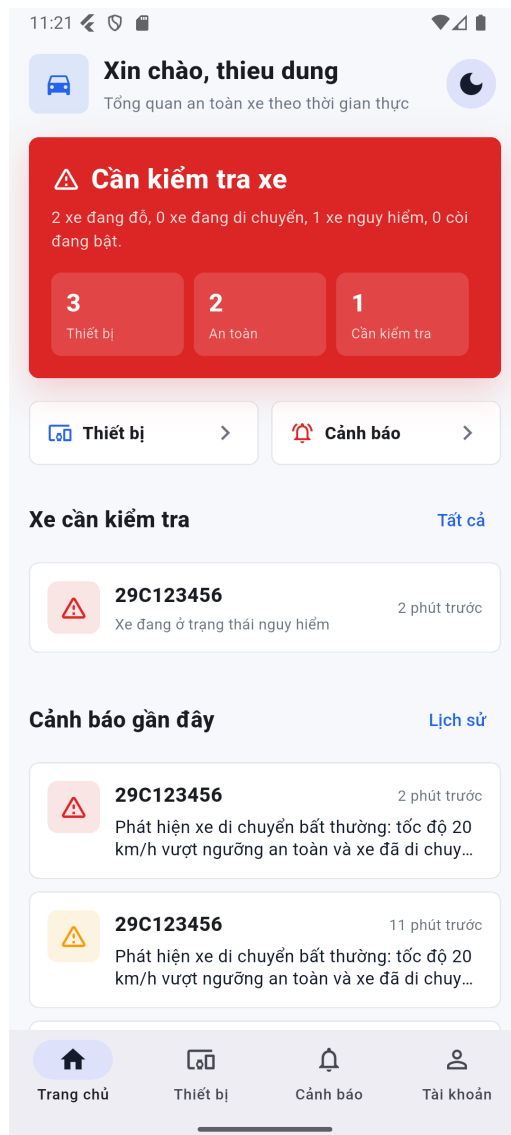
Thành phần	Số tệp mã nguồn	Số dòng mã	Số gói	Dung lượng
Ứng dụng di động	49	≈ 9.000	5	0,27 MB
Máy chủ backend	41	≈ 2.500	9	0,1 MB
Chương trình IoT	1	≈ 1000	1	0,05 MB
Cơ sở dữ liệu	8	≈ 250	1	0,01 MB
Tổng cộng	99	≈ 12.750	16	0,43 MB

Ghi chú: Bảng thống kê chỉ tính các tệp mã nguồn chính do em xây dựng, không bao gồm thư viện phụ thuộc, file build, file cache và các tệp sinh tự động trong quá trình phát triển.

Bảng 4.4: Thống kê mã nguồn và thành phần triển khai

4.3.3 Minh họa các chức năng chính

a, Màn hình trang chủ

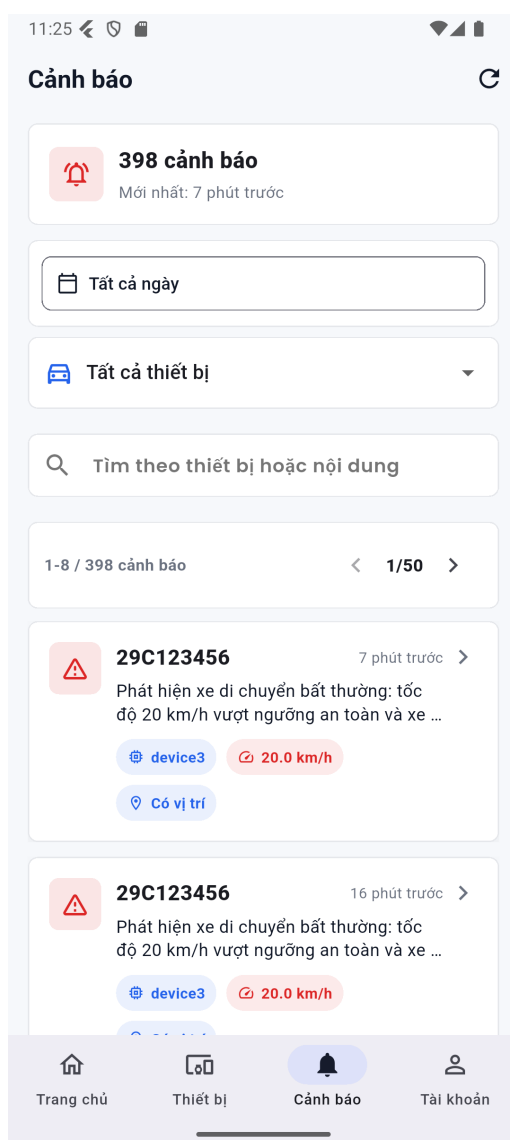


Hình 4.13: Màn hình Trang chủ của ứng dụng

Hình 4.13 trình bày màn hình Trang chủ của ứng dụng. Màn hình này hiển thị tổng quan trạng thái an toàn của các thiết bị, bao gồm số lượng thiết bị, số xe an toàn và số xe cần kiểm tra. Khi có xe ở trạng thái nguy hiểm, hệ thống sử dụng màu đỏ để làm nổi bật cảnh báo, giúp người dùng nhanh chóng nhận biết tình huống bất thường.

Phía dưới màn hình là danh sách xe cần kiểm tra và các cảnh báo gần đây. Mỗi cảnh báo hiển thị biển số xe, thời gian phát sinh và nội dung mô tả ngắn gọn. Màn hình cũng cung cấp các lối tắt đến chức năng quản lý thiết bị và xem cảnh báo, hỗ trợ người dùng thao tác nhanh với các chức năng chính.

b, Màn hình cảnh báo



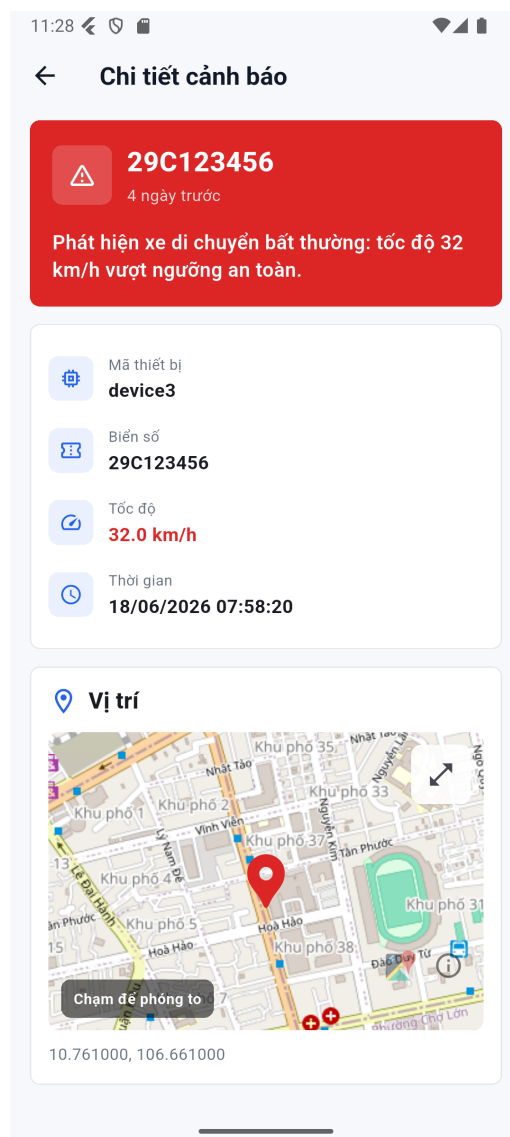
Hình 4.14: Màn hình Cảnh báo của ứng dụng

Hình 4.14 minh họa màn hình Cảnh báo của ứng dụng. Màn hình này hiển thị danh sách các cảnh báo phát sinh từ hệ thống khi phát hiện xe có dấu hiệu di chuyển

bất thường. Phía trên màn hình thể hiện tổng số cảnh báo và thời điểm cảnh báo mới nhất, giúp người dùng nhanh chóng nắm được tình trạng hiện tại.

Ứng dụng cung cấp các bộ lọc theo thời gian, thiết bị và ô tìm kiếm để hỗ trợ người dùng tra cứu cảnh báo thuận tiện hơn. Mỗi mục cảnh báo hiển thị biển số xe, thời gian phát sinh, nội dung cảnh báo, tên thiết bị, tốc độ ghi nhận và trạng thái có vị trí. Nhờ đó, người dùng có thể kiểm tra nhanh thông tin cảnh báo và chuyển sang màn hình chi tiết khi cần xử lý thêm.

c, Màn hình chi tiết cảnh báo



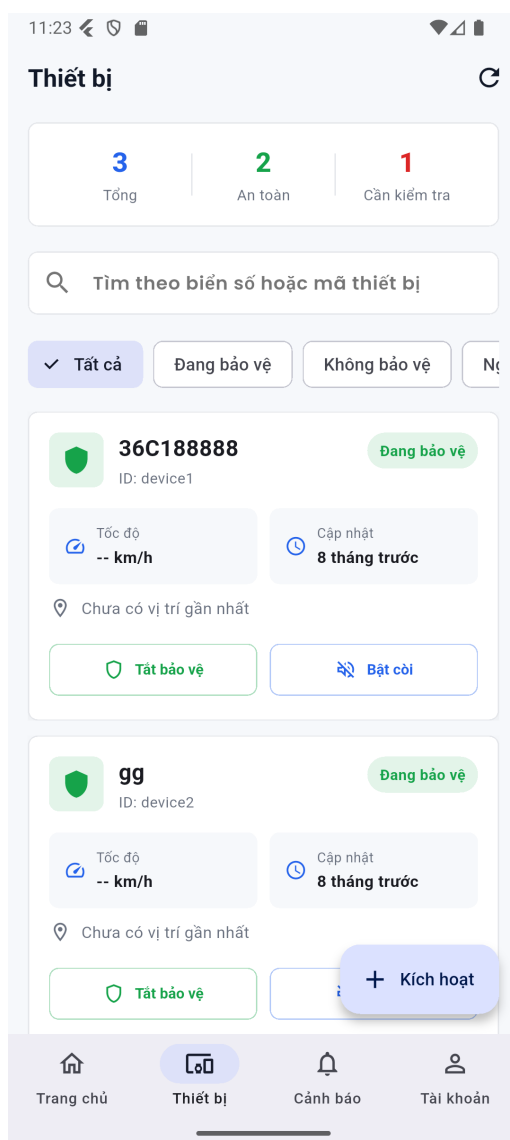
Hình 4.15: Màn hình Chi tiết cảnh báo của ứng dụng

Hình 4.15 minh họa màn hình Chi tiết cảnh báo của ứng dụng. Màn hình này hiển thị đầy đủ thông tin của một cảnh báo cụ thể khi hệ thống phát hiện xe có dấu hiệu di chuyển bất thường. Phần đầu màn hình sử dụng màu đỏ để nhấn mạnh mức độ nghiêm trọng của cảnh báo, đồng thời hiển thị biển số xe, thời điểm phát sinh

và nội dung cảnh báo.

Bên dưới là khu vực thông tin chi tiết, bao gồm mã thiết bị, biển số xe, tốc độ ghi nhận tại thời điểm cảnh báo và thời gian xảy ra sự kiện. Ngoài ra, màn hình còn hiển thị vị trí phát sinh cảnh báo trên bản đồ, kèm theo tọa độ địa lý. Thông tin này giúp người dùng nhanh chóng xác định vị trí của xe và đưa ra phương án xử lý phù hợp.

d, Màn hình thiết bị



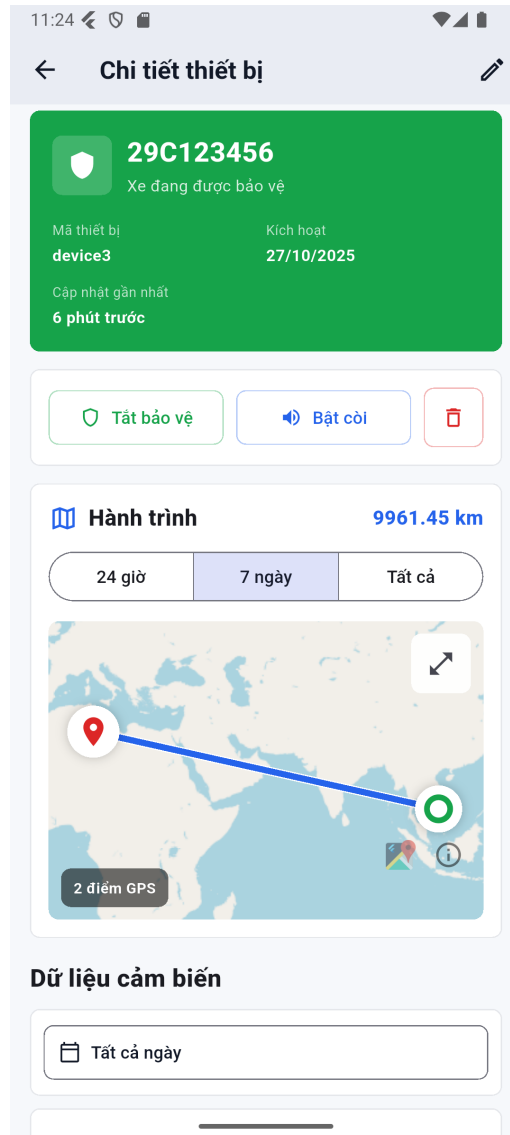
Hình 4.16: Màn hình Thiết bị của ứng dụng

Hình 4.16 minh họa màn hình Thiết bị của ứng dụng. Màn hình này cho phép người dùng theo dõi và quản lý danh sách các thiết bị đã được kích hoạt trong hệ thống. Phần đầu màn hình hiển thị thống kê tổng số thiết bị, số thiết bị đang ở trạng thái an toàn và số thiết bị cần kiểm tra.

Bên dưới là khu vực tìm kiếm và bộ lọc theo trạng thái thiết bị, giúp người dùng

nhANH chóng tra cứu thiết bị theo biển số, mã thiết bị hoặc trạng thái bảo vệ. Mỗi thẻ thiết bị hiển thị các thông tin chính như biển số xe, mã thiết bị, trạng thái bảo vệ, tốc độ gần nhất, thời gian cập nhật và vị trí gần nhất nếu có. Ngoài ra, người dùng có thể thực hiện nhanh các thao tác như bật/tắt chế độ bảo vệ, điều khiển còi hoặc kích hoạt thiết bị mới.

e, Màn hình chi tiết thiết bị



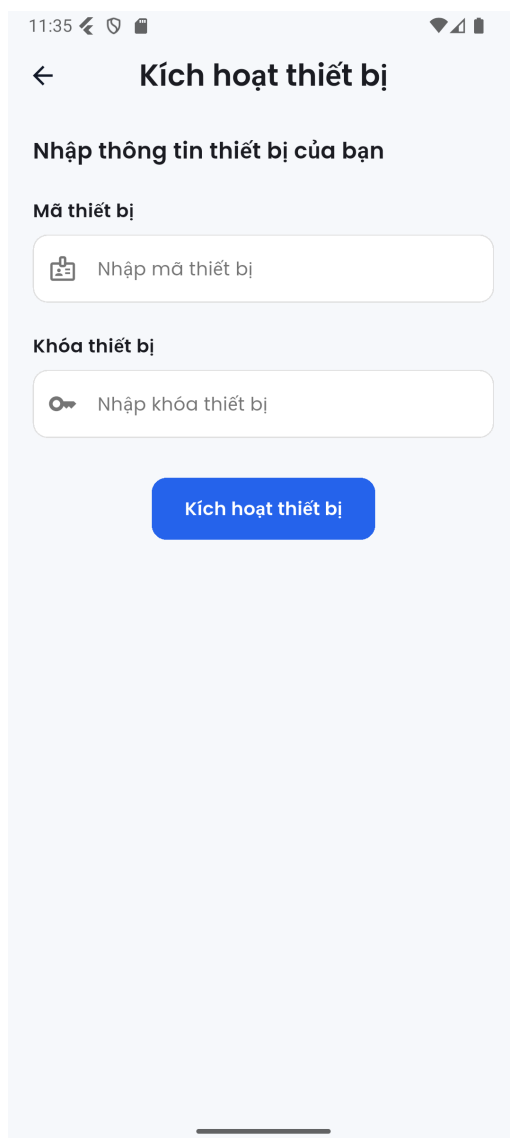
Hình 4.17: Màn hình Chi tiết thiết bị của ứng dụng

Hình 4.17 minh họa màn hình Chi tiết thiết bị của ứng dụng. Màn hình này hiển thị thông tin chi tiết của một thiết bị đang được người dùng quản lý. Phần đầu màn hình thể hiện trạng thái hiện tại của xe, bao gồm biển số, mã thiết bị, ngày kích hoạt và thời gian cập nhật gần nhất. Khi xe đang ở trạng thái an toàn, giao diện sử dụng màu xanh để biểu thị thiết bị đang được bảo vệ.

Bên dưới phần thông tin tổng quan, người dùng có thể thực hiện các thao tác

điều khiển như bật/tắt chế độ bảo vệ, bật còi cảnh báo hoặc xóa thiết bị. Màn hình cũng hiển thị thông tin hành trình của xe trên bản đồ, cho phép người dùng xem quãng đường di chuyển theo các khoảng thời gian như 24 giờ, 7 ngày hoặc toàn bộ dữ liệu. Ngoài ra, phần dữ liệu cảm biến phía dưới hỗ trợ người dùng theo dõi các thông tin thu thập được từ thiết bị IoT.

f, Màn hình kích hoạt thiết bị



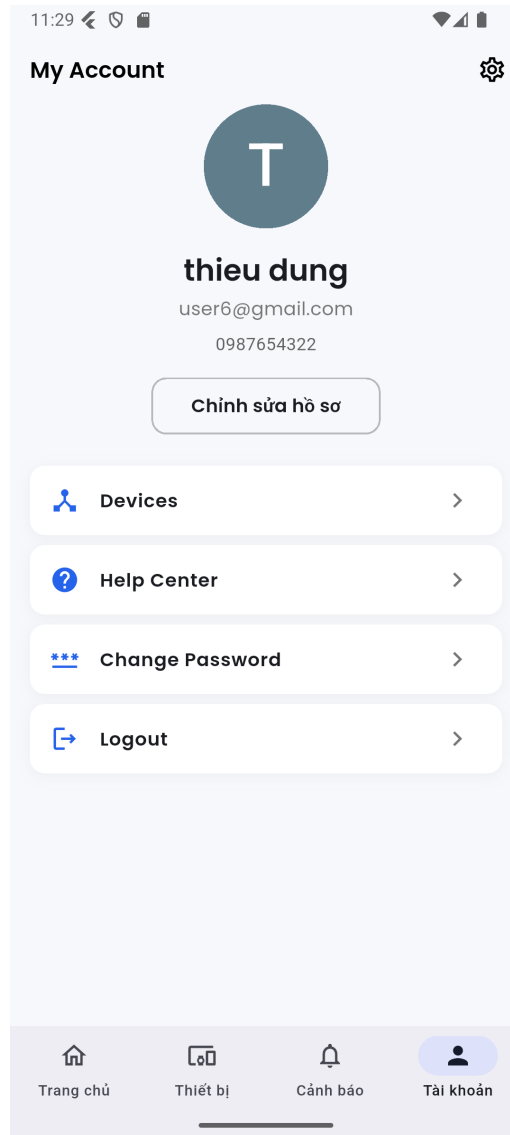
Hình 4.18: Màn hình Kích hoạt thiết bị của ứng dụng

Hình 4.18 minh họa màn hình Kích hoạt thiết bị của ứng dụng. Màn hình này cho phép người dùng liên kết thiết bị IoT với tài khoản cá nhân trước khi sử dụng các chức năng giám sát và cảnh báo. Người dùng cần nhập mã thiết bị và khóa thiết bị được cung cấp tương ứng với thiết bị vật lý.

Sau khi người dùng nhấn nút kích hoạt, ứng dụng gửi thông tin lên máy chủ để kiểm tra tính hợp lệ của thiết bị. Nếu thông tin chính xác và thiết bị chưa được kích

hoạt bởi tài khoản khác, hệ thống sẽ gán thiết bị cho người dùng hiện tại. Chức năng này giúp đảm bảo chỉ người dùng có thông tin xác thực hợp lệ mới có thể quản lý thiết bị trong hệ thống.

g, Màn hình tài khoản cá nhân



Hình 4.19: Màn hình Tài khoản của ứng dụng

Hình 4.19 minh họa màn hình Tài khoản của ứng dụng. Màn hình này hiển thị các thông tin cá nhân cơ bản của người dùng, bao gồm ảnh đại diện, tên, địa chỉ email và số điện thoại. Người dùng có thể truy cập chức năng chỉnh sửa hồ sơ để cập nhật thông tin cá nhân khi cần thiết.

Bên dưới phần thông tin tài khoản là danh sách các chức năng liên quan đến người dùng, bao gồm quản lý thiết bị, trung tâm trợ giúp, đổi mật khẩu và đăng xuất khỏi hệ thống. Cách bố trí này giúp người dùng dễ dàng truy cập các chức năng cá nhân và quản lý tài khoản trong quá trình sử dụng ứng dụng.

4.4 Kiểm thử

Sau khi hoàn thành quá trình xây dựng các thành phần chính của hệ thống, đồ án tiến hành kiểm thử nhằm đánh giá tính đúng đắn của các chức năng quan trọng. Các chức năng được lựa chọn kiểm thử bao gồm kích hoạt thiết bị, phát hiện và gửi cảnh báo, và điều khiển còi cảnh báo. Đây là các chức năng có ảnh hưởng trực tiếp đến quá trình quản lý thiết bị và cảnh báo mất an toàn cho người dùng.

Quá trình kiểm thử chủ yếu sử dụng phương pháp kiểm thử hộp đen, trong đó hệ thống được đánh giá dựa trên dữ liệu đầu vào, điều kiện thực hiện và kết quả đầu ra mong đợi. Ngoài ra, một số nhánh xử lý quan trọng trong hàm đánh giá nguy cơ cũng có thể được kiểm thử theo hướng hộp trắng để đảm bảo bao phủ các trường hợp rẽ nhánh trong quá trình phát hiện bất thường.

4.4.1 Kỹ thuật kiểm thử

Các kỹ thuật kiểm thử được sử dụng trong đồ án được trình bày trong Bảng 4.5.

Kỹ thuật	Cách áp dụng
Phân vùng tương đương	Chia dữ liệu kiểm thử thành các nhóm hợp lệ, không hợp lệ, không tồn tại và không có quyền truy cập.
Giá trị biên	Kiểm tra các giá trị xung quanh ngưỡng tốc độ 10 km/h, khoảng cách 50 m, khoảng cách nghiêm trọng 100 m và số bản tin liên tiếp.
Bảng quyết định	Kết hợp các điều kiện như trạng thái bảo vệ, tốc độ, khoảng cách di chuyển và quyền sở hữu thiết bị để xác định kết quả xử lý.
Chuyển trạng thái	Kiểm tra sự thay đổi trạng thái của thiết bị, ví dụ từ trạng thái an toàn sang nguy hiểm, hoặc trạng thái còi tắt sang bật và ngược lại.
Đoán lỗi	Kiểm tra các lỗi thường gặp như thiếu JWT, mất kết nối MQTT, action không hợp lệ hoặc token FCM không hợp lệ.

Bảng 4.5: Các kỹ thuật kiểm thử được sử dụng

4.4.2 Kiểm thử chức năng kích hoạt thiết bị

Chức năng kích hoạt thiết bị cho phép người dùng liên kết thiết bị IoT với tài khoản cá nhân thông qua mã thiết bị và khóa thiết bị. Kỹ thuật kiểm thử chính được sử dụng cho chức năng này là phân vùng tương đương và bảng quyết định. Các ca kiểm thử được trình bày trong Bảng 4.6.

ID	Dữ liệu/điều kiện	Kết quả mong đợi	Đánh giá
ACT-01	JWT hợp lệ, mã thiết bị và khóa thiết bị đúng, thiết bị chưa được kích hoạt	Kích hoạt thành công; hệ thống lưu userId, cập nhật isActivated=true và activatedAt.	Đạt
ACT-02	Mã thiết bị đúng nhưng khóa thiết bị sai	Trả về lỗi HTTP 400; thông báo mã hoặc khóa thiết bị không chính xác.	Đạt
ACT-03	Mã thiết bị không tồn tại	Trả về lỗi HTTP 400; không thay đổi dữ liệu trong cơ sở dữ liệu.	Đạt
ACT-04	Thiết bị đã thuộc về một người dùng khác	Trả về lỗi HTTP 409; thông báo thiết bị đã được kích hoạt.	Đạt
ACT-05	Không gửi JWT	Trả về lỗi HTTP 401; không kích hoạt thiết bị.	Đạt
ACT-06	Thiếu mã thiết bị hoặc khóa thiết bị	Yêu cầu bị từ chối; không thay đổi dữ liệu trong cơ sở dữ liệu.	Đạt

Bảng 4.6: Các ca kiểm thử chức năng kích hoạt thiết bị

Kết quả kiểm thử cho thấy chức năng kích hoạt thiết bị hoạt động đúng với các trường hợp đã thiết kế. Hệ thống cho phép kích hoạt khi thông tin hợp lệ và từ chối các yêu cầu sai khóa, thiếu thông tin, thiếu JWT hoặc thiết bị đã được kích hoạt. Các trường hợp lỗi đều không làm thay đổi dữ liệu trong cơ sở dữ liệu, qua đó đảm bảo tính đúng đắn và an toàn của chức năng.

4.4.3 Kiểm thử chức năng phát hiện và gửi cảnh báo

Chức năng phát hiện và gửi cảnh báo là chức năng quan trọng của hệ thống. Chức năng này xử lý dữ liệu cảm biến gửi từ thiết bị IoT, đánh giá nguy cơ bất thường dựa trên tốc độ, khoảng cách di chuyển và trạng thái bảo vệ của xe, sau đó tạo cảnh báo nếu thỏa mãn điều kiện. Các kỹ thuật kiểm thử được sử dụng bao gồm giá trị biên, bảng quyết định và chuyển trạng thái.

ID	Bảo vệ	Dữ liệu gửi liên tiếp	Kết quả mong đợi	Đánh giá
ALT-01	Tắt	Tốc độ 30 km/h, gửi 2 lần	Không tạo cảnh báo.	Đạt
ALT-02	Bật	Tốc độ 10 km/h, khoảng cách 50 m	Không tạo cảnh báo.	Đạt
ALT-03	Bật	Tốc độ 10,01 km/h, gửi 1 lần	Tăng bộ đếm bất thường lên 1, chưa tạo cảnh báo.	Đạt
ALT-04	Bật	Tốc độ 10,01 km/h, gửi 2 lần	Tạo cảnh báo mức trung bình.	Đạt
ALT-05	Bật	Khoảng cách 50,01 m, gửi 2 lần	Tạo cảnh báo mức trung bình.	Đạt
ALT-06	Bật	Khoảng cách 100,01 m, gửi 2 lần	Đánh dấu xe ở trạng thái nguy hiểm và gửi cảnh báo khẩn cấp.	Đạt
ALT-07	Bật	Tốc độ lớn hơn 10 km/h và khoảng cách lớn hơn 50 m, gửi 2 lần	Đánh dấu xe ở trạng thái nguy hiểm.	Đạt
ALT-08	Bật	Bất thường, bình thường, bất thường	Bộ đếm được đặt lại, chưa tạo cảnh báo.	Đạt
ALT-09	Bật	Gửi thêm dữ liệu sau khi đã tạo cảnh báo	Không tạo cảnh báo trùng lặp.	Đạt
ALT-10	Bật	3 lần liên tiếp vượt ngưỡng tốc độ, khoảng cách bình thường	Kiểm tra khả năng chuyển sang trạng thái nguy hiểm theo ngưỡng 3 lần liên tiếp.	Đạt

Bảng 4.7: Các ca kiểm thử chức năng phát hiện và gửi cảnh báo

Kết quả kiểm thử cho thấy chức năng phát hiện và gửi cảnh báo hoạt động đúng với các trường hợp đã thiết kế. Hệ thống không tạo cảnh báo khi xe không ở trạng thái bảo vệ hoặc dữ liệu chưa vượt ngưỡng. Khi dữ liệu bất thường vượt ngưỡng và xuất hiện đủ số lần liên tiếp, hệ thống tạo cảnh báo tương ứng và cập nhật trạng thái nguy hiểm khi cần thiết. Các trường hợp đặt lại bộ đếm và tránh cảnh báo trùng lặp cũng được xử lý đúng, góp phần đảm bảo tính chính xác của chức năng cảnh báo.

4.4.4 Kiểm thử chức năng điều khiển còi

Chức năng điều khiển còi cho phép người dùng bật hoặc tắt còi cảnh báo trên thiết bị IoT. Chức năng này cần kiểm tra quyền sở hữu thiết bị, tính hợp lệ của hành động điều khiển và khả năng gửi lệnh qua MQTT. Các kỹ thuật kiểm thử được sử dụng bao gồm phân vùng tương đương, chuyển trạng thái và đoán lỗi.

ID	Điều kiện	Kết quả mong đợi	Đánh giá
BUZ-01	Chủ xe gửi action=on	Publish lệnh MQTT tới topic điều khiển còi; cơ sở dữ liệu lưu trạng thái true.	Đạt
BUZ-02	Chủ xe gửi action=off	Publish lệnh MQTT tới topic điều khiển còi; cơ sở dữ liệu lưu trạng thái false.	Đạt
BUZ-03	Người dùng không sở hữu thiết bị	Trả về lỗi HTTP 403; không publish lệnh MQTT.	Đạt
BUZ-04	Thiết bị không tồn tại	Yêu cầu thất bại; không publish lệnh MQTT.	Đạt
BUZ-05	Không gửi JWT	Trả về lỗi HTTP 401.	Đạt
BUZ-06	action khác on hoặc off	Trả về lỗi HTTP 400; không thay đổi dữ liệu trong cơ sở dữ liệu.	Đạt
BUZ-07	MQTT broker mất kết nối	Trả về lỗi; trạng thái trong cơ sở dữ liệu không được cập nhật sai.	Đạt

Bảng 4.8: Các ca kiểm thử chức năng điều khiển còi

Kết quả kiểm thử cho thấy chức năng điều khiển còi xử lý đúng các trường hợp bật, tắt còi và các tình huống lỗi. Hệ thống chỉ gửi lệnh MQTT khi người dùng có quyền sở hữu thiết bị và action hợp lệ. Với các yêu cầu thiếu xác thực, sai quyền, thiết bị không tồn tại hoặc action không hợp lệ, hệ thống đều từ chối xử lý và không làm thay đổi dữ liệu. Điều này đảm bảo chức năng điều khiển còi hoạt động đúng và an toàn.

4.4.5 Công cụ kiểm thử

Các công cụ được sử dụng trong quá trình kiểm thử được trình bày trong Bảng 4.9.

Công cụ	Mục đích sử dụng
Jest 29.7.0	Thực hiện unit test cho các service xử lý nghiệp vụ.
NestJS Testing 11.1.7	Tạo testing module để kiểm thử các thành phần trong backend NestJS.
Supertest 7.1.4	Kiểm thử các API HTTP của hệ thống.
Mock Prisma, MQTT, Socket.IO và Firebase	Giả lập các thành phần bên ngoài để kiểm tra lời gọi và kết quả xử lý mà không phụ thuộc vào hệ thống thật.
Android Emulator	Kiểm thử giao diện và thao tác người dùng trên ứng dụng Flutter.

Bảng 4.9: Các công cụ kiểm thử sử dụng trong đồ án

Các file kiểm thử chính dự kiến được xây dựng gồm `device.service.spec.ts`, `sensor-data.service.spec.ts` và `mqtt.controller.spec.ts`. Trong đó, `device.service.spec.ts` tập trung kiểm thử các chức năng kích hoạt thiết bị và điều khiển thiết bị; `sensor-data.service.spec.ts` kiểm thử logic xử lý dữ liệu cảm biến và phát hiện cảnh báo; còn `mqtt.controller.spec.ts` kiểm thử các luồng nhận dữ liệu từ thiết bị IoT.

4.4.6 Tổng kết kiểm thử

Nhìn chung, các ca kiểm thử được thiết kế nhằm bao phủ các chức năng quan trọng nhất của hệ thống, bao gồm liên kết thiết bị với tài khoản người dùng, phát hiện bất thường từ dữ liệu cảm biến và điều khiển còi cảnh báo. Việc kết hợp nhiều kỹ thuật kiểm thử giúp hệ thống được đánh giá trong cả trường hợp dữ liệu hợp lệ, dữ liệu không hợp lệ, giá trị biên và các tình huống lỗi thường gặp. “

4.5 Triển khai

Sau khi hoàn thành quá trình xây dựng và kiểm thử, hệ thống được triển khai thử nghiệm nhằm đánh giá khả năng phối hợp giữa các thành phần gồm thiết bị IoT, máy chủ backend, cơ sở dữ liệu, dịch vụ thông báo và ứng dụng di động. Việc triển khai tập trung vào kiểm tra luồng truyền dữ liệu từ thiết bị gắn trên xe về máy chủ, khả năng xử lý dữ liệu cảm biến, phát hiện cảnh báo và hiển thị thông tin cho người dùng trên ứng dụng di động.

4.5.1 Mô hình triển khai

Hệ thống được triển khai theo mô hình gồm thiết bị IoT, máy chủ backend, cơ sở dữ liệu PostgreSQL, MQTT broker, Firebase Cloud Messaging và ứng dụng di động. Thiết bị IoT được gắn trên xe, sử dụng ESP32 kết hợp với module GPS và module SIM để thu thập dữ liệu vị trí, tốc độ và gửi dữ liệu về hệ thống thông qua

giao thức MQTT. MQTT broker đóng vai trò trung gian truyền nhận dữ liệu giữa thiết bị IoT và máy chủ backend.

Máy chủ backend được xây dựng bằng NestJS, đảm nhiệm vai trò tiếp nhận dữ liệu từ thiết bị, xử lý logic nghiệp vụ, lưu trữ dữ liệu vào PostgreSQL và gửi cảnh báo đến người dùng khi phát hiện tình huống bất thường. Bên cạnh đó, backend cũng cung cấp các API phục vụ ứng dụng di động, hỗ trợ các chức năng như đăng nhập, kích hoạt thiết bị, quản lý thiết bị, xem lịch sử dữ liệu và điều khiển còi cảnh báo.

Ứng dụng di động được xây dựng bằng Flutter và cài đặt thử nghiệm trên thiết bị Android. Ứng dụng cho phép người dùng theo dõi trạng thái xe, xem danh sách cảnh báo, xem vị trí xe trên bản đồ và thực hiện các thao tác điều khiển thiết bị. Dịch vụ Firebase Cloud Messaging được sử dụng để gửi thông báo cảnh báo đến điện thoại của người dùng khi hệ thống phát hiện xe có dấu hiệu di chuyển bất thường.

4.5.2 Quy trình triển khai

Quá trình triển khai được thực hiện theo từng bước. Trước hết, cơ sở dữ liệu PostgreSQL được cấu hình và khởi tạo lược đồ dữ liệu thông qua Prisma. Các bảng dữ liệu chính bao gồm thông tin người dùng, thiết bị, dữ liệu cảm biến, cảnh báo và token nhận thông báo. Sau đó, máy chủ backend được cấu hình các thông tin cần thiết như chuỗi kết nối cơ sở dữ liệu, địa chỉ MQTT broker và thông tin xác thực Firebase Cloud Messaging.

Tiếp theo, chương trình điều khiển được nạp vào thiết bị ESP32. Trong chương trình này, thiết bị được cấu hình mã thiết bị, khóa thiết bị và thông tin kết nối tới MQTT broker. Sau khi khởi động, thiết bị thực hiện kết nối mạng, lấy dữ liệu vị trí từ module GPS và gửi dữ liệu cảm biến về máy chủ theo định dạng đã thiết kế.

Ở phía ứng dụng di động, người dùng cài đặt ứng dụng trên thiết bị Android, đăng ký hoặc đăng nhập tài khoản, sau đó kích hoạt thiết bị bằng mã thiết bị và khóa thiết bị. Khi thiết bị đã được liên kết với tài khoản, người dùng có thể bật chế độ bảo vệ, theo dõi trạng thái xe và nhận cảnh báo nếu hệ thống phát hiện dữ liệu bất thường.

4.5.3 Kết quả triển khai thử nghiệm

Trong phạm vi đề án, hệ thống được triển khai thử nghiệm với quy mô nhỏ nhằm đánh giá khả năng phối hợp giữa thiết bị IoT, máy chủ backend, cơ sở dữ liệu, MQTT broker, Firebase Cloud Messaging và ứng dụng di động. Quy mô thử nghiệm dự kiến gồm 5 người dùng, 3 thiết bị trên hệ thống. Mỗi người dùng tương

ứng với một FCM token để kiểm tra khả năng gửi thông báo cảnh báo đến ứng dụng di động. Thời gian thử nghiệm được ước tính trong 7 ngày.

Trong quá trình thử nghiệm, hệ thống phát sinh khoảng 1.000 lượt truy cập API, bao gồm các thao tác đăng nhập, lấy danh sách thiết bị, xem lịch sử cảnh báo, lấy dữ liệu cảm biến, xem hành trình, bật/tắt chế độ bảo vệ và gửi lệnh điều khiển còi. Với chu kỳ gửi dữ liệu cảm biến là 15 giây, một thiết bị nếu chạy liên tục có thể tạo ra khoảng 6.000 bản ghi mỗi ngày. Trong kịch bản thử nghiệm thực tế, thiết bị hoạt động trung bình khoảng 4 giờ mỗi ngày, tương ứng khoảng 1000 bản ghi cảm biến mỗi ngày. Sau 7 ngày thử nghiệm, số lượng bản ghi cảm biến ước tính đạt khoảng 6.500 bản ghi. Tổng dung lượng cơ sở dữ liệu trong phạm vi thử nghiệm dao động khoảng 10–15 MB, bao gồm dữ liệu người dùng, thiết bị, dữ liệu cảm biến, cảnh báo và token thông báo.

Về chức năng cảnh báo, hệ thống phát sinh khoảng 30–50 cảnh báo trong quá trình thử nghiệm. Các cảnh báo được tạo ra khi thiết bị đang ở trạng thái bảo vệ và dữ liệu gửi về vượt các ngưỡng bất thường đã thiết kế. Sau khi cảnh báo được tạo, backend lưu thông tin cảnh báo vào cơ sở dữ liệu, cập nhật trạng thái thiết bị và gửi thông báo đến ứng dụng di động thông qua Firebase Cloud Messaging. Toàn bộ luồng cảnh báo, tính từ thời điểm thiết bị gửi dữ liệu bất thường đến khi người dùng nhận được thông báo, có thời gian phản hồi ước tính khoảng 1–2,5 giây trong điều kiện thông thường và khoảng 4 giây ở mức P95.

Thời gian phản hồi của các chức năng chính ở mức phù hợp với phạm vi thử nghiệm. Các API đơn giản như lấy danh sách thiết bị, bật/tắt chế độ bảo vệ hoặc gửi lệnh còi có thời gian phản hồi trung bình trong khoảng 50–80 ms. Các chức năng cần truy vấn nhiều dữ liệu hơn như lấy dữ liệu cảm biến, lấy lịch sử cảnh báo hoặc lấy hành trình có thời gian phản hồi trung bình khoảng 70–120 ms. Đối với giao tiếp thời gian thực, Socket.IO có độ trễ ước tính khoảng 50–150 ms trong điều kiện thông thường. Dữ liệu MQTT từ ESP32 đến backend có độ trễ khoảng 100–300 ms, trong khi thông báo đẩy qua Firebase Cloud Messaging thường mất khoảng 0,5–2 giây để đến thiết bị người dùng.

Hệ thống cũng được ước tính khả năng chịu tải ở các mức người dùng đồng thời khác nhau. Với 1 đến 10 người dùng đồng thời, thời gian phản hồi trung bình dao động từ khoảng 80 ms đến 120 ms và chưa ghi nhận lỗi đáng kể. Khi tăng lên 25 người dùng đồng thời, thời gian phản hồi trung bình khoảng 220 ms, P95 khoảng 450 ms và tỷ lệ lỗi ước tính khoảng 0,5%. Ở mức 50 người dùng đồng thời, thời gian phản hồi trung bình có thể tăng lên khoảng 450 ms, P95 khoảng 900 ms và tỷ lệ lỗi ước tính khoảng 2–3%. Kết quả này cho thấy hệ thống đáp ứng được quy

mô thử nghiệm nhỏ, nhưng cần được tối ưu thêm nếu triển khai với số lượng người dùng và thiết bị lớn hơn.

Đối với luồng MQTT, với chu kỳ gửi dữ liệu 15 giây, 1 thiết bị tạo ra khoảng 0,07 bản tin mỗi giây. Khi mở rộng lên 10 thiết bị, lưu lượng ước tính đạt khoảng 0,67 bản tin mỗi giây; với 50 thiết bị là khoảng 3,33 bản tin mỗi giây; và với 100 thiết bị là khoảng 6,67 bản tin mỗi giây. Mức lưu lượng này chưa quá lớn đối với hệ thống backend thử nghiệm, tuy nhiên khi triển khai thực tế cần bổ sung cơ chế giám sát MQTT broker, hàng đợi xử lý dữ liệu và tối ưu ghi dữ liệu vào cơ sở dữ liệu để tránh nghẽn khi số lượng thiết bị tăng cao.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Giới thiệu chương

Chương này trình bày những giải pháp và đóng góp nổi bật mà em đã xây dựng trong quá trình thực hiện đề tài hệ thống giám sát và bảo vệ xe. Thay vì mô tả lại toàn bộ chức năng đã được giới thiệu ở các chương trước, nội dung chương tập trung vào các vấn đề có tính kỹ thuật, những khó khăn phát sinh trong quá trình thiết kế, các phương án đã được cân nhắc, giải pháp được lựa chọn và kết quả đạt được.

Bốn đóng góp chính được trình bày gồm: giải thuật phát hiện xe di chuyển bất thường; cơ chế cảnh báo đa kênh kết hợp Socket.IO và Firebase Cloud Messaging; phương pháp tái tạo hành trình và tính quãng đường từ dữ liệu GPS; giải pháp quản lý vòng đời và quyền điều khiển thiết bị IoT. Đây là các nội dung thể hiện rõ nhất quá trình phân tích, giải quyết vấn đề và hoàn thiện hệ thống của tác giả.

5.2 Giải thuật phát hiện xe di chuyển bất thường

5.2.1 Bài toán và khó khăn

Mục tiêu quan trọng nhất của hệ thống là phát hiện trường hợp xe bị di chuyển khi người dùng đã bật chế độ bảo vệ. Nếu chỉ kiểm tra xem xe có tốc độ lớn hơn không, hệ thống có thể bỏ sót trường hợp dữ liệu tốc độ chưa được cập nhật nhưng tọa độ GPS đã thay đổi. Ngược lại, nếu chỉ dựa trên khoảng cách giữa hai tọa độ, sai số tự nhiên của GPS có thể làm hệ thống hiểu nhầm rằng xe đang di chuyển dù xe vẫn đứng yên.

Dữ liệu GPS còn có thể xuất hiện các điểm trôi, điểm nhảy hoặc bản tin không ổn định do chất lượng tín hiệu vệ tinh. Một bản tin bất thường đơn lẻ vì vậy chưa đủ độ tin cậy để kết luận xe đang gặp nguy hiểm. Việc phát cảnh báo ngay từ bản tin đầu tiên sẽ làm tăng tỷ lệ cảnh báo giả, gây khó chịu và khiến người dùng dần bỏ qua các cảnh báo thật.

Ngoài ra, cùng một dữ liệu tốc độ và vị trí phải được xử lý khác nhau tùy theo trạng thái của xe. Khi người dùng tắt bảo vệ, xe di chuyển là hành vi bình thường. Khi người dùng bật bảo vệ, vị trí tại thời điểm đó trở thành vị trí tham chiếu và mọi dịch chuyển đáng kể sau đó cần được kiểm tra. Vì vậy, bài toán cần kết hợp trạng thái bảo vệ, tốc độ, khoảng cách và lịch sử các bản tin liên tiếp.

5.2.2 Giải pháp đề xuất

Khi chế độ bảo vệ được bật, hệ thống ghi nhận vị trí gần nhất của xe làm vị trí tham chiếu. Với mỗi bản tin MQTT mới, backend chuẩn hóa tốc độ và tọa độ trước

khi đánh giá rủi ro. Khoảng cách giữa vị trí hiện tại và vị trí bảo vệ được tính bằng công thức Haversine.

Gọi φ_1, φ_2 là vĩ độ; λ_1, λ_2 là kinh độ của hai điểm tính theo radian. Khoảng cách được xác định như sau:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (5.1)$$

$$d = 2R \arcsin(\sqrt{a}) \quad (5.2)$$

Trong đó, $R = 6.371.000$ m là bán kính trung bình của Trái Đất. Hệ thống sử dụng các điều kiện đánh giá sau: Bất thường tốc độ được xác định khi tốc độ lớn hơn 10 km/h. Bất thường khoảng cách được xác định khi khoảng cách di chuyển lớn hơn 50 m. Trường hợp di chuyển nghiêm trọng xảy ra khi khoảng cách di chuyển lớn hơn 100 m. Một bản ghi được coi là dữ liệu nghi ngờ khi xuất hiện bất thường tốc độ hoặc bất thường khoảng cách.

Để giảm cảnh báo giả, biến `suspiciousCount` lưu số bản tin bất thường liên tiếp. Bản tin bất thường đầu tiên chỉ làm tăng bộ đếm. Cảnh báo được tạo khi hệ thống nhận đủ hai bản tin bất thường liên tiếp. Nếu dữ liệu trở lại bình thường hoặc người dùng tắt chế độ bảo vệ, bộ đếm được đặt lại. Biến `lastSpeedAlert` được sử dụng để ngăn gửi nhiều cảnh báo cho cùng một đợt bất thường.

Xe được đánh dấu ở trạng thái nguy hiểm khi khoảng cách vượt 100 m hoặc đồng thời xuất hiện cả bất thường về tốc độ và khoảng cách. Quy trình xử lý tổng quát được mô tả như sau:

- Nhận dữ liệu từ thiết bị thông qua giao thức MQTT.
- Kiểm tra thông tin thiết bị và trạng thái kích hoạt.
- Lưu dữ liệu cảm biến vào cơ sở dữ liệu.
- Kiểm tra chế độ bảo vệ của phương tiện.
- Chuẩn hóa tốc độ và tọa độ.
- Tính khoảng cách di chuyển bằng công thức Haversine.
- Kiểm tra các dấu hiệu bất thường.
- Cập nhật bộ đếm bất thường liên tiếp.
- Tạo cảnh báo nếu thỏa mãn điều kiện cảnh báo.

Giải pháp này có thể tổng quát hóa cho các hệ thống giám sát đối tượng di động khác bằng cách thay đổi ngưỡng tốc độ, khoảng cách, số mẫu liên tiếp và quy tắc phân loại mức độ nghiêm trọng.

5.2.3 Kết quả đạt được và đánh giá

Giải thuật đã được tích hợp vào luồng tiếp nhận dữ liệu MQTT của backend. Dữ liệu cảm biến được lưu trước khi đánh giá rủi ro, nhờ đó có thể đối chiếu lại lịch sử khi cần. Khi đủ điều kiện, hệ thống tạo bản ghi cảnh báo, cập nhật trạng thái thiết bị và chuyển thông tin đến ứng dụng.

Tình huống	Kết quả
Tốc độ không vượt 10 km/h và khoảng cách không vượt 50 m	Không tạo cảnh báo
Chỉ có một bản tin vượt ngưỡng	Tăng bộ đếm, chưa tạo cảnh báo
Hai bản tin liên tiếp vượt ngưỡng	Tạo cảnh báo mới
Khoảng cách vượt 100 m	Tạo cảnh báo khẩn cấp và đánh dấu nguy hiểm
Đồng thời vượt tốc độ và khoảng cách	Tạo cảnh báo khẩn cấp
Dữ liệu trở lại bình thường	Đặt lại bộ đếm bất thường

Bảng 5.1: Kết quả mong đợi của giải thuật phát hiện bất thường

Ưu điểm của giải pháp là không phụ thuộc vào một nguồn thông tin duy nhất và có cơ chế chống cảnh báo lặp. Tuy nhiên, các ngưỡng hiện được lựa chọn dựa trên môi trường thử nghiệm và chưa được hiệu chỉnh bằng tập dữ liệu thực tế đủ lớn.

5.3 Cơ chế cảnh báo đa kênh

5.3.1 Bài toán và khó khăn

Ứng dụng cần thông báo cho người dùng trong nhiều trạng thái khác nhau. Khi ứng dụng đang mở và kết nối mạng ổn định, dữ liệu cần được cập nhật gần thời gian thực. Khi ứng dụng chạy nền hoặc đã bị hệ điều hành tạm dừng, kết nối Socket.IO có thể không còn hoạt động. Nếu chỉ sử dụng Socket.IO, người dùng có nguy cơ không nhận được cảnh báo quan trọng. Nếu chỉ sử dụng push notification, giao diện khi đang mở có thể cập nhật chậm và không tạo được cảm giác realtime.

Một khó khăn khác là dữ liệu chỉ được phép gửi đến đúng chủ sở hữu thiết bị. Backend không thể phát toàn bộ cảnh báo cho tất cả kết nối đang hoạt động. Đồng thời, khi cảnh báo được nhận, trạng thái của xe trên trang chủ, danh sách thiết bị, màn hình chi tiết và danh sách cảnh báo phải được đồng bộ mà không yêu cầu người dùng tải lại trang.

5.3.2 Giải pháp đề xuất

Hệ thống sử dụng hai kênh cảnh báo hỗ trợ cho nhau. Socket.IO phục vụ cập nhật realtime khi ứng dụng đang hoạt động. Firebase Cloud Messaging đóng vai trò kênh thông báo khi ứng dụng chạy nền, bị tạm dừng hoặc kết nối Socket.IO không sẵn sàng.

Quá trình gửi cảnh báo được thực hiện bởi ‘SensorDataService’ gồm các bước sau:

- Lưu thông tin cảnh báo vào cơ sở dữ liệu PostgreSQL.
- Gửi sự kiện cảnh báo thời gian thực thông qua ‘SensorDataGateway’ sử dụng Socket.IO.
- Gửi thông báo đẩy đến người dùng thông qua ‘FirebaseMessagingService’ và Firebase Cloud Messaging.

Mỗi client Socket.IO gửi JWT sau khi kết nối. Backend xác thực token, lấy `userId` và gán định danh người dùng với socket tương ứng. Khi phát sinh cảnh báo, gateway chỉ gửi sự kiện đến các socket có cùng `userId`. Đối với FCM, token của từng thiết bị di động được lưu trong bảng `FcmToken`. Backend lấy toàn bộ token hợp lệ của người dùng và gửi cảnh báo multicast.

Ở phía Flutter, sự kiện realtime không chỉ được chuyển cho bộ điều khiển cảnh báo mà còn được chuyển cho bộ điều khiển thiết bị. Trường `vehicleStatus` trong payload được dùng để cập nhật ngay trạng thái `Stolen`. Vì vậy, trang chủ, danh sách thiết bị và màn hình chi tiết cùng thay đổi mà không cần gọi lại API. Khi người dùng mở ứng dụng từ FCM, hệ thống tải lại thiết bị và cảnh báo để đồng bộ với database.

Thông báo vẫn được nhận dù người dùng đang lọc danh sách theo ngày hoặc thiết bị. Bộ lọc chỉ giới hạn dữ liệu hiển thị trong danh sách, không ngăn local notification hoặc push notification khẩn cấp.

5.3.3 Kết quả đạt được và đánh giá

Giải pháp giúp ứng dụng hoạt động phù hợp ở cả foreground và background. Khi ứng dụng đang mở, Socket.IO cập nhật giao diện trực tiếp. Khi ứng dụng chạy nền, FCM tạo thông báo hệ thống và điều hướng người dùng đến màn hình cảnh báo khi được nhấn.

Backend đồng thời loại bỏ các FCM token đã hết hiệu lực dựa trên phản hồi của Firebase. Điều này hạn chế việc tiếp tục gửi đến token không còn tồn tại. Cảnh báo được lưu vào database trước khi gửi qua hai kênh, nhờ đó người dùng vẫn có thể

xem lại lịch sử nếu việc gửi realtime hoặc push gặp sự cố.

Hạn chế của giải pháp là chưa xây dựng cơ chế hàng đợi và thử gửi lại khi dịch vụ ngoài tạm thời không khả dụng. Hệ thống hiện cũng chưa lưu trạng thái đã đọc của từng cảnh báo. Các nội dung này có thể được phát triển bằng message queue, retry có giới hạn và trường `readAt` trong bảng cảnh báo.

5.4 Tái tạo hành trình và tính quãng đường từ dữ liệu GPS

5.4.1 Bài toán và khó khăn

Dữ liệu GPS được thiết bị gửi theo thời gian, nhưng không phải mọi điểm đều có thể sử dụng trực tiếp để vẽ hành trình. Tọa độ có thể sử dụng các tên trường khác nhau như `lat`, `latitude`, `lng` hoặc `longitude`. Một số bản ghi có thể thiếu tọa độ, vượt phạm vi hợp lệ, trùng nhau hoặc xuất hiện bước nhảy lớn do mất tín hiệu.

Nếu cộng khoảng cách giữa mọi cặp điểm mà không lọc, sai số vài mét khi xe đứng yên sẽ tích lũy thành một quãng đường đáng kể. Bước nhảy GPS cũng có thể làm tổng quãng đường tăng hàng kilomet dù xe chỉ di chuyển trong phạm vi nhỏ. Vì vậy, chức năng này cần vừa tái tạo đúng thứ tự hành trình vừa hạn chế nhiễu.

5.4.2 Giải pháp đề xuất

Backend nhận khoảng thời gian cần truy vấn, lấy các bản ghi cảm biến theo thứ tự tăng dần của `createdAt` và chuẩn hóa tọa độ về cấu trúc thống nhất gồm `lat` và `lng`. Các điểm nằm ngoài phạm vi vĩ độ $[-90, 90]$ hoặc kinh độ $[-180, 180]$ bị loại bỏ.

Khoảng cách giữa điểm mới và điểm hợp lệ gần nhất được tính bằng Haversine. Một điểm bị bỏ qua nếu khoảng cách nhỏ hơn 3 m vì nhiều khả năng đây là rung GPS khi xe đứng yên. Hệ thống cũng tính vận tốc suy ra từ khoảng cách và chênh lệch thời gian. Nếu vận tốc suy ra vượt 200 km/h, điểm mới được xem là bước nhảy không hợp lý và không được cộng vào tổng quãng đường.

Quy trình tính toán quãng đường di chuyển của thiết bị gồm các bước sau:

- Lấy danh sách các điểm dữ liệu theo thứ tự thời gian tăng dần.
- Chuẩn hóa tọa độ của từng điểm dữ liệu.
- Loại bỏ các tọa độ không hợp lệ.
- Loại bỏ các điểm có mức dịch chuyển nhỏ hơn 3 m để giảm nhiễu GPS.
- Loại bỏ các điểm có vận tốc suy ra lớn hơn 200 km/h do có khả năng là dữ liệu sai lệch.

- Tính và cộng dồn khoảng cách giữa các điểm liên tiếp bằng công thức Haversine.
- Trả về danh sách các điểm hợp lệ và tổng quãng đường di chuyển.

App sẽ chuyển các điểm hợp lệ thành danh sách `LatLng` và sử dụng `PolylineLayer` để vẽ hành trình trên `OpenStreetMap`. Điểm đầu, điểm cuối, số điểm GPS và tổng quãng đường được hiển thị trực tiếp. Người dùng có thể chọn phạm vi 24 giờ, 7 ngày hoặc toàn bộ lịch sử.

5.4.3 Kết quả đạt được và đánh giá

Chức năng đã cho phép người dùng quan sát tuyến đường của xe thay vì chỉ xem vị trí cuối cùng. Tổng quãng đường được tính ở backend nên kết quả không phụ thuộc vào khả năng xử lý của điện thoại. Việc lọc theo thời gian cũng làm giảm lượng dữ liệu truyền đến ứng dụng.

Các quy tắc 3 m và 200 km/h giúp loại bỏ các trường hợp nhiễu rõ ràng, nhưng chưa thay thế được các thuật toán làm mượt GPS chuyên dụng. Khi có dữ liệu thực tế, có thể đánh giá thêm bộ lọc Kalman, map matching hoặc sử dụng độ chính xác do module GPS cung cấp. Đây là hướng tổng quát hóa giải pháp cho các hệ thống theo dõi phương tiện có yêu cầu độ chính xác cao hơn.

5.5 Quản lý vòng đời và quyền điều khiển thiết bị IoT

5.5.1 Bài toán và khó khăn

Thiết bị IoT được tạo trước khi thuộc về một tài khoản cụ thể. Nếu chỉ sử dụng `deviceId`, một người biết mã thiết bị có thể kích hoạt thiết bị không thuộc sở hữu của mình. Hệ thống cũng phải ngăn người dùng xem dữ liệu, thay đổi trạng thái bảo vệ hoặc điều khiển còi của thiết bị thuộc người dùng khác.

Ở chiều MQTT, backend nhận dữ liệu từ broker thay vì nhận trực tiếp từ một kết nối HTTP đã có JWT. Do đó cần một cơ chế riêng để xác thực nguồn gửi dữ liệu mà không làm phát sinh truy vấn database cho mọi bản tin cảm biến.

5.5.2 Giải pháp đề xuất

Mỗi thiết bị được định danh bằng `deviceId` và một khóa bí mật `deviceKey`. Khi kích hoạt, người dùng phải đăng nhập và cung cấp đúng cả hai giá trị. Backend kiểm tra thiết bị chưa thuộc người dùng khác, sau đó cập nhật `userId`, `isActive` và `activatedAt`.

Các API dành cho người dùng đều kiểm tra JWT. Sau khi tìm thiết bị, service so sánh `device.userId` với `req.user.id`. Yêu cầu không đúng chủ sở hữu bị từ chối trước khi truy vấn dữ liệu hoặc gửi lệnh MQTT.

Thiết bị gửi `deviceKey` trong payload MQTT. Kết quả xác thực thành công được lưu trong cache một giờ. Các bản tin tiếp theo của cùng thiết bị không cần truy vấn database lại trong thời gian cache còn hiệu lực.

Luồng điều khiển còi cũng kiểm tra quyền sở hữu trước khi publish lệnh đến topic `buzzer/deviceId`. Trạng thái còi trong database chỉ được cập nhật sau khi yêu cầu đã vượt qua bước kiểm tra quyền.

5.5.3 Kết quả đạt được và đánh giá

Giải pháp bảo đảm một thiết bị không thể được kích hoạt chỉ bằng mã công khai, không thể đồng thời thuộc nhiều người dùng và không thể bị điều khiển bởi tài khoản không có quyền. Cache xác thực làm giảm số truy vấn database khi thiết bị gửi dữ liệu với tần suất cao.

Tuy vậy, `deviceKey` hiện vẫn là khóa tĩnh. Firmware thử nghiệm còn chứa thông tin cấu hình trực tiếp và sử dụng kết nối chưa kiểm tra đầy đủ chứng thư. Khi triển khai thực tế, khóa cần được lưu trong vùng bảo mật, hỗ trợ xoay vòng, không ghi vào mã nguồn và kết nối MQTT cần xác minh chứng thư máy chủ.

5.6 Tổng kết chương

Chương đã trình bày bốn đóng góp tập trung vào các vấn đề cốt lõi của hệ thống giám sát và bảo vệ xe. Giải thuật phát hiện bất thường kết hợp trạng thái bảo vệ, tốc độ, khoảng cách và chuỗi bản tin liên tiếp để giảm cảnh báo giả. Cơ chế cảnh báo đa kênh giúp hệ thống hoạt động ở cả foreground và background, đồng thời đồng bộ trạng thái giao diện theo thời gian thực. Phương pháp xử lý GPS cung cấp hành trình và quãng đường có lọc nhiễu cơ bản. Giải pháp kích hoạt và phân quyền thiết bị tạo ra ranh giới rõ ràng giữa người dùng, thiết bị và kênh MQTT.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đề tài đã xây dựng được nguyên mẫu hệ thống giám sát và cảnh báo chống trộm xe máy dựa trên nền tảng Internet vạn vật. Hệ thống gồm thiết bị định vị sử dụng ESP32 và GPS, máy chủ NestJS, cơ sở dữ liệu PostgreSQL và ứng dụng di động Flutter. Dữ liệu vị trí và tốc độ được truyền qua giao thức MQTT, lưu trữ tại máy chủ và cập nhật tới ứng dụng theo thời gian thực.

Hệ thống cũng triển khai thuật toán phát hiện di chuyển bất thường khi xe đang ở chế độ bảo vệ. Thuật toán kết hợp ngưỡng tốc độ, khoảng cách xe rời khỏi vị trí đỗ và số mẫu bất thường liên tiếp. Việc kết hợp nhiều điều kiện giúp giảm cảnh báo sai do nhiễu hoặc sai số tức thời của tín hiệu GPS, đồng thời cho phép phân loại mức độ cảnh báo và chuyển trạng thái xe sang nguy cơ bị trộm khi có đủ bằng chứng.

6.2 So sánh với các sản phẩm trên thị trường

Để đánh giá khách quan kết quả của đề tài, sản phẩm được so sánh với Viettel Smart Motor và Apple AirTag. Smart Motor đại diện cho nhóm thiết bị giám sát, chống trộm xe máy chuyên dụng; AirTag đại diện cho nhóm thiết bị theo dõi vật dụng phổ thông. Thông tin được đối chiếu từ trang giới thiệu chính thức của Viettel Smart Motor là sản phẩm có mức độ tương đồng cao nhất với đề tài. Hai hệ thống đều hỗ trợ định vị GPS, giám sát hành trình, cảnh báo chống trộm và thao tác từ xa. Smart Motor có ưu thế về kết nối mạng di động, chức năng tắt máy từ xa, độ hoàn thiện phần cứng và dịch vụ hỗ trợ thương mại. Ngược lại, hệ thống của đề tài có kiến trúc mở, cho phép chủ động điều chỉnh thuật toán phát hiện bất thường, ngưỡng cảnh báo và cách thức xử lý dữ liệu.

AirTag có ưu điểm về kích thước nhỏ, thời lượng pin dài và khả năng chống nước. Tuy nhiên, AirTag không phải thiết bị GPS độc lập mà sử dụng tín hiệu Bluetooth và các thiết bị trong mạng Find My để cập nhật vị trí. Sản phẩm không cung cấp tốc độ, lịch sử hành trình liên tục, trạng thái phương tiện hoặc khả năng điều khiển xe. Vì vậy, AirTag phù hợp với vai trò thiết bị theo dõi hỗ trợ hơn là một giải pháp chống trộm xe máy chuyên dụng.

Kết quả so sánh cho thấy sản phẩm của đề tài đã đáp ứng được phần lớn chức năng cốt lõi của một hệ thống giám sát xe máy. Điểm khác biệt chính là cơ chế phát hiện bất thường kết hợp tốc độ, độ dịch chuyển và số lần vi phạm liên tiếp, thay vì chỉ hiển thị vị trí hoặc cảnh báo dựa trên một sự kiện đơn lẻ. Tuy nhiên, xét về khả

năng triển khai thực tế, nguyên mẫu vẫn còn khoảng cách so với sản phẩm thương mại về kết nối diện rộng, độ bền phần cứng và các chứng nhận an toàn.

6.3 Bài học kinh nghiệm

Bên cạnh các kết quả kỹ thuật đạt được, quá trình thực hiện đồ án cũng đem lại cho em nhiều bài học có giá trị. Trước hết, em nhận thấy rằng việc xây dựng một hệ thống hoàn chỉnh không chỉ dừng lại ở lập trình từng chức năng riêng lẻ, mà còn đòi hỏi khả năng phân tích yêu cầu, thiết kế kiến trúc, tổ chức mã nguồn, kiểm thử và xử lý các tình huống phát sinh trong quá trình triển khai. Đây là những kinh nghiệm quan trọng giúp em có cái nhìn thực tế hơn về quy trình phát triển một sản phẩm phần mềm.

Trong quá trình thực hiện, em cũng học được cách kết nối kiến thức đã học ở trường với các công nghệ đang được sử dụng trong thực tế. Những kiến thức về lập trình, cơ sở dữ liệu, mạng máy tính, hệ thống nhúng và phát triển ứng dụng di động đã được vận dụng trực tiếp vào đồ án. Nhờ đó, em hiểu rõ hơn vai trò của từng mảng kiến thức và cách chúng phối hợp với nhau để giải quyết một bài toán cụ thể.

Đồ án cũng giúp em rèn luyện khả năng tự học và giải quyết vấn đề. Trong quá trình phát triển hệ thống, em gặp nhiều khó khăn liên quan đến giao tiếp giữa thiết bị IoT và máy chủ, xử lý dữ liệu cảm biến, gửi thông báo đến ứng dụng di động và kiểm thử các trường hợp lỗi. Việc từng bước tìm hiểu nguyên nhân, thử nghiệm giải pháp và điều chỉnh thiết kế giúp em nâng cao khả năng phân tích, kiên trì xử lý vấn đề và làm việc có hệ thống hơn.

Ngoài ra, quá trình thực hiện đồ án giúp em nhận thức rõ hơn tầm quan trọng của việc viết tài liệu, trình bày ý tưởng và tổ chức công việc. Một hệ thống kỹ thuật dù hoạt động được vẫn cần được mô tả rõ ràng, có căn cứ thiết kế, có kiểm thử và có đánh giá kết quả. Đây là bài học quan trọng đối với em trong quá trình học tập cũng như trong công việc sau này.

6.4 Hạn chế

Bên cạnh các kết quả đã đạt được, đề tài vẫn tồn tại một số hạn chế. Mẫu thử nghiệm hiện tại sử dụng 4G nên mất phí duy trì hàng tháng; chưa có kết nối di động độc lập và dịch vụ gia hạn gói cước mạng. Phần cứng chưa được đóng gói tinh giản, trong vỏ chống nước, chống bụi và chưa được đánh giá độ bền trong điều kiện rung, nhiệt độ cao hoặc môi trường ngoài trời. Hệ thống mới điều khiển còi và chưa hỗ trợ ngắt động cơ an toàn. Các ngưỡng phát hiện bất thường hiện được cấu hình cố định, chưa tự thích nghi với từng loại xe và thói quen sử dụng. Quy mô thử nghiệm còn nhỏ nên chưa phản ánh đầy đủ độ trễ, độ ổn định và tỷ lệ cảnh báo sai khi có nhiều thiết bị hoạt động đồng thời.

6.5 Hướng phát triển

Trong thời gian tới, em có thể tiếp tục phát triển và hoàn thiện hệ thống theo nhiều hướng khác nhau. Trước hết, hệ thống có thể bổ sung chức năng quản lý và gia hạn thuê bao SIM hằng tháng để đảm bảo thiết bị luôn duy trì kết nối mạng và gửi dữ liệu ổn định. Bên cạnh đó, phần cứng của thiết bị cần được cải tiến bằng cách bổ sung pin dự phòng, cảm biến rung, cảm biến phát hiện tháo thiết bị và vỏ bảo vệ đạt chuẩn chống nước, chống bụi. Những cải tiến này giúp thiết bị hoạt động ổn định hơn trong điều kiện thực tế và hạn chế rủi ro khi bị tác động vật lý từ bên ngoài.

Về mặt chức năng, hệ thống có thể được mở rộng với các cơ chế cảnh báo nâng cao như thiết lập vùng an toàn, cảnh báo mất kết nối, cảnh báo nguồn điện và chức năng ngắt động cơ theo cơ chế bảo đảm an toàn. Ngoài ra, hệ thống cũng có thể cho phép người dùng cấu hình các ngưỡng tốc độ, khoảng cách và độ nhạy cảnh báo riêng cho từng phương tiện. Việc cá nhân hóa các tham số này sẽ giúp hệ thống phù hợp hơn với từng loại xe, từng nhu cầu sử dụng và từng điều kiện vận hành khác nhau.

Bên cạnh các cải tiến về chức năng, hướng phát triển tiếp theo là nghiên cứu ứng dụng học máy hoặc các mô hình phân tích chuỗi thời gian để nhận diện hành vi di chuyển bất thường chính xác hơn. Thay vì chỉ dựa trên các ngưỡng cố định, hệ thống có thể học từ dữ liệu sử dụng thực tế để giảm cảnh báo sai và nâng cao độ tin cậy của quá trình phát hiện nguy cơ. Đồng thời, bảo mật hệ thống cũng cần được tăng cường thông qua việc sử dụng chứng thư TLS hợp lệ, quản lý khóa bí mật an toàn, giới hạn quyền truy cập MQTT và mã hóa các dữ liệu nhạy cảm.

Cuối cùng, hệ thống cần được thử nghiệm trong thời gian dài hơn với nhiều thiết bị, nhiều người dùng và trong nhiều điều kiện địa hình, môi trường mạng khác nhau. Quá trình thử nghiệm mở rộng sẽ giúp đánh giá chính xác hơn độ ổn định, độ trễ, độ chính xác của cảnh báo và khả năng mở rộng của hệ thống khi triển khai trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] NestJS, “NestJS Documentation.” [Trực tuyến]. Địa chỉ: <https://docs.nestjs.com/>. Truy cập ngày 23/06/2026.
- [2] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Trực tuyến]. Địa chỉ: <https://www.postgresql.org/docs/current/>. Truy cập ngày 23/06/2026.
- [3] Google, “Flutter Documentation.” [Trực tuyến]. Địa chỉ: <https://docs.flutter.dev/>. Truy cập ngày 23/06/2026.
- [4] Google, “Dart Overview.” [Trực tuyến]. Địa chỉ: <https://dart.dev/overview>. Truy cập ngày 23/06/2026.
- [5] Espressif Systems, “ESP32 Series Datasheet.” [Trực tuyến]. Địa chỉ: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Truy cập ngày 23/06/2026.
- [6] OASIS Open, “MQTT Version 3.1.1 Plus Errata 01,” 2015. [Trực tuyến]. Địa chỉ: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. Truy cập ngày 23/06/2026.
- [7] Prisma Data, Inc., “Prisma ORM Documentation.” [Trực tuyến]. Địa chỉ: <https://www.prisma.io/docs/orm>. Truy cập ngày 23/06/2026.
- [8] OpenStreetMap Foundation, “About OpenStreetMap.” [Trực tuyến]. Địa chỉ: <https://www.openstreetmap.org/about>. Truy cập ngày 23/06/2026.
- [9] R. W. Sinnott, “Virtues of the Haversine,” *Sky and Telescope*, tập 68, số 2, tr. 159, 1984.
- [10] Socket.IO, “Socket.IO Documentation, Version 4.” [Trực tuyến]. Địa chỉ: <https://socket.io/docs/v4/>. Truy cập ngày 23/06/2026.
- [11] Google, “Firebase Cloud Messaging Documentation.” [Trực tuyến]. Địa chỉ: <https://firebase.google.com/docs/cloud-messaging>. Truy cập ngày 23/06/2026.
- [12] M. B. Jones, J. Bradley và N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, 2015, doi: 10.17487/RFC7519. [Trực tuyến]. Địa chỉ: <https://www.rfc-editor.org/rfc/rfc7519>.

PHỤ LỤC